

嵌入式系统及应用 实验指导书

刘 政



重庆理工大学
CHONGQING UNIVERSITY OF TECHNOLOGY

计算机科学与工程学院
物联网工程专业

2025.01

目录

1. 实验硬件平台介绍	5
1.1 MiniSTM32 开发板	6
1.1.1 硬件资源说明	8
1.1.2 MiniSTM32-V3 的 IO 引脚分配	14
1.2 miniIO 扩展板	16
1.3 miniIO 接口信号分配	18
2. 实验 1—MDK 平台使用	19
2.1 需要准备的资料及安装顺序	19
2.2 安装 MDK3.23	19
2.3 破解注册	22
2.4 STM32F1xx 芯片支持包安装	25
2.5 STLINK/V2 驱动程序安装	25
2.6 安装 CH340 驱动程序	26
2.7 新建工程	27
2.8 工程样例下载测试	39
3. 实验 2—GPIO 输出实验-LED 闪烁实验	40
3.1 STM32 IO 简介	40
3.2 硬件设计	45
3.3 软件设计	46
3.4 仿真与下载	48
4. 实验 3~4—GPIO 输出实验-蜂鸣器发声实验	51
4.1 蜂鸣器发声原理	51
4.2 miniIO 蜂鸣器电路	51
4.3 实验任务	52
4.4 演示程序分析	52
4.5 蜂鸣器播放音乐	55
4.6 实验连线	57
4.7 ISP 下载程序	57
4.8 实验步骤	60
4.9 程序流程图	60
4.10 C 语言源程序	61
4.11 思考题	61
5. 实验 5—按键输入实验	62
5.1 按键输入原理	62
5.2 实验设备与软件环境	63
5.3 实验任务	63
5.4 相关库函数分析	63
5.5 演示程序分析	64
5.6 实验步骤	65
5.7 思考题	65
6. 实验 6—外部中断实验	66
6.1 STM32 外部中断简介	66

6.2 实验设备与软件环境	67
6.3 实验任务	67
6.4 演示程序	67
6.4.1 工程结构	67
6.4.2 外中断编程步骤及演示程序	68
6.4.3 多线外中断判断	69
6.5 实验步骤	69
6.6 思考题	70
7. 实验 7—串口发送实验	71
7.1 STM32 的 USART 简介	71
7.2 实验设备与软件环境	73
7.3 实验任务	74
7.4 演示程序	74
7.4.1 工程结构	75
7.4.2 串口初始化过程	75
7.4.3 发送字符串	76
7.4.4 printf 函数使用	77
7.5 实验步骤	77
7.6 思考题	78
8. 实验 8—UART 中断接收的数学运算实验	79
8.1 实验设备与软件环境	79
8.2 查询方式接收发送	79
8.3 中断方式接收	80
8.4 简单通讯协议格式	81
8.5 实验任务	83
8.6 实验步骤	84
8.7 思考题	84
9. 实验 9~10—定时器更新中断实验	85
9.1 实验设备与软件环境	85
9.2 STM32 定时器基本原理	85
9.2.1 STM32F103 定时器概况	85
9.2.2 定时器的计数模式	86
9.2.3 通用定时器电路结构	86
9.3 更新中断程序设计	88
9.4 实验任务	90
9.5 实验步骤	90
9.6 思考题	90
10. 实验 11—PWM 实验	91
10.1 实验设备与软件环境	91
10.2 PWM 简介	91
10.3 PWM 工作过程	91
10.4 PWM 程序设计过程	92
10.5 PWM 程序	93

10.6 实验任务	94
10.7 实验步骤	94
10.8 思考题	94
11. 实验 12—AD 实验	95
11.1 实验设备与软件环境	95
11.2 ADC 简介	95
11.3 ADC 库函数初始化相关定义	96
11.4 ADC 程序设计过程	97
11.5 ADC 程序	99
11.6 实验任务	100
11.7 实验步骤	100
11.8 思考题	101
12. 实验 13—SysTick 实验	102
12.1 实验设备与软件环境	102
12.2 SysTick 简介	102
12.3 SysTick 库函数初始化相关定义	102
12.4 SysTick 毫秒定时器延时程序设计过程	103
12.5 SysTick 微秒定时器延时程序设计过程	103
12.6 实验任务	104
12.7 实验步骤	104
12.8 思考题	104

1. 实验硬件平台介绍

嵌入式实验硬件平台主要由：MiniSTM32 开发板、STLINK 调试下载器（仿真器）、miniIO 扩展板等部分组成。



图 1.1 MiniSTM32 开发板实物图

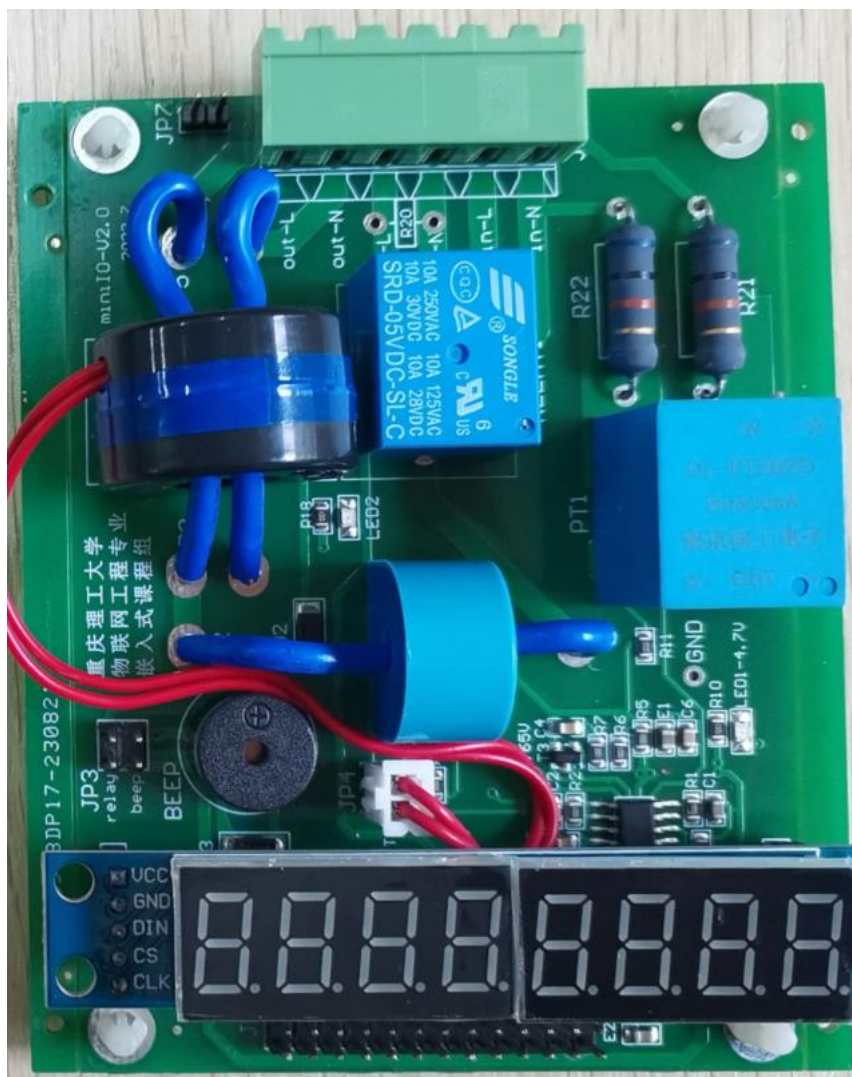


图 1.2 miniIO 扩展板实物图

1.1 MiniSTM32 开发板

ALIENTEK MiniSTM32 开发板是一款迷你型的 STM32F103 开发板，小巧而不小气，简约而不简单。该开发板自推出以来，深得广大 STM32 学习者喜爱，总销量超过 1.6W 套。目前最新版本为 V3，最新 MiniSTM32 开发板资源图如图 1.3 所示。

这款 MiniSTM32 V3 开发板，设计精良，结构小巧！板子的设计充分考虑了成本与功能这两个矛盾面，再结合实际使用的经验及 STM32 的特点，最终确定了这样的设计。总体来说是该有的都有，不该有的坚决不要，可有可无的选择性价比最高的留下。

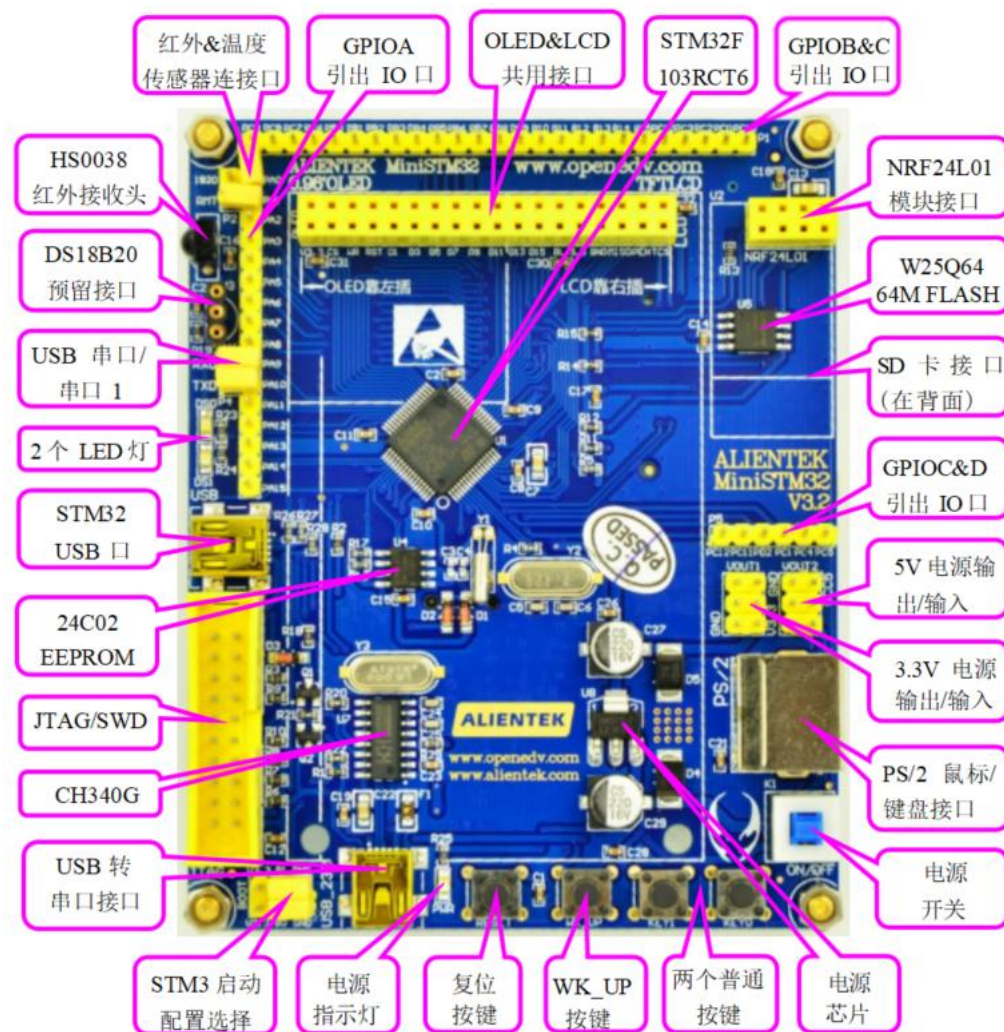


图 1.3 MiniSTM32 开发板资源图

ALIENTEK MiniSTM32 开发板板载资源如下：

- ◆ CPU: STM32F103RCT6 , LQFP64 , FLASH:256K, SRAM: 48K;
- ◆ 1 个标准的 JTAG/SWD 调试下载口
- ◆ 1 个电源指示灯 (蓝色)
- ◆ 2 个状态指示灯 (DS0: 红色, DS1: 绿色)
- ◆ 1 个红外接收头, 配备一款小巧的红外遥控器
- ◆ 1 个 IIC 接口的 EEPROM 芯片, 24C02, 容量 256 字节
- ◆ 1 个 SPI FLASH 芯片, W25Q64, 容量为 8M 字节 (即 64M bit)
- ◆ 1 个 DS18B20/DS1820 温度传感器预留接口
- ◆ 1 个标准的 2.4/2.8/3.5/4.3/7 寸 LCD 接口, 支持触摸屏
- ◆ 1 个 OLED 模块接口 (与 LCD 接口部分共用)
- ◆ 1 个 USB 串口接口, 可用于程序下载和代码调试
- ◆ 1 个 USB SLAVE 接口, 用于 USB 通信
- ◆ 1 个 SD 卡接口
- ◆ 1 个 PS/2 接口, 可外接鼠标、键盘
- ◆ 1 组 5V 电源供应/接入口
- ◆ 1 组 3.3V 电源供应/接入口

- ◆ 1 个启动模式选择配置接口
- ◆ 1 个 2.4G 无线通信接口
- ◆ 1 个 RTC 后备电池座，并带电池
- ◆ 1 个复位按钮，可用于复位 MCU 和 LCD
- ◆ 3 个功能按钮，其中 WK_UP 兼具唤醒功能
- ◆ 1 个电源开关，控制整个板的电源
- ◆ 3.3V 与 5V 电源 TVS 保护，有效防止烧坏芯片
- ◆ 独创的一键下载功能
- ◆ 除晶振占用的 IO 口外，其余所有 IO 口全部引出，其中 GPIOA 和 GPIOB 按顺序引出

从上面的板载资源可以看出，MiniSTM32 开发板的板载资源是很丰富的，加上灵活的设计，让您的开发变得更加简单。

ALIENTEK MiniSTM32 V3 开发板的特点包括：

- 1) 小巧。整个板子尺寸为 8cm*10cm*2cm(包括液晶，但不计算铜柱的高度)。
- 2) 灵活。板上除晶振外的所有的 IO 口全部引出，特别还有 GPIOA 和 GPIOB 的 IO 口是按顺序引出的，可以极大的方便大家扩展及使用，另外板载独特的一键下载功能，避免了频繁设置 B0、B1 带来的麻烦，直接在电脑上一键下载。
- 3) 资源丰富。板载十多种外设及接口，可以充分挖掘 STM32 的潜质。
- 4) 质量过硬。沉金 PCB+全新优质元器件+定制全铜镀金排针/排座+电源 TVS 保护，坚若磐石。
- 5) 人性化设计。各个接口都有丝印标注，使用起来一目了然；接口位置设计安排合理，方便顺手。资源搭配合理，物尽其用。

1.1.1 硬件资源说明

这里我们首先详细介绍 MiniSTM32 开发板的各个部分(图 1.3 中的标注部分)的硬件资源，我们将按逆时针的顺序依次介绍。

(1) HS0038 红外信号接收头

这是开发板板载的标准 38K 红外信号接收头，用于接收红外遥控器的信号，有了它，就可以用红外遥控器控制这款开发板了，也可以用来做红外解码等其他相关实验。ALIENTEK MiniSTM32 开发板标配了一个红外遥控器，其外观如图 1.4 所示：

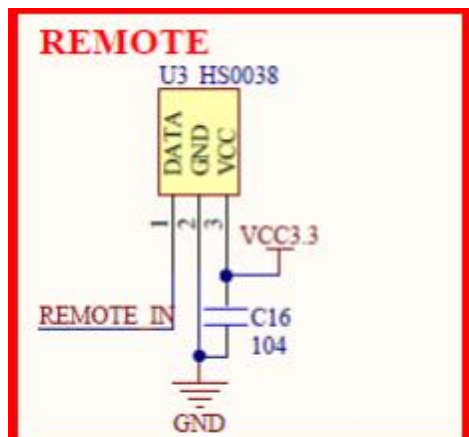


图 1.4 红外遥控原理图

(2) DS18B20 预留接口

这是开发板预留的数字温度传感器 DS18B20/DS1820 接口，采用的是镀金的圆孔母座。当 要做 DS18B20 实验的时候，直接插到这个母座上即可，很方便。DS18B20 需自备，插上就可 以用的。同样 ALIENTEK 提供了 DS18B20 的相关例程。

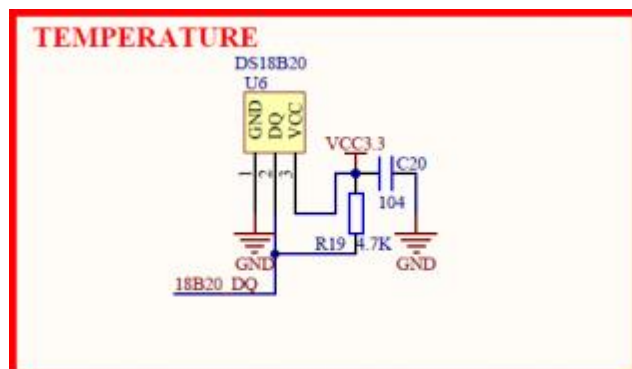


图 1.5 DS1820 接口图

(3) USB、串口、ISP 及电源

这是 USB 转串口(P4)同 STM32F103RCT6 的串口 1 进行连接的接口，标号 RXD 和 TXD 是 USB 转串口的 2 个数据口(对 CH340G 来说)，而 PA9(TXD)和 PA10(RXD)则是 STM32 的串 口 1 的两个数据口(复用功能下)。他们通过跳线帽对接，就可以和连接在一起了，从而实现 STM32 的程序下载以及串口通信。

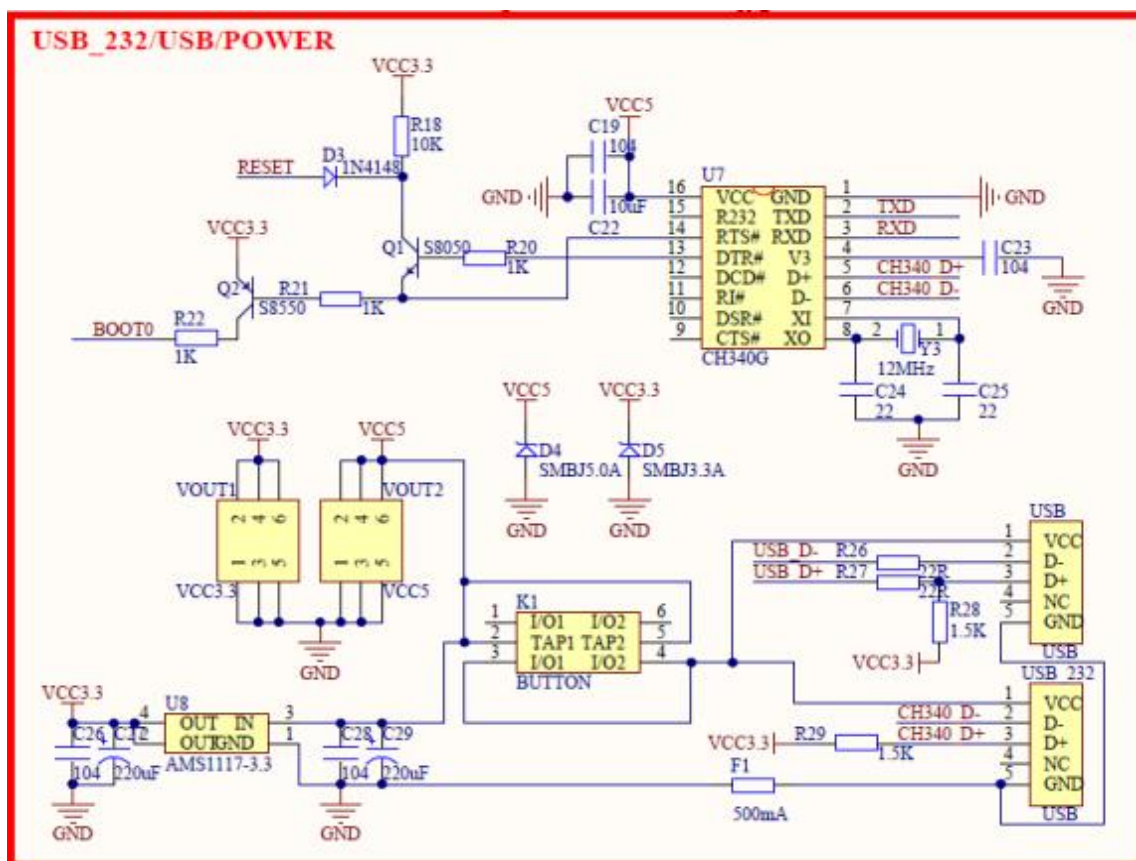


图 1.6 USB 串口、ISP、USB、电源部分原理图

这里三个部分一起介绍，ALIENTEK MiniSTM32 开发板板载了 USB 串口，并且由 USB 提 供电源，使得我们只需要一根 USB 线就可以使用 ALIENTEK MiniSTM32 开发板了，包括串口 下载代码、供电、串口通信 3 位一体。

开发板的供电部分还引出了 5V (VOUT2) 和 3.3V (VOUT1) 的排针，可以用来为外部设备提供电源或者从外部引入电源，这在很多时候是非常有用的，有时候你突然要一个 3.3V 的电源，但找半天就是没这样的电源，而我们的板子则可直接向外部提供 3.3V 电源，有了它，你就可以给外部设备提供 3.3V、5V 电源了。注意电流不能太大哦！

开发板的 USB 接口 (USB) 通过独立的 Mini USB 头引出，不和 USB 转串口 (USB_232) 共用，这样不但可以同时使用，还可以给系统提供更大的电流。

图中的 Q1 和 Q2 外加几个电阻和一个二极管就构成了开发板的一键下载电路，此电路通过 RST 和 DTR 信号来控制 BOOT0 和 RESET 信号，从而实现一键下载的功能。很多朋友问一键下载的原理，这里和大家讲一下，先说一个前提：DTR_N 和 RTS_N 的输出和 DTR/RTS 的设置是相反的。必须先记下这个前提。

一键下载电路的具体实现过程：首先，mcuisp 控制 DTR 输出低电平，则 DTR_N 输出高，然后 RTS 置高，则 RTS_N 输出低，这样 Q2 导通了，BOOT0 被拉高，即实现设置 BOOT0 为 1，同时 Q1 也会导通，STM32 的复位脚被拉低，实现复位。然后，延时 100ms 后，mcuisp 控制 DTR 为高电平，则 DTR_N 输出低电平，RTS 维持高电平，则 RTS_N 继续为低电平，此时 STM32 的复位引脚，由于 Q1 不再导通，变为高电平，STM32 结束复位，但是 BOOT0 还是维持为 1，从而进入 ISP 模式，接着 mcuisp 就可以开始连接 STM32，下载代码了，从而实现一键下载。

另外，此部分还有一个开关 K1，用来控制整个系统的供电，如果断开则整个系统的 3.3V 部分都将断电。而 5V 部分的电源还是开启的。图中 F1 为可恢复保险丝，用于保护 USB。

图中的 D4 和 D5 这两个 TVS 管，用于保护开发板，防止外部高压脉冲/静电损坏开发板上的元器件，让大家用的更加放心。

(4) 两个 LED 灯

这是开发板板载的两个 LED 灯，它们在开发板上的标号为：DS0 和 DS1。DS0 是红色的，DS1 是绿色的，主要是方便大家识别。一般的应用 2 个 LED 足够了，在调试代码的时候，使用 LED 来指示程序状态，是非常不错的一个辅助调试方法。ALIENTEK 开发板几乎每个实例都使用了 LED 来指示程序的运行状态。

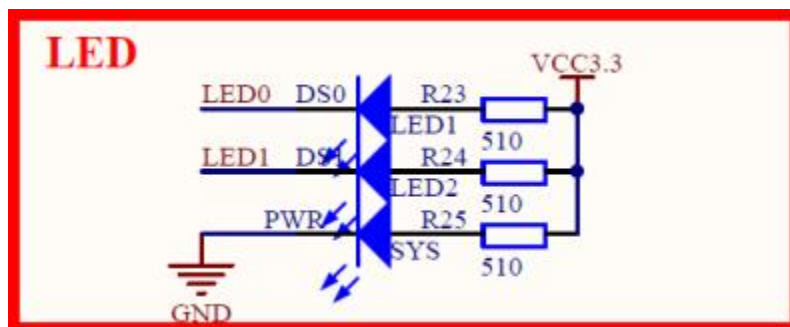


图 1.7 LED 原理图

其中 PWR 是开发板电源指示灯，为蓝色。LED0 和 LED1 分别接在 PA8 和 PD2 上，PA8 还可以通过 TIM1 的通道 1 的 PWM 输出来控制 DS0 的亮度。为了方便大家判断，我们选择了 DS0 为红色，DS1 为绿色的 LED 灯。

(5) 24C02 EEPROM

这是开发板板载的 2Kbit (256 个字节) EEPROM，型号为：24C02，用于掉电数据保存。因为 STM32 内部没有 EEPROM，所开发板外扩了 24C02，用于存储重要数据，也可以用来做 IIC 实验，及其他应用。该芯片直接挂在 STM32 的 I/O 口上。其原理图如下：

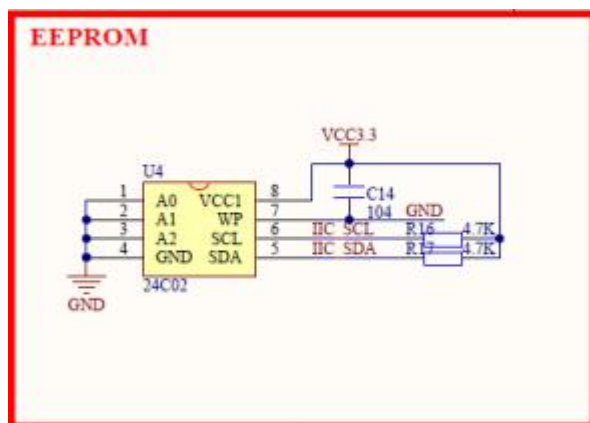


图 1.8 EEPROM 原理图

(6) JTAG/SWD

这是开发板板载的 20 针标准 JTAG 调试口，在开发板上的标号为：JTAG。该 JTAG 口直接可以和 ULINK 或者 JLINK 或者 STLINK 等调试器(仿真器)连接，同时由于 STM32 支持 SWD 调试，这个 JTAG 口也可以用 SWD 模式来连接。

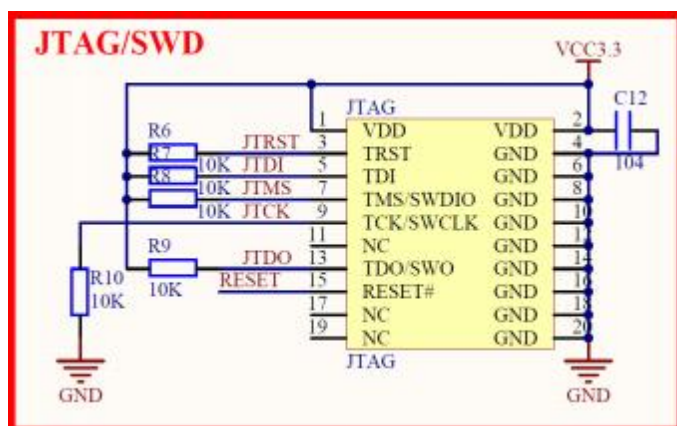


图 1.9 JTAG 原理图

用标准的 JTAG 调试，需要占用 5 个 I/O 口，很多时候，可能造成 I/O 口不够用，而用 SWD 则只需要 2 个 I/O 口，大大节约了 I/O 数量，但他们达到的效果是一样的。所以调试下载的时候，**强烈建议使用 SWD 模式!!!**

(7) USB 转串口

这是开发板板载的 USB 转串口芯片，型号为：CH340G。有了这个芯片，我们就可以实现 USB 转串口，从而实现 USB 下载代码，串口通信等，原理如图 1.5。

这是开发板板载的另外一个 MiniUSB 头(USB_232)，用于 USB 连接 CH340G 芯片，从而实现 USB 转串口，所以串口下载代码的时候，USB 一定是要接在这个口上的。同时，此 MiniUSB 接头也是开发板电源的主要提供口。

(8) STM32 启动配置选择

这是开发板板载的启动模式选择开关，在开发板上的标号为：BOOT。STM32 有 BOOT0 (B0)和 BOOT1 (B1)两个启动选择引脚，用于选择复位后 STM32 的启动模式，默认 B0，B1 都是连接在 GND 的。作为开发板，这两个是必须的。在开发板上，我们通过跳线帽选择 STM32 的启动模式。

表 1.1 BOOT0、BOOT1 启动模式表

BOOT0	BOOT1	启动模式	说明
0	X	用户闪存存储器	用户闪存存储器，也就是FLASH启动
1	0	系统存储器	系统存储器启动，用于串口下载
1	1	SRAM启动	SRAM启动，用于在SRAM中调试代码

(9) 电源指示灯

这是开发板板载的一颗蓝色的 LED，用于指示电源状态，在开发板上的标号为：PWR。在电源开启的时候(通过板上的电源开关控制)，该灯会亮，否则不亮。通过这个 LED，可以判断开发板的上电情况，开发板必须在上电的条件下(电源灯亮)，才可以正常使用。原理如图 1.6 所示。

(10) 复位按键

这是开发板板载的复位按键，用于复位 STM32，同时还具有复位液晶的功能，因为液晶模块的复位引脚和 STM32 的复位引脚是连接在一起的，此按键在开发板上的标号为：RESET。当按下该键的时候，STM32 和液晶一并被复位。

(11) WK_UP 按键

这是开发板板载的一个唤醒按键，该按键连接到 STM32 的 WAKE_UP (PA0) 引脚，可用于待机模式下的唤醒，在不使用唤醒功能的时候，也可以做为普通按键输入使用，此按键在开发板上的标号为：WK_UP。

(12) 两个普通按键

这是开发板板载的两个普通按键，可以用于人机交互的输入，这两个按键是直接连接在 STM32 的 I/O 口上的，两个按键在开发板上的标号分别为：KEY0、KEY1。

(13) 电源芯片

这是开发板的电源稳压芯片，型号为：AMS1117-3.3。因为 STM32 是 3.3V 供电的，所以我们需要将 USB 的 5V 电压转换为 3.3V，这个芯片就是将 5V 转换为 3.3V 的线性稳压芯片。

(14) 电源开关

这是开发板板载的电源开关，此开关在开发板上的标号为：K1，并标有 ON/OFF 丝印。该开关用于控制整个开发板的供电，如果切断，则整个开发板都将断电，电源指示灯(PWR)会随着此开关的状态而亮灭。

(15) PS/2 接口

这是开发板板载的一个标准 PS/2 接头，用于连接电脑鼠标和键盘等 PS/2 设备，在开发板上的标号为：PS/2。通过该接口，我们仅需要 2 个 I/O 口，就可以扩展一个键盘，所以大家不必要对板上只有 3 个按键而感到担忧。ALIENTEK 提供了标准的鼠标驱动例程，方便大家学习 PS/2 协议。原理图如下：

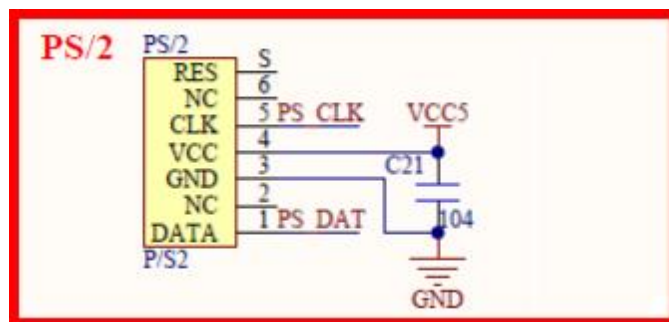


图 1.10 PS/2 接口原理图

PS_CLK 和 PS_DAT 分别接 PA15 和 PC5，PS/2 的信号线是需要外部提供上拉电阻的，这

里 PS_CLK 与 JTCK 共用一个上拉电阻,而 PS_DAT 则需要使用 STM32 内部的上拉电阻了,在使用的時候,需要注意,记得开启 PC5 的上拉电阻。

(16) 3.3V 电源输出/输入

这是开发板板载的一组 3.3V 电源输入输出排针(2*3),在开发板上的标号为: VOUT1。该排针用于给外部提供 3.3V 的电源,也可以用于从外部取 3.3V 的电源给板子供电。大家在实验的时候可能经常会为没有 3.3V 电源而苦恼不已,ALIENTEK 充分考虑到了大家需求,有了这组 3.3V 排针,您就可以很方便的拥有一个简单的 3.3V 电源(最大电流不能超过 500ma),另外板载了 3.3V TVS 管,能有效吸收高压脉冲,防止外接设备/电源可能对开发板造成的损坏。

(17) 5V 电源输出/输入

这是开发板板载的一组 5V 电源输入输出排针(2*3),在开发板上的标号为: VOUT2,用于给外部提供 5V 的电源,也可以用于从外部取 5V 的电源给板子供电。同样大家在实验的时候可能经常会为没有 5V 电源而苦恼不已,有了 ALIENTEK MiniSTM32 开发板,您就可以很方便的拥有一个简单的 5V 电源(最大电流不能超过 500ma),另外板载了 5V TVS 管,能有效吸收高压脉冲,防止外接设备/电源可能对开发板造成的损坏。

(18) GPIOC&D 引出 IO 口

这是开发板板载的 GPIOC 与 GPIOD 等 IO 口的引出排针,在开发板上的标号为: P5。我们可以用这些引出的 IO 口来连接外部模块,方便大家外接其他模块。

(19) SD 卡接口

这是开发板板载的 SD 卡接口。SD 卡作最常见的存储设备之一,是很多数码设备的存储媒介,比如数码相机、数码相机、MP5、手机、平板电脑等。我们的开发板自带了 SD 卡接口(大卡),可以用于 SD 卡实验,方便大家学习 SD 卡,TF 卡通过转接座也可以很方便的接到我们的开发板上。

有了它,开发板就相当于拥有了一个大容量的外部存储器,不但可以用来提供数据,也可以用来存储数据,使得这款开发板可以完成更多的功能。

这里要特别说明一下:该 SD 卡卡座是在开发板的背面!

(20) W25Q64 64M FLASH

这是开发板板载的一颗 FLASH 芯片,型号为 W25Q64。这颗芯片的容量为 64M bit,也就是 8M 字节。有了这颗芯片,我们就可以存储一些不常修改的数据到里面,比如字库等,从而大大节省对 STM32 内部 FLASH 的占用。关于该芯片的使用见 SPI 实验这个章节。

(21) NRF24L01 模块接口

这是开发板板载的 NRF24L01 模块接口,只要插入 NRF24L01 无线模块,我们便可以实现无线通信功能。但是提醒大家: NRF24L01 通信,至少需要 2 个模块和 2 个开发板同时工作才可以。如果只有 1 个开发板或 1 个模块,是没法实现无线通信的。

(22) GPIOB&C 引出 IO 口

这是开发板板载的 GPIOB 与 GPIOC 的引出口,该接口用于将 STM32 的 GPIOB 和部分的 GPIOC 引出,方便大家的使用,在开发板上的标号为: P1。这里 GPIOB 全部使用顺序引出的方式,尤其适合外部

总线型器件的接入。

(23) STM32F103RCT6

这是开发板的核心芯片，从 3.0 版本开始，升级到 RCT6，详细型号为：STM32F103RCT6。该芯片具有 48K SRAM、256K FLASH、2 个 16 位基本定时器、4 个 16 位通用定时器、2 个 16 位高级定时器、2 个 DMA 控制器、3 个 SPI、2 个 IIC、5 个串口、1 个 USB、1 个 CAN、3 个 12 位 ADC、1 个 12 位 DAC、1 个 SDIO 接口、51 个通用 IO 口。

(24) OLED&LCD 共用接口

这是 ALIENTEK 开发板的特色设计，一个接口，兼容两种模块。在此部分，LCD 的部分 IO 和 OLED 的 IO 共用，具体请参看后面的开发板原理图。这样我们一个接口既可以接 LCD 模块，又可以接 OLED 模块。OLED 模块使用的是 ALIENTEK 的 OLED 模块，分辨率为 128*64，模块大小为 2.6cm*2.7cm。而 LCD 模块，则可以使用 ALIENTEK 全系列的 TFTLCD 模块，包括：2.4 寸(电阻屏，240*320)、2.8 寸(电阻屏，240*320)、3.5 寸(电阻屏，320*480)、4.3 寸(电容屏，800*480)、7 寸(电容屏，800*480)。

这里特别提醒：在使用的时候，OLED 模块是靠左插的，而 LCD 模块，则是靠右插，在后续章节我们将分别介绍 OLED 模块和 LCD 模块的使用。

(25) GPIOA 引出 IO 口

这是开发板 GPIOA 的引出排针，在开发板上的标号为 P3。ALIENTEK 开发板将所有的 IO 口(除了 2 个晶振占用的 4 个 IO 口)都用排针引出来了，而且 GPIOA 和 GPIOB 是按顺序引出的。按顺序引出，在很多时候能方便大家的实验和测试，比如外接带并行控制的器件，有了并行引出的排针，那么就可以很方便的通过这些排针连接到外部设备了。

将开发板的 IO 口全部排针引出，大家就可以用来外接其他模块等，不论调试还是功能扩展都是很方便的。

(26) 红外&温度传感器接口

这是开发板板载的红外与温度传感器的连接接口，开发板虽然自带了红外接收头和 DS18B20 的接口，但是并没有将这两个器件直接挂在 IO 口上，而是通过跳线帽来连接，以防止在不使用这两器件的时候，他们对 IO 口的干扰，当然我们也可以用跳线，把 DS18B20 和红外遥控接收模块接到其他电路上使用。

1.1.2 MiniSTM32-V3 的 IO 引脚分配

为了让大家更快更好的使用 MiniSTM32 V3 开发板，这里特地将 MiniSTM32 V3 开发板主芯片：STM32F103RCT6 的 IO 资源分配做了一个总表，以便大家查阅。MiniSTM32 V3 的 IO 引脚分配总表如表 1.2 所示：

表1.2 MiniSTM32 V3 IO 资源分配表

MiniSTM32 V3 IO 资源分配表					
引脚	GPIO	连接资源		独立	连接关系说明
14	PA0	WK_UP	18B20_DQ	Y	1, 按键 KEY_UP 2, 可以做待机唤醒脚(WKUP) 3, 可以接 DS18B20 传感器接口(P2 设置)
15	PA1	NRF_IRQ	REMOTE_IN	Y	1, NRF24L01 接口 IRQ 信号 2, 接 HS0038 红外接收头(P2 设置)
16	PA2	F_CS		N	W25Q64 的片选信号
17	PA3	SD_CS		N	SD 卡接口的片选脚
20	PA4	NRF_CE		Y	NRF24L01 接口的 CE 信号
21	PA5	SPI1_SCK		N	W25Q64、SD 卡和 NRF24L01 接口的 SCK 信号
22	PA6	SPI1_MISO		N	W25Q64、SD 卡和 NRF24L01 接口的 MISO 信号
23	PA7	SPI1_MOSI		N	W25Q64、SD 卡和 NRF24L01 接口的 MOSI 信号
41	PA8	LED0		N	接 DS0 LED 灯(红色)
42	PA9	U1_TXD		Y	串口 1 TX 脚, 默认连接 CH340 的 RX(P4 设置)
43	PA10	U1_RXD		Y	串口 1 RX 脚, 默认连接 CH340 的 TX(P4 设置)
44	PA11	USB_D-		Y	接 USB D-引脚
45	PA12	USB_D+		Y	接 USB D+引脚
46	PA13	JTMS	SWDIO	N	JTAG/SWD 仿真接口, 没接任何外设 注意: 如要做普通 IO, 需先禁止 JTAG&SWD
49	PA14	JTCK	SWDCLK	N	JTAG/SWD 仿真接口, 没接任何外设 注意: 如要做普通 IO, 需先禁止 JTAG&SWD
50	PA15	JTDI	PS_CLK KEY1	N	1, JTAG 仿真口(JTDI) 2, PS/2 接口的 CLK 信号 3, 接按键 KEY1
26	PB0	LCD_D0		Y	TFTLCD 接口的 D0 脚
27	PB1	LCD_D1		Y	TFTLCD 接口的 D1 脚
28	PB2	BOOT1	LCD_D2	N	1, BOOT1, 启动选择配置引脚(仅上电时用) 2, TFTLCD 接口的 D2 脚
55	PB3	JTD0	LCD_D3	N	1, JTAG 仿真口(JTD0) 2, TFTLCD 接口的 D3 脚(使用时, 需先禁止 JTAG, 才可以当普通 IO 使用)
56	PB4	JTRST	LCD_D4	N	1, JTAG 仿真口(JTRST) 2, TFTLCD 接口的 D4 脚(使用时, 需先禁止 JTAG, 才可以当普通 IO 使用)

57	PB5	LCD_D5		Y	TFTLCD 接口的 D5 脚
58	PB6	LCD_D6		Y	TFTLCD 接口的 D6 脚
59	PB7	LCD_D7		Y	TFTLCD 接口的 D7 脚
61	PB8	LCD_D8		Y	TFTLCD 接口的 D8 脚
62	PB9	LCD_D9		Y	TFTLCD 接口的 D9 脚
29	PB10	LCD_D10		Y	TFTLCD 接口的 D10 脚
30	PB11	LCD_D11		Y	TFTLCD 接口的 D11 脚
33	PB12	LCD_D12		Y	TFTLCD 接口的 D12 脚
34	PB13	LCD_D13		Y	TFTLCD 接口的 D13 脚
35	PB14	LCD_D14		Y	TFTLCD 接口的 D14 脚
36	PB15	LCD_D15		Y	TFTLCD 接口的 D15 脚
8	PC0	T_SCK		Y	TFTLCD 接口触摸屏 SCK 信号
9	PC1	T_PEN		Y	TFTLCD 接口触摸屏 PEN 信号
10	PC2	T_MISO		Y	TFTLCD 接口触摸屏 MISO 信号
11	PC3	T_MOSI		Y	TFTLCD 接口触摸屏 MOSI 信号
24	PC4	NRF_CS		Y	NRF24L01 接口的 CS 信号
25	PC5	PS_DAT	KEY0	Y	1, PS/2 接口的 DAT 信号 2, 接按键 KEY0
37	PC6	LCD_RD		Y	TFTLCD 接口的 RD 脚
38	PC7	LCD_WR		Y	TFTLCD 接口的 WR 脚
39	PC8	LCD_RS		Y	TFTLCD 接口的 RS 脚
40	PC9	LCD_CS		Y	TFTLCD 接口的 CS 脚
51	PC10	LCD_BL		Y	TFTLCD 接口的 BL 脚
52	PC11	IIC_SDA		N	接 24C02 的 SDA
53	PC12	IIC_SCL		N	接 24C02 的 SCL
2	PC13	T_CS		Y	TFTLCD 接口触摸屏 CS 信号
3	PC14		RTC 晶振	N	接 32.768K 晶振, 不可用做 IO
4	PC15		RTC 晶振	N	接 32.768K 晶振, 不可用做 IO
5	PD0		HSE 晶振	N	接 HSE 晶振, 不可用做 IO
6	PD1		HSE 晶振	N	接 HSE 晶振, 不可用做 IO
54	PD2	LED1		N	接 DS1 LED 灯(绿色)

1.2 miniIO 扩展板

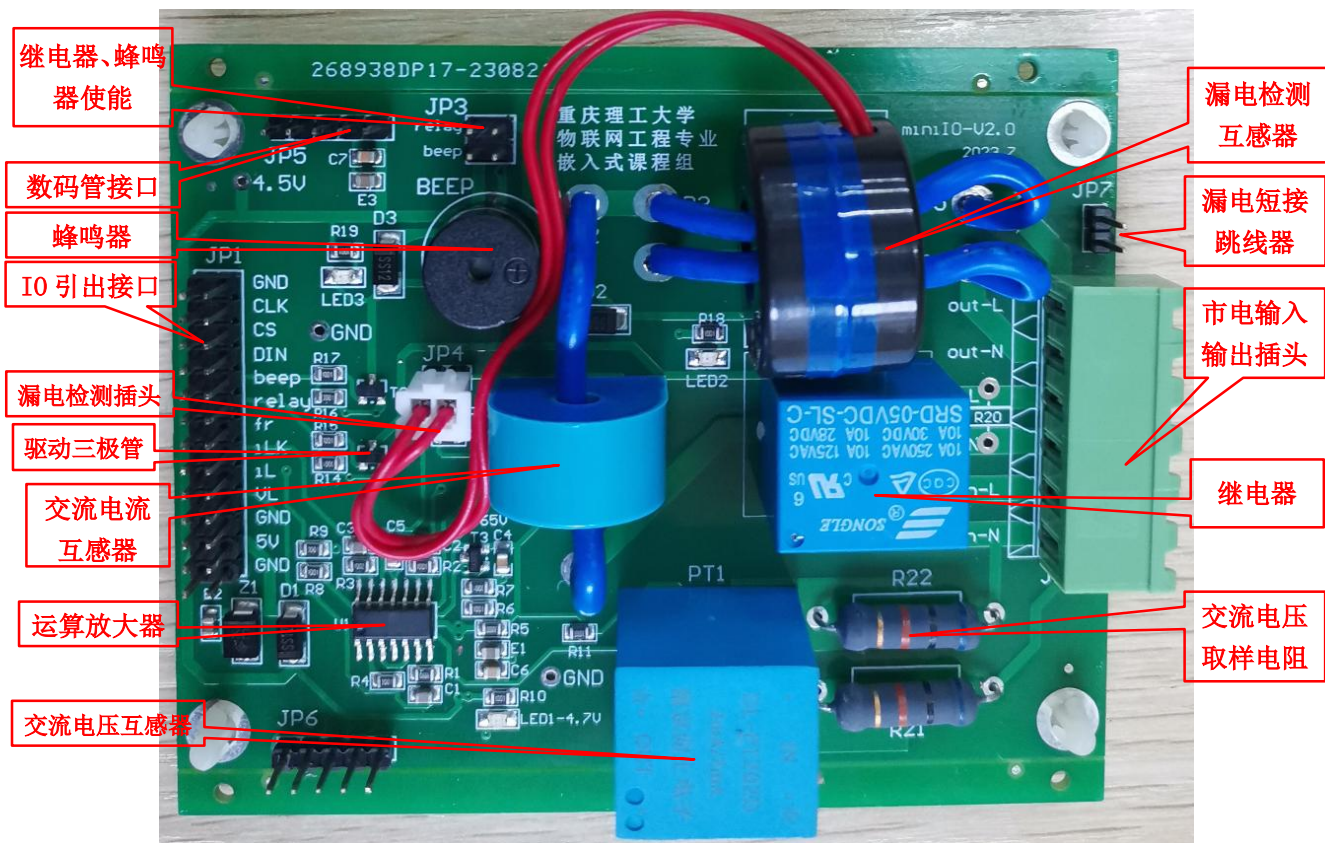


图 1.11 miniIO 扩展接口板资源图

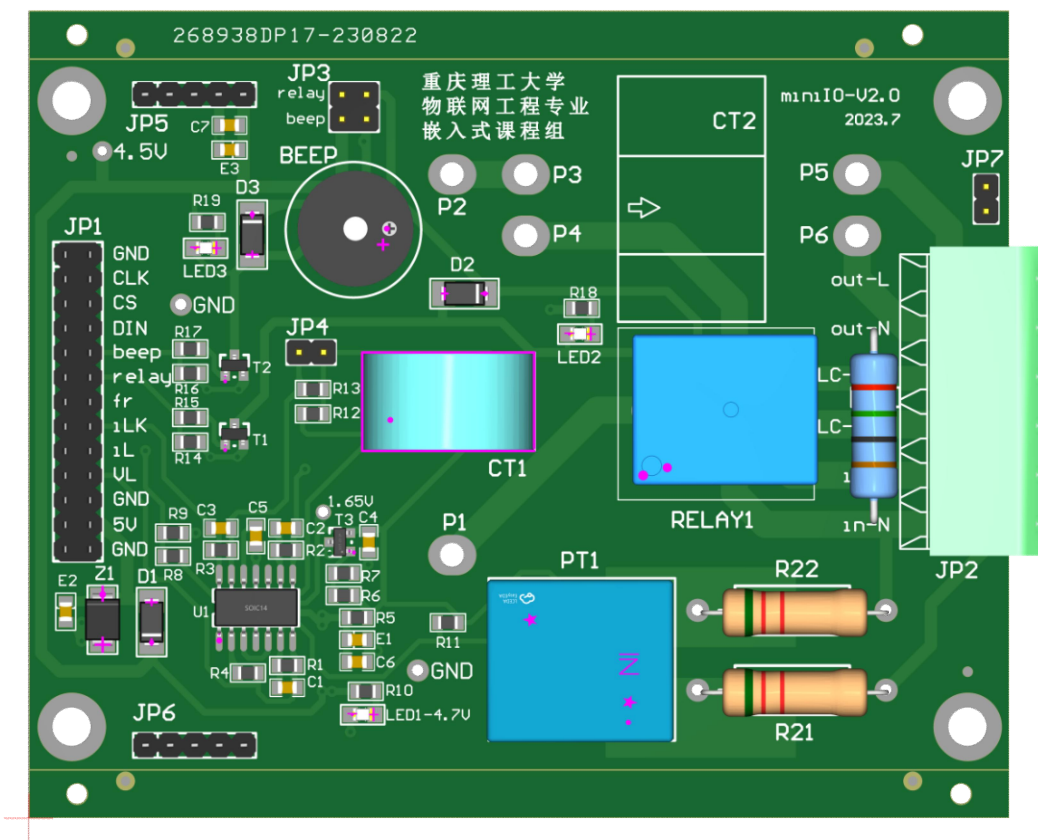


图 1.12 miniIO 元件布置补效果图

miniIO 原理图如下：

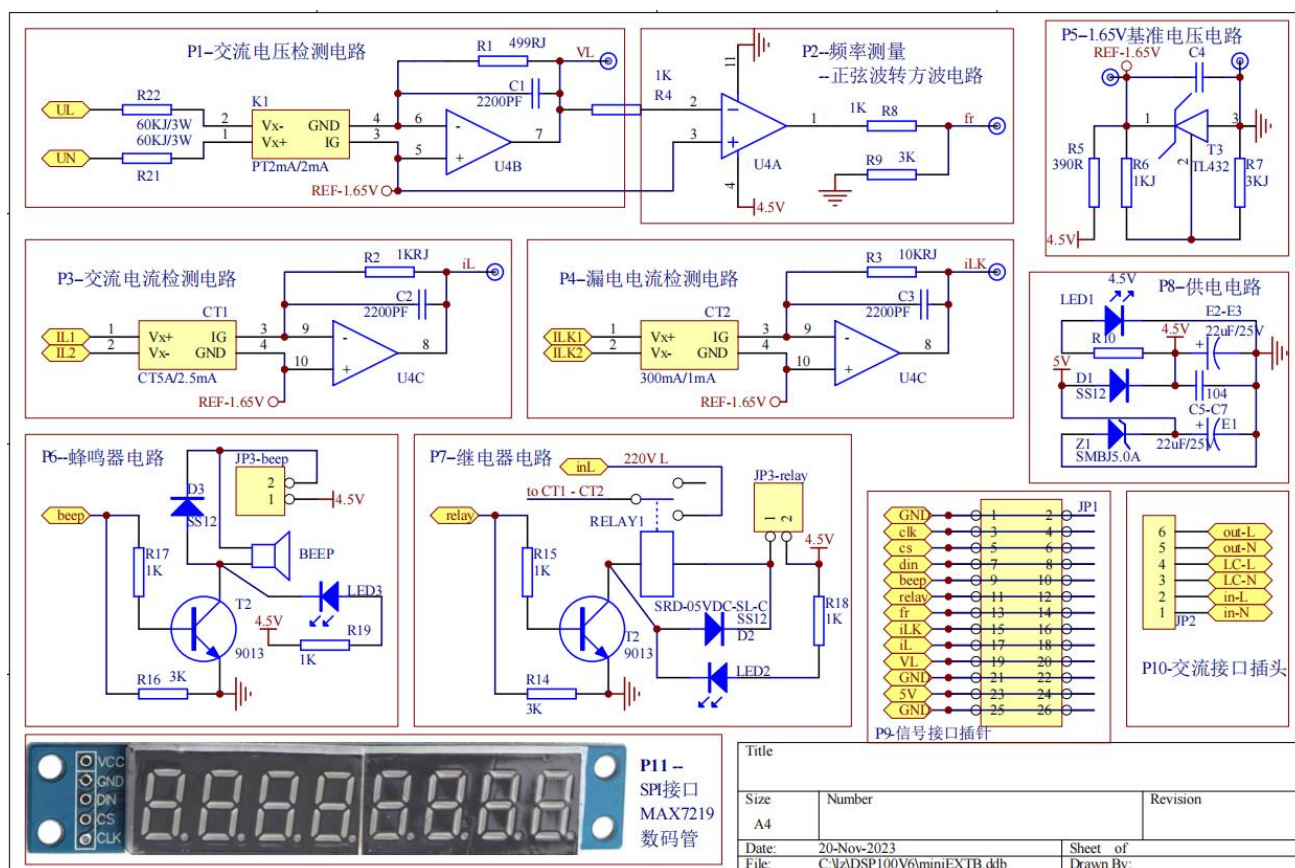


图 1.13 miniIO 原理图

1.3 miniIO 接口信号分配

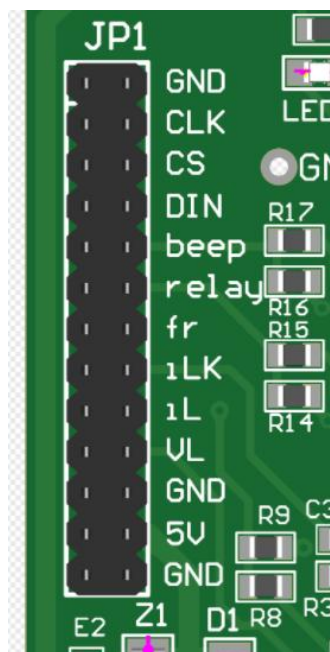


图 1.14 调理后的输入输出信号分配图

miniIO 信号分配如左面图 1.12 所示，图中信号引出为 JP1，横向 2 针为同一个信号，主要信号如下：

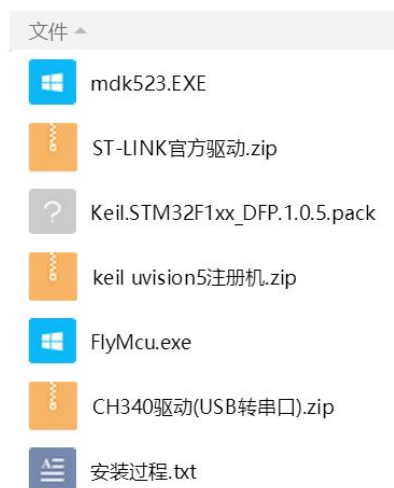
- (1) 5V，供电电源，占用 JP1-23、24 针；
- (2) GND，电源地，占用 JP1-1、2、21、22、25、26 针；
- (3) CLK，SPI 时钟，占用 JP1-3、4 针；
- (4) CS，SPI 片选，占用 JP1-5、6 针；
- (5) DIN，SPI 的 MOSI，占用 JP1-7、8 针；
- (6) beep，蜂鸣器控制，占用 JP1-9、10 针；
- (7) relay，继电器控制，占用 JP1-11、12 针，高电平有效（继电器动作）；
- (8) fr，市电交流频率方波，占用 JP1-13、14 针；
- (9) iLK，市电交流漏电电流检测，占用 JP1-15、16 针；
- (10) iL，市电交流电流检测，占用 JP1-17、18 针；
- (11) VL，市电交流电压检测，占用 JP1-19、20 针；

除 5V 供电外，其余信号都是幅值为 3.3V 信号，可兼容 5V 电压。

2. 实验 1—MDK 平台使用

这是一个独立的建立工程的资料，如果您手头没有我们 ALIENTEKSTM32 开发板的源码和资料，可以到 ALIENTEK 官方技术论坛下载，初学者搭建开发环境建立工程模版大约需要 2-3 个小时，请耐心等待手册的讲解来。也可以在开课前的班级学习群的 soft 目录下载，所需文件清单如下：

群文件 > soft

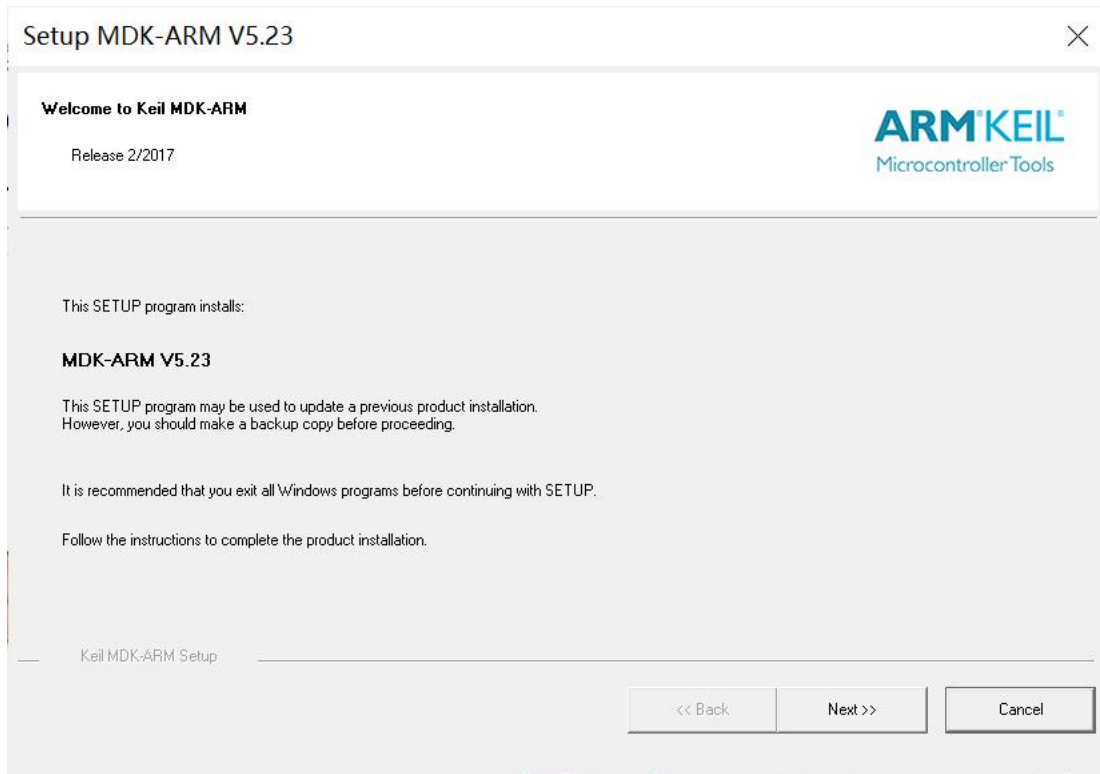


2.1 需要准备的资料及安装顺序

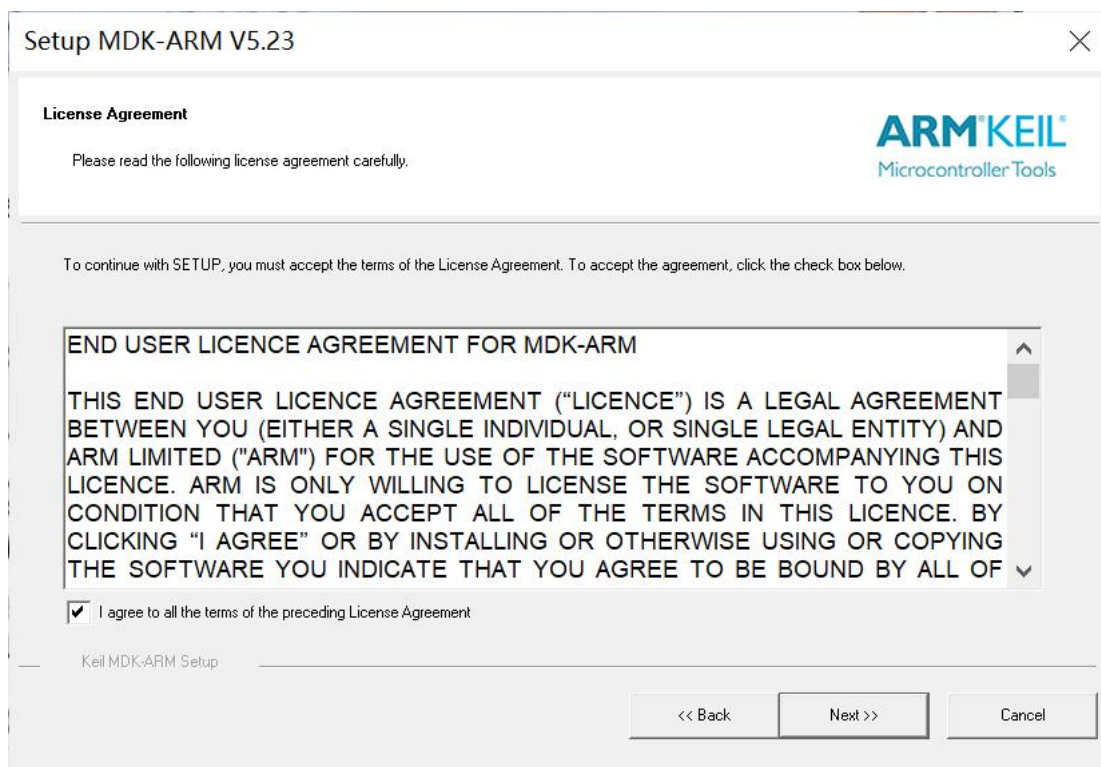
- (1) MDK323.EXE, MDK 安装文件
- (2) KEIL_Lic.exe, keiluvision5 注册机
- (3) Keil.STM32F1xx_DFP.1.0.5.pack, 芯片支持包 (STM32F10X)
- (4) STLINK/V2 调试器驱动程序安装
- (5) 板载 USB-SERIAL CH340 串口驱动程序安装
- (6) ISP 下载程序 FlyMcu.exe 绿色免安装版
- (7) STM32F10x_StdPeriph_Lib_V3.5.0, ST 固件库完整版 (工程模板)

2.2 安装 MDK3.23

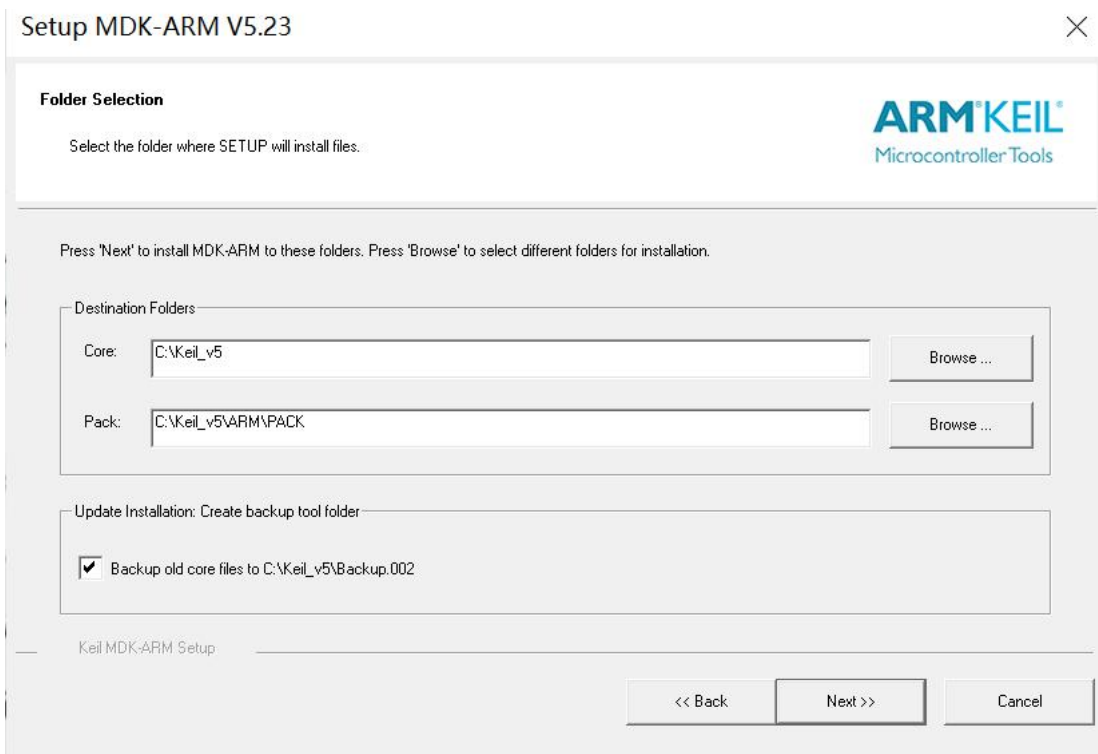
- (1). 找到 MDK 的安装文件并点击安装  mdk523.EXE
- (2). 点击之后出现界面，选择 Next



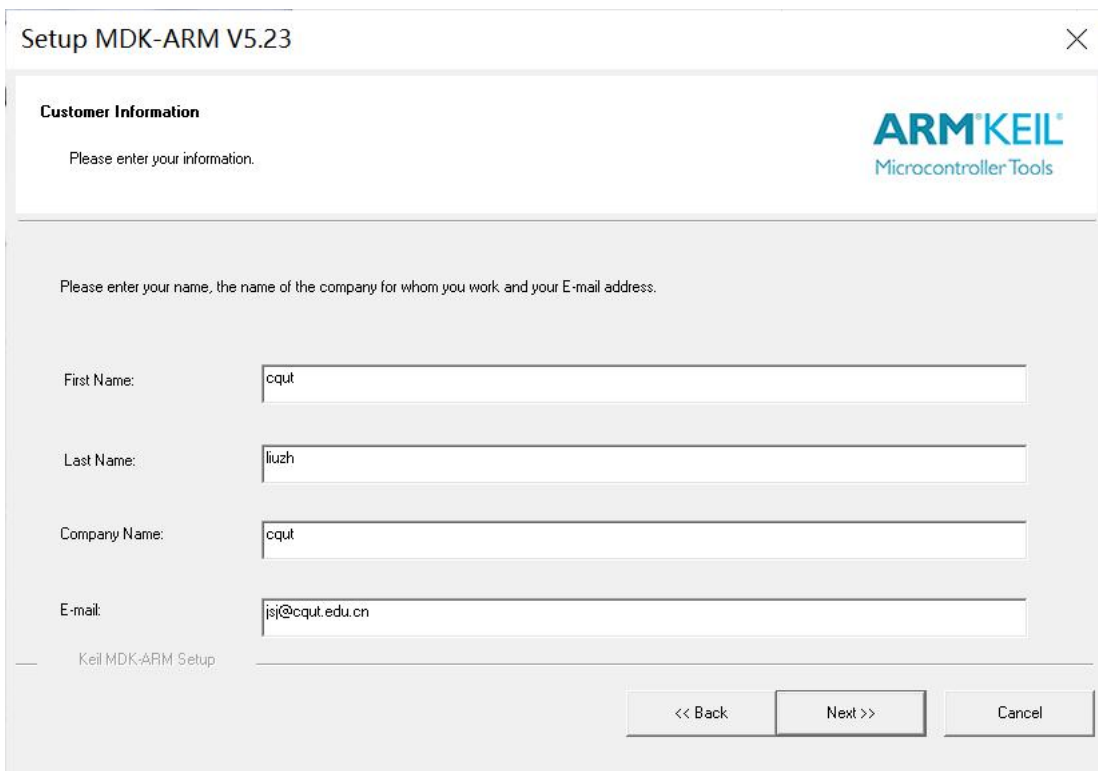
(3). 选择”I Agree...Licenceagreement”同意协议：



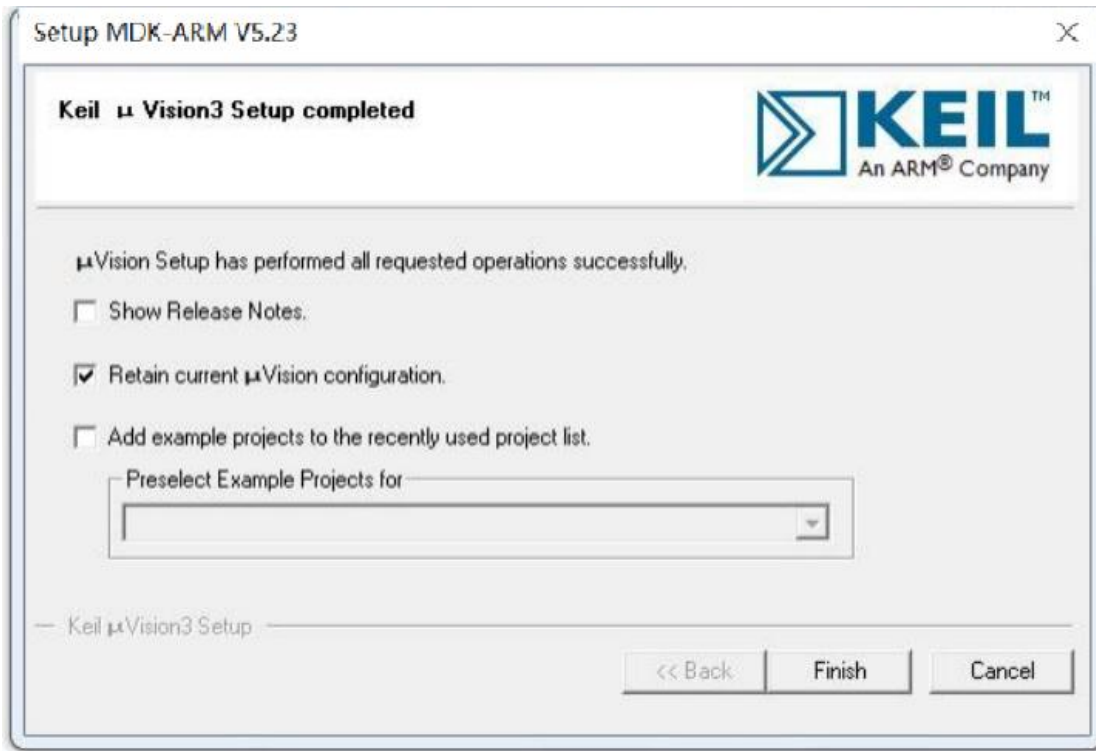
(4). 选择安装目录，这里用户自行选择安装的目录后，点击 Next 即可：



(5). 随便输入邮箱之类的信息即可，点击 Next 开始安装：



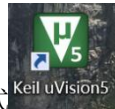
(6). 按图配置，然后点击 Finish, 左面会出现 keil 快捷图标。



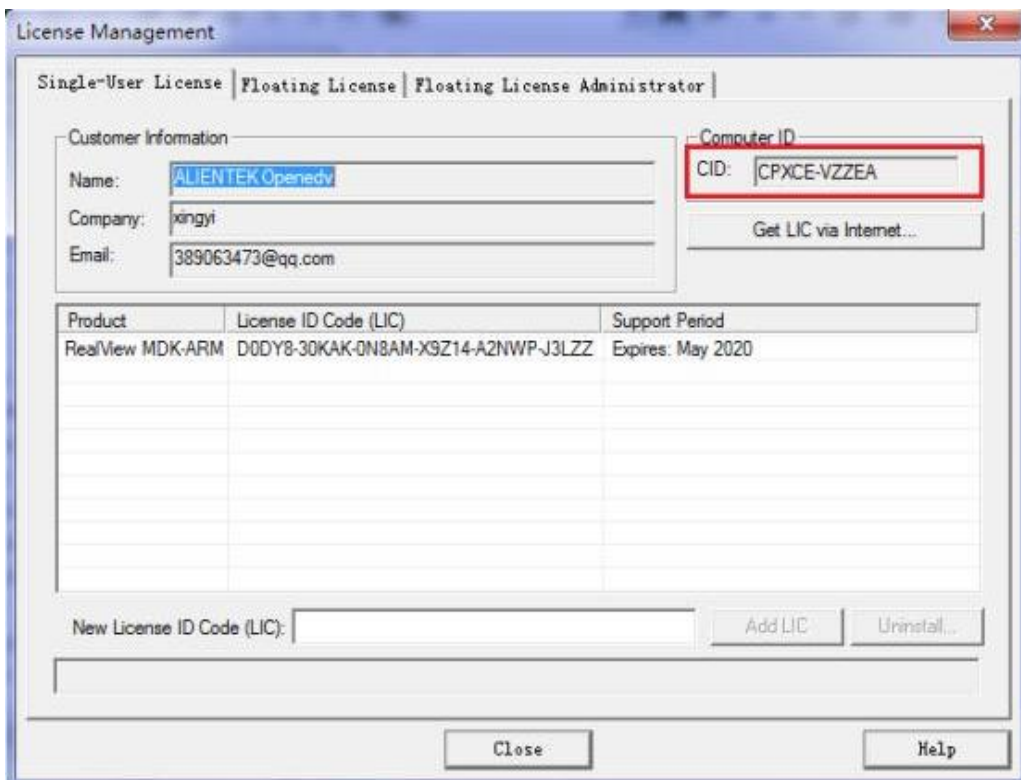
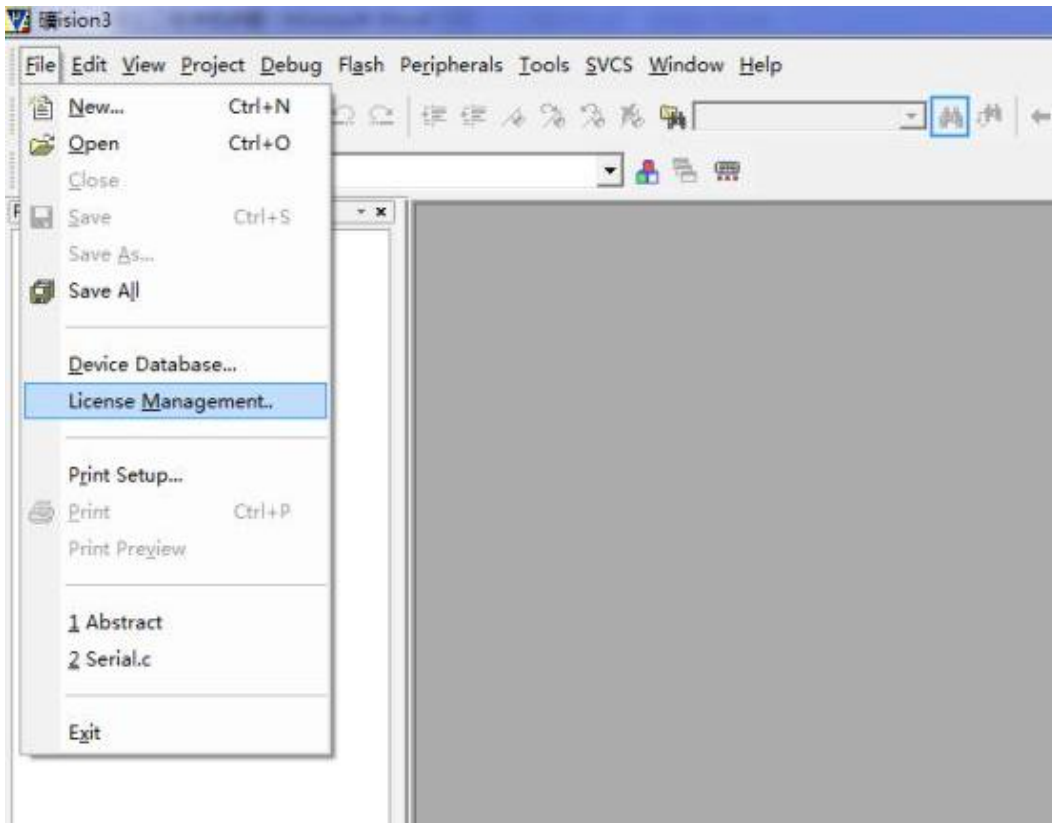
2.3 破解注册

License（如果不破解只能支持32K的代码），步骤简介：

在MDK针对每台机会会有一个CID，copy这个CID到注册机相应位置，点击注册机Genertate按钮，生成LicenseKey，然后再将这个LicenseKey添加到MDK里面去注册。



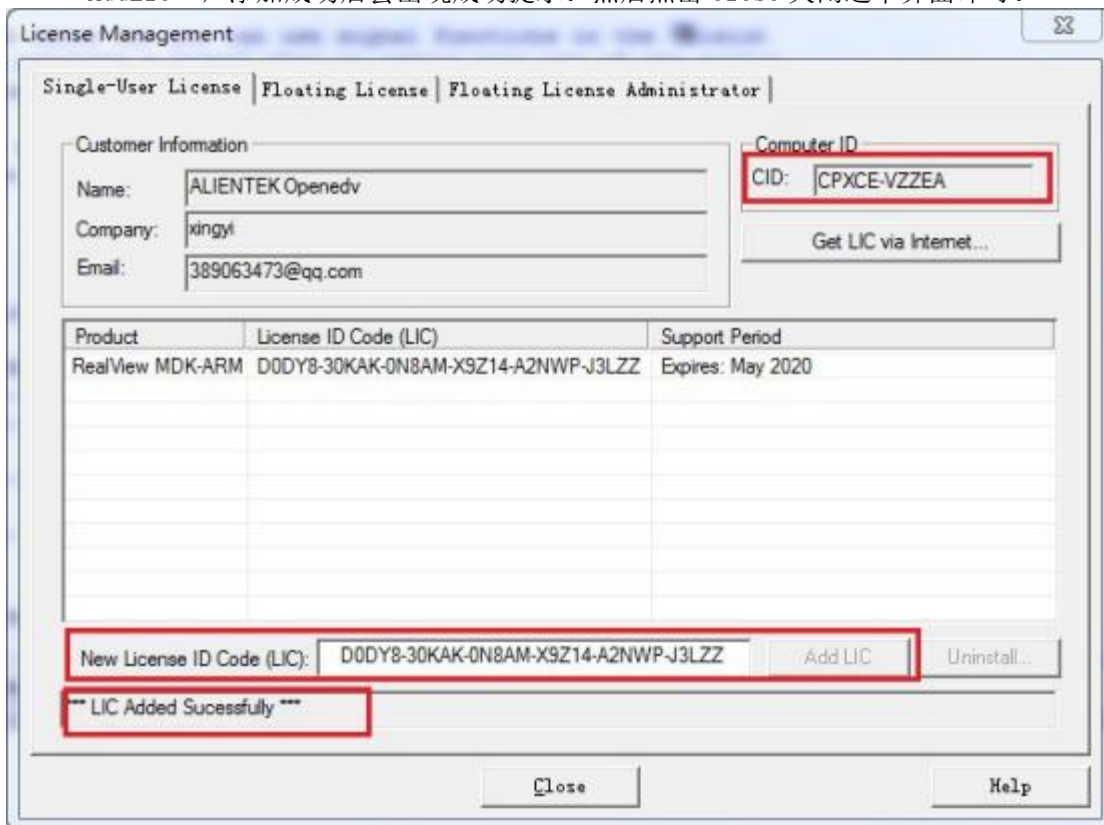
- (1) 右键点击桌面的MDK快捷方式，选择“以管理员身份运行”，因为注册license需要管理员权限。
- (2) 点击：File->LicenseManagement, 弹出一个LicenseManagement界面, copy界面中的(CID)：



- (3) 打开注册机 KEIL_Lic.exe，该文件从压缩文件 keil uvision5注册机.zip 中解压缩得到。
- (4) 出现注册界面，粘贴刚才 copy 的 cid 到 CID 一栏，然后 Target 选择 ARM

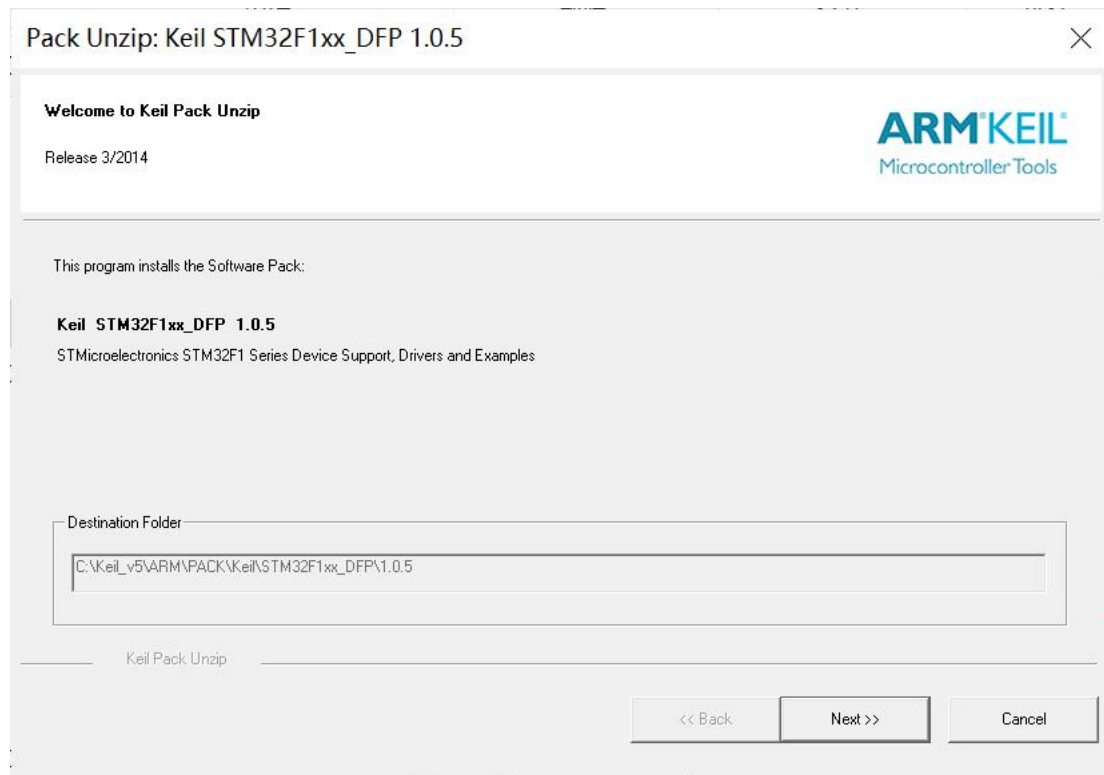


- (5) 选择好之后点击“Generate”，下面的空白栏会生成一个 LicenseKey，类似：D0DY8-30KAK-0N8AM-X9Z14-A2NWP-J3LZZ，copy 这个 license。
- (6) 将这个 LicenseKey 粘贴到 Keil 的 LicenseManagement 界面的 NewLicenseIdCode 一栏，然后点击“AddLIC”，添加成功后会出现成功提示。然后点击 Close 关闭这个界面即可。



2.4 STM32F1xx 芯片支持包安装

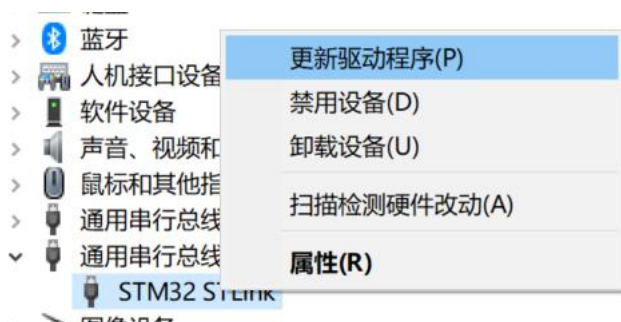
双击文件  Keil.STM32F1xx_DFP.1.0.5.pack，开始安装，选择默认选项，直至安装成功。



2.5 STLINK/V2 驱动程序安装

STLINK/V2 调试下载器，我经常简称仿真器，兼容 JTAG 和 SWD 调试方式。STLINK/V2 实际上是一种 OCD 设备，而不是真正意义上的仿真器。

鼠标右键点击  ST-LINK官方驱动.zip，选择  解压到 ST-LINK官方驱动(E) 解压到  ST-LINK官方驱动 目录，然后打开电脑设备管理器，找到  通用串行总线设备 鼠标右键点击 STM32 STLINK，如下图



点击“更新驱动程序”，选择最后一个选项，手动查找并安装驱动程序：

← 更新驱动程序 - STM32 STLink

你要如何搜索驱动程序?

→ 自动搜索驱动程序(S)

Windows 将在你的计算机中搜索最佳可用驱动程序，并将其安装在你的设备上。

→ 浏览我的电脑以查找驱动程序(R)

手动查找并安装驱动程序。

然后找到刚才解压缩到的目录并点击下一步完成驱动程序安装。

← 更新驱动程序 - STM32 STLink

浏览计算机上的驱动程序

在以下位置搜索驱动程序:

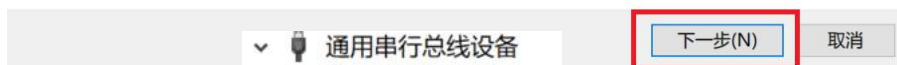
E:\教学\STM32所需软件\ST-LINK官方驱动

浏览(R)...

☒ 包括子文件夹(I)

→ 让我从计算机上的可用驱动程序列表中选择(L)

此列表将显示与该设备兼容的可用驱动程序，以及与该设备属于同一类别的所有驱动程序。



当安装成功后，显示
号说明驱动程序或硬件或连接出现问题，需检查。

如果图标上标有感叹号、问号、叉号等红色或黄色符

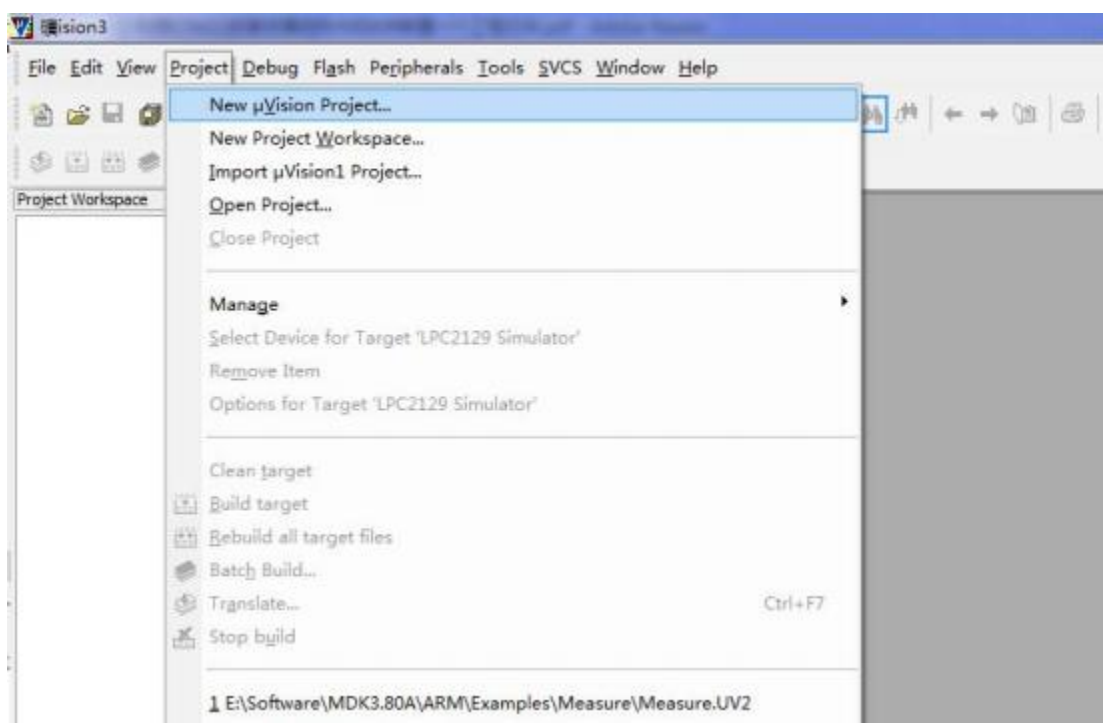
2.6 安装 CH340 驱动程序

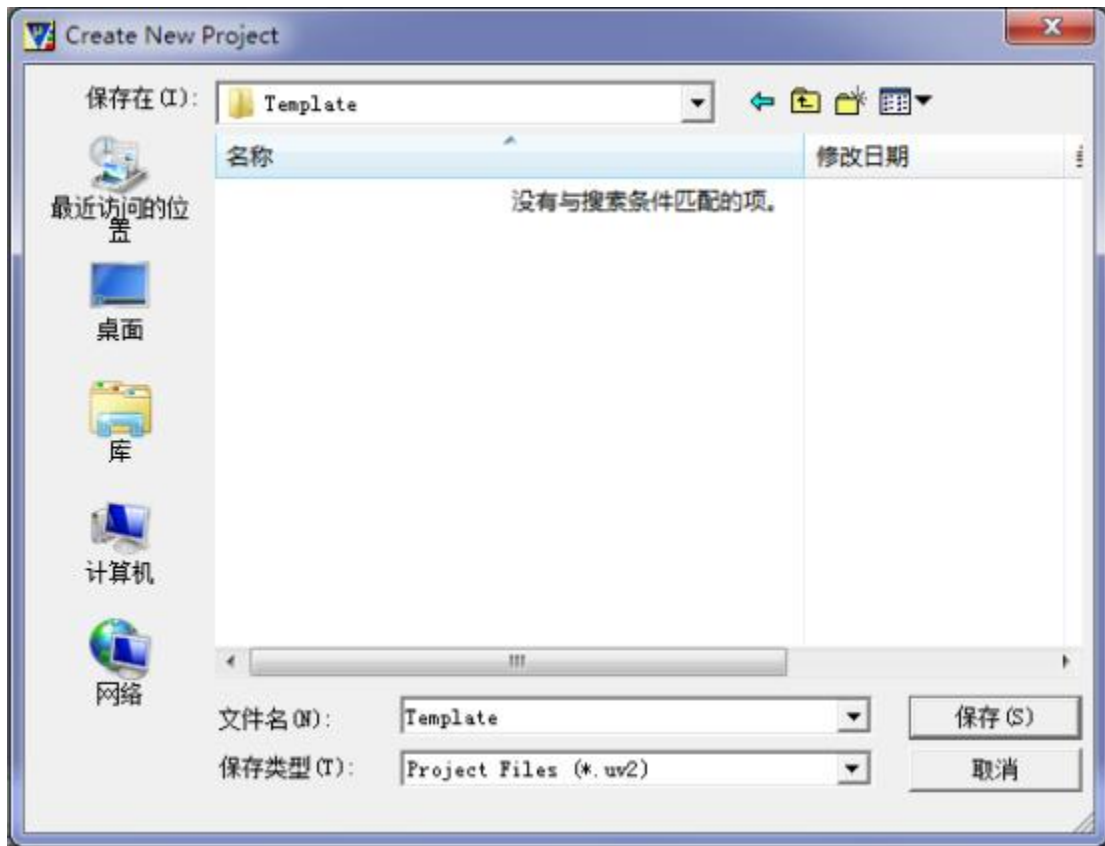
挤压或直接运行压缩文件 CH340驱动(USB转串口).zip 中的 SETUP.EXE 当安装成功后连接实验板并通电后在 PC 电脑设备管理器会出现以下设备图标，如果图标上标有感叹号、问号、叉号等红色或黄色符号说明驱动程序或硬件或连接出现问题，需检查。

- ▼ 端口 (COM 和 LPT)
 - ELTIMA Virtual Serial Port (COM7->COM8)
 - ELTIMA Virtual Serial Port (COM8->COM7)
 - USB-SERIAL CH340 (COM1)**

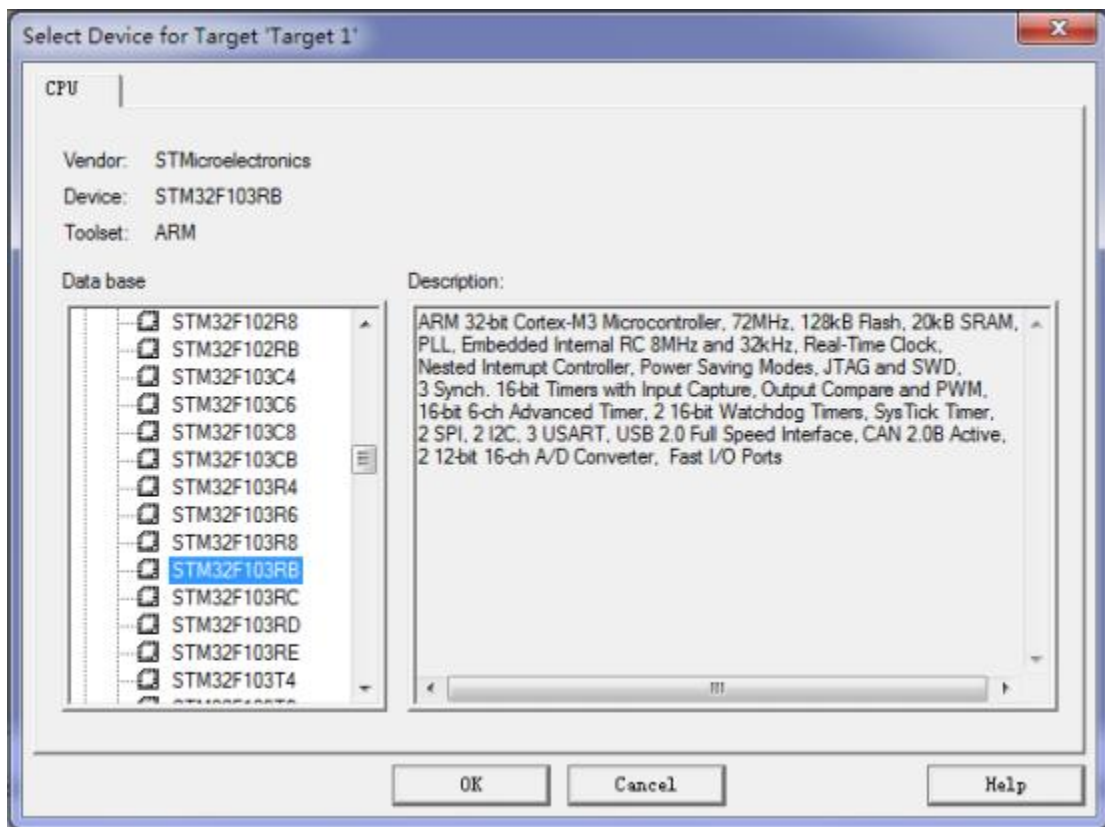
2.7 新建工程

- (1) 回到 MDK 主界面, 可以看到工程中有一个默认的工程, 点击这个工程名字, 然后选择菜单 Project->CloseProject, 就关闭掉这个工程了! 这样整个 MDK 就是一个空的了, 接下来我们将建立我们的工程模版。
- (2) 在建立工程之前, 我们建议用户在电脑的某个目录下面建立一个文件夹, 后面所建立的工程都可以放在这个文件夹下面, 这里我们建立一个文件夹为:STM32-Projects。
- (3) 点击 Keil 的菜单: Project ->NewUvisionProject, 然后将目录定位到刚才建立的文件夹 STM32-Projecst 之下, 在这个目录下面建立子文件夹 Tempalte, 然后定位到 Template 目录下面, 我们的工程文件就都保存到 Template 文件夹下面。工程命名为 Template, 点击保存。

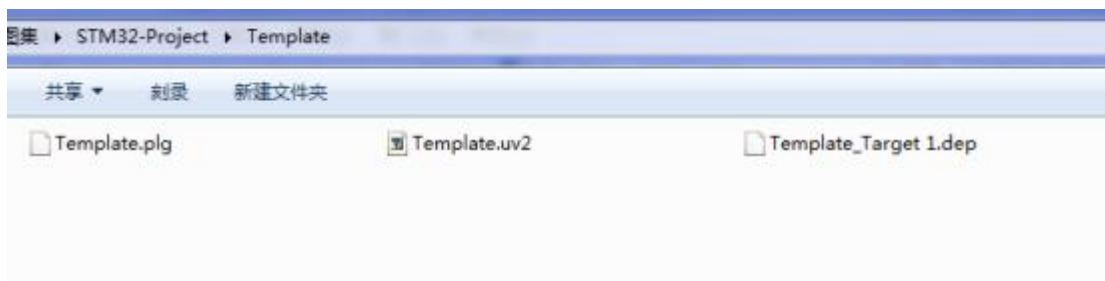




- (4) 接下来会出现一个选择Device的界面，就是选择我们的芯片型号，这里我们定位到STMicroelectronics下面的STM32F103RB(针对我们的mini板子是这个型号，如果是其他芯片，请选择对应的型号即可)。



- (5) 弹出对话框“CopySTM32StartupCodetoproject...”，询问是否添加启动代码到我们的工程中，这里我们选择“否”，因为我们使用的ST固件库文件已经包含了启动文件。
- (6) 可以看到工程建立了，我们回到Template目录下面，可以看到只有三个文件：

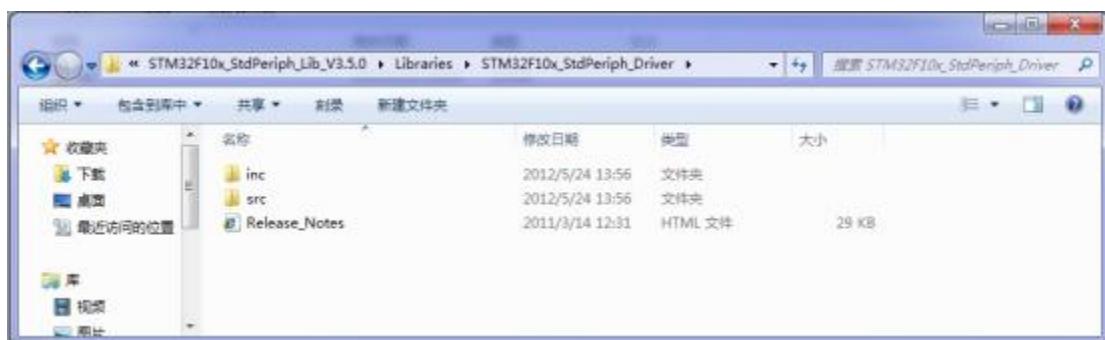


- (7) 接下来，我们在Template工程目录下面，新建3个文件夹CORE, USER, STM32F10x_FWLib。USER用来放我们主函数文件main.c,以及其他包括system_stm32f10x.c等等，CORE用来存放启动文件等，STM32F10x_FWLib文件夹顾名思义用来存放ST官方提供的库函数源码文件。



- (8) 下面我们要将官方的固件库包里的源码文件复制到我们的工程目录文件夹下面。

打开官方固件库包，定位到我们之前准备好的固件库包的目录STM32F10x_StdPeriph_Lib_V3.5.0\Libraries\STM32F10x_StdPeriph_Driver下面，将目录下面的src, inc文件夹copy到我们刚才建立的STM32F10x_FWLib文件夹下面。src存放的是固件库的.c文件，inc存放的是对应的.h文件，您不妨打开这两个文件目录过目一下里面的文件，每个外设对应一个.c文件和一个.h头文件。

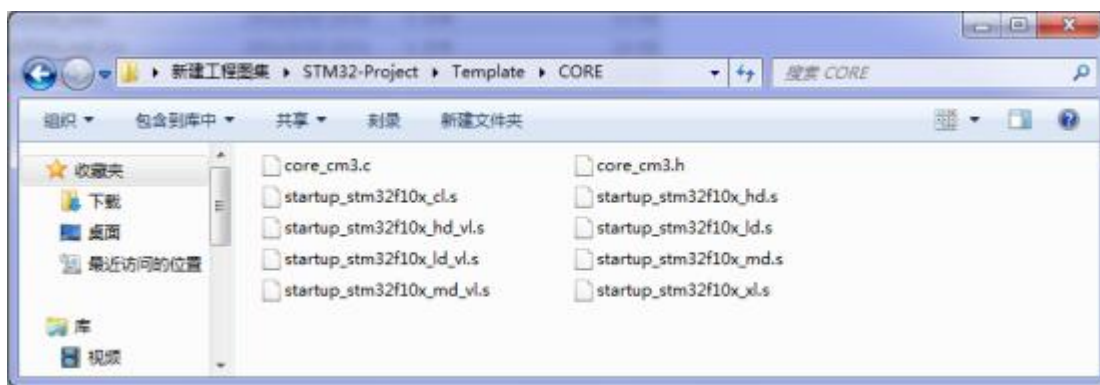




(9) 下面我们要将固件库包里面相关的启动文件复制到我们的工程目录 CORE 之下。

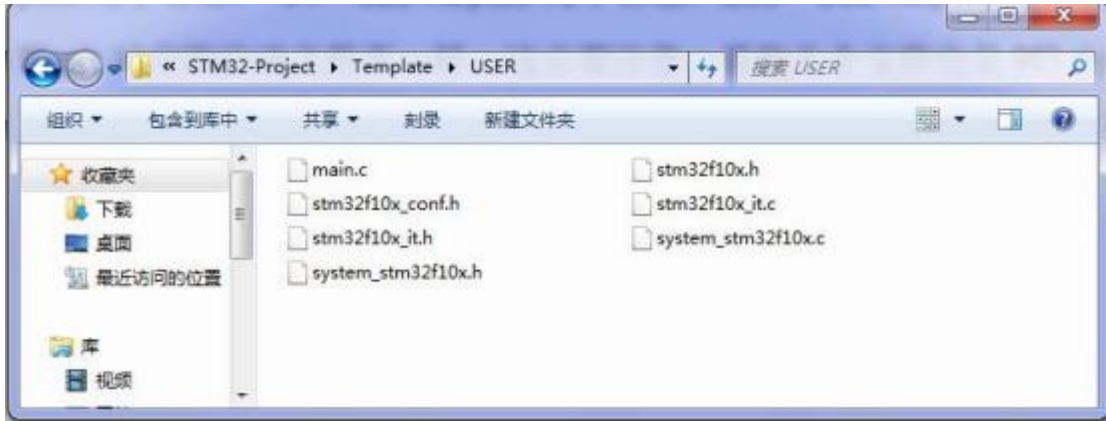
打开官方固件库包，定位到目录 STM32F10x_StdPeriph_Lib_V3.5.0\Libraries\CMSIS\CM3\CoreSupport 下面，将文件 core_cm3.c 和 文件 core_cm3.h 复制到 CORE 下面去。然后定位到目录 STM32F10x_StdPeriph_Lib_V3.5.0\Libraries\CMSIS\CM3\DeviceSupport\ST\STM32F10x\startup\arm 下面，将里面所有的文件同样复制到 CORE 下面。这里我们解释一下，其实我们只用到 arm 目录下面的 startup_stm32f10x_md.s 文件，这个文件是针对中等容量芯片的启动文件。其他两个主要的为 startup_stm32f10x_ld.s 为小容量，startup_stm32f10x_hs.c 为大容量芯片的启动文件。这里 copy 进来是方便其他开发者使用小容量或者大容量芯片的用户。

现在看看我们的 CORE 文件夹下面的文件：

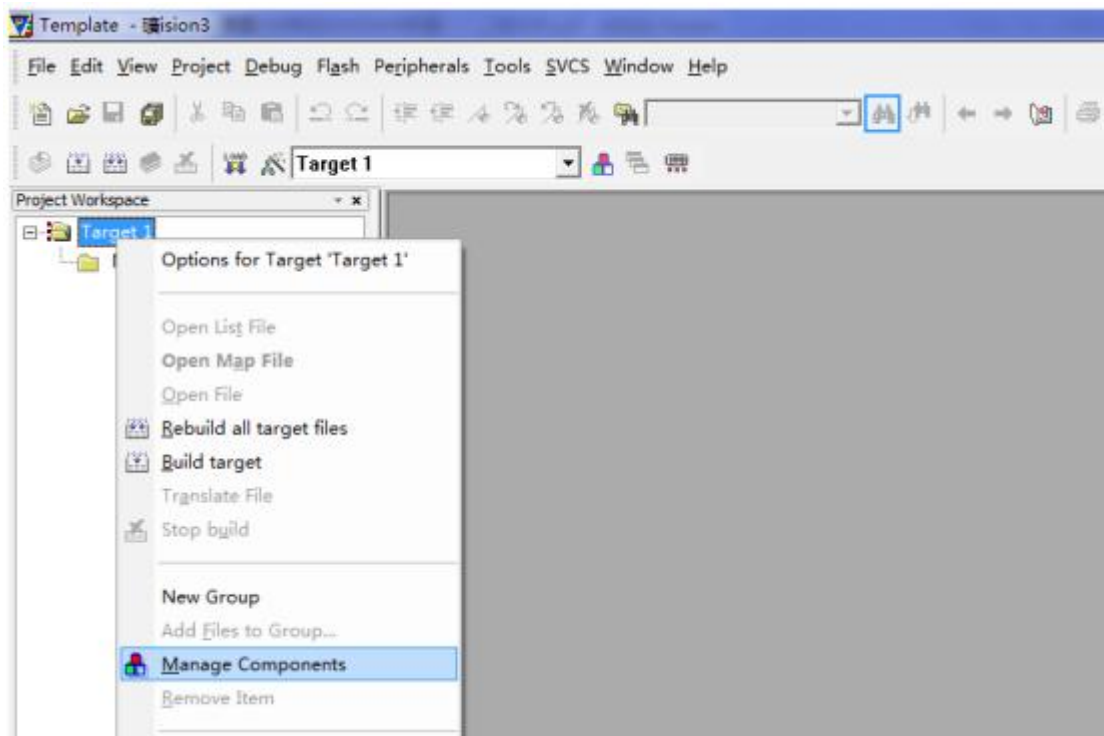


(10) 定位到目录：

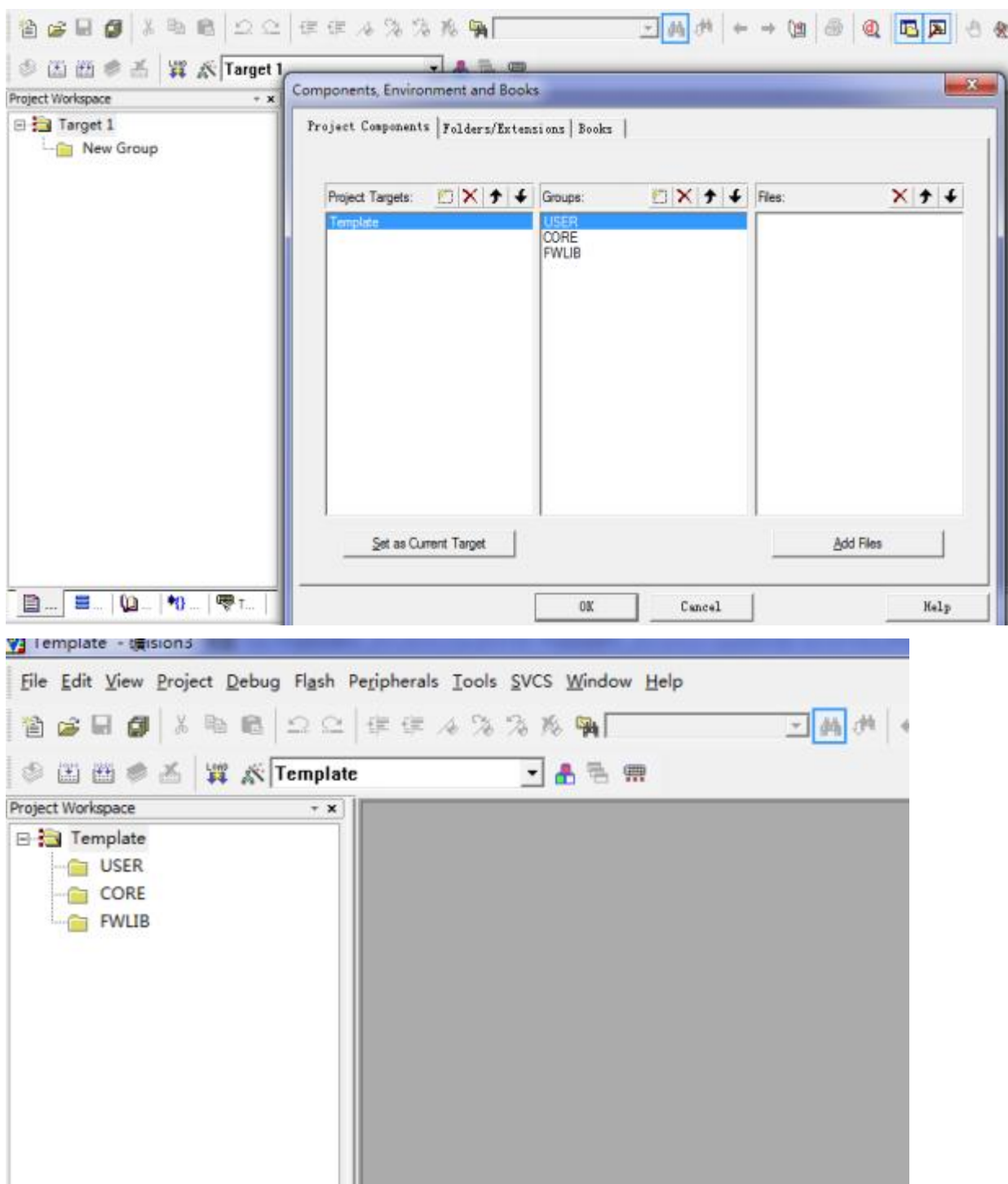
STM32F10x_StdPeriph_Lib_V3.5.0\Libraries\CMSIS\CM3\DeviceSupport\ST\STM32F10x 将里面的三个文件 stm32f10x.h， system_stm32f10x.c， system_stm32f10x.h， 复制到我们的 USER 目录之下。然后将 STM32F10x_StdPeriph_Lib_V3.5.0\Project\STM32F10x_StdPeriph_Template 下面的 4 个文件 main.c, stm32f10x_conf.h, stm32f10x_it.c, stm32f10x_it.h 复制到 USER 目录下面。



(11) 前面 10 个步骤，我们将需要的固件库相关文件复制到了我们的工程目录下面，下面我们将这些文件加入我们的工程中去。右键点击 Target1，选择 ManageComponents



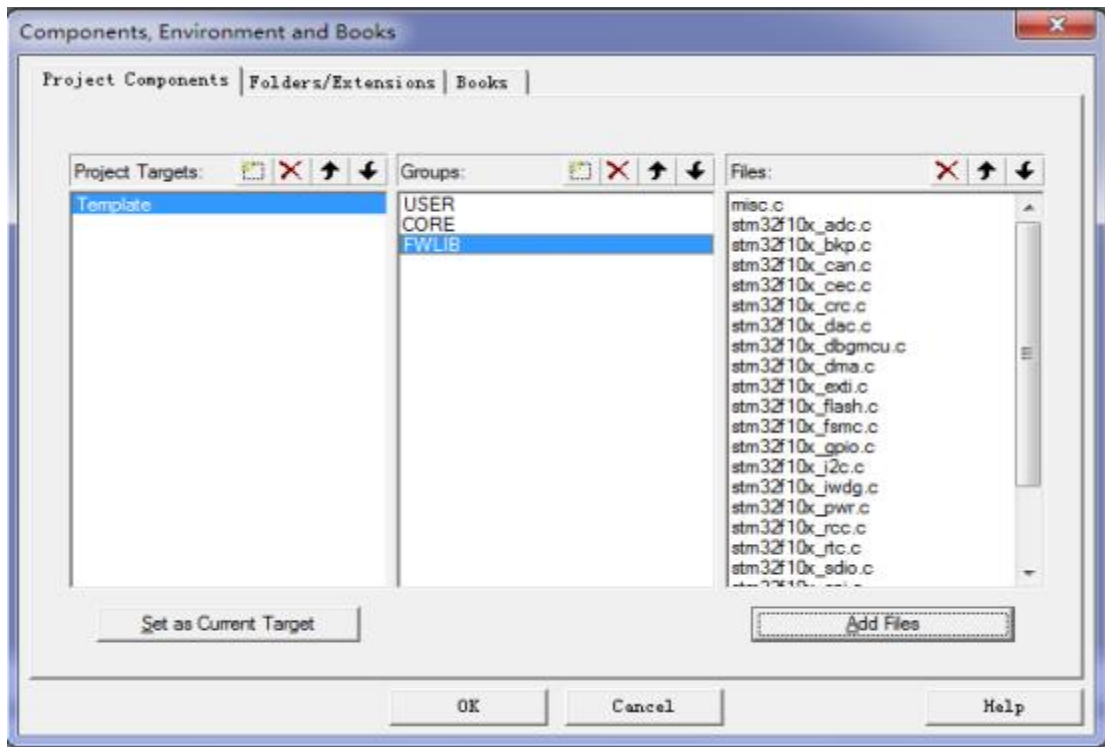
(12) ProjectTargets 一栏，我们将 Target 名字修改为 Template，然后在 Groups 一栏删掉一个，建立三个 Groups：USER, CORE, FWLIB. 点击 OK. 可以看到我们的 Target 名字以及 Groups 情况。



(13) 下面我们往 Group 里面添加我们需要的文件

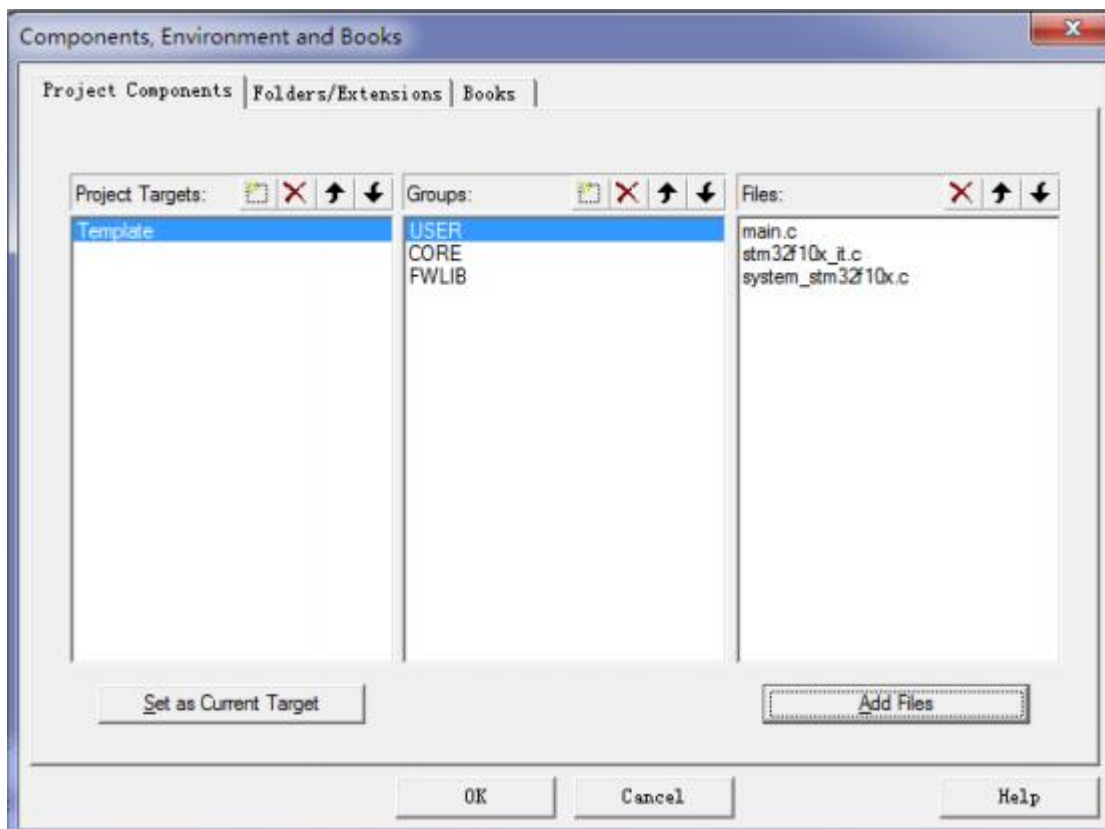
我们按照步骤 12 的方法，右键点击点击 Template，选择选择 ManageComponents. 然后选择需要添加文件的 Group，这里第一步我们选择 FWLIB，然后点击右边的 AddFiles，定位到我们刚才建立的目录 STM32F10x_FWLib/src 下面，将里面所有的文件选中(Ctrl+A)，然后点击 Add，然后 Close. 可以看到 Files 列表下面包含我们添加的文件。

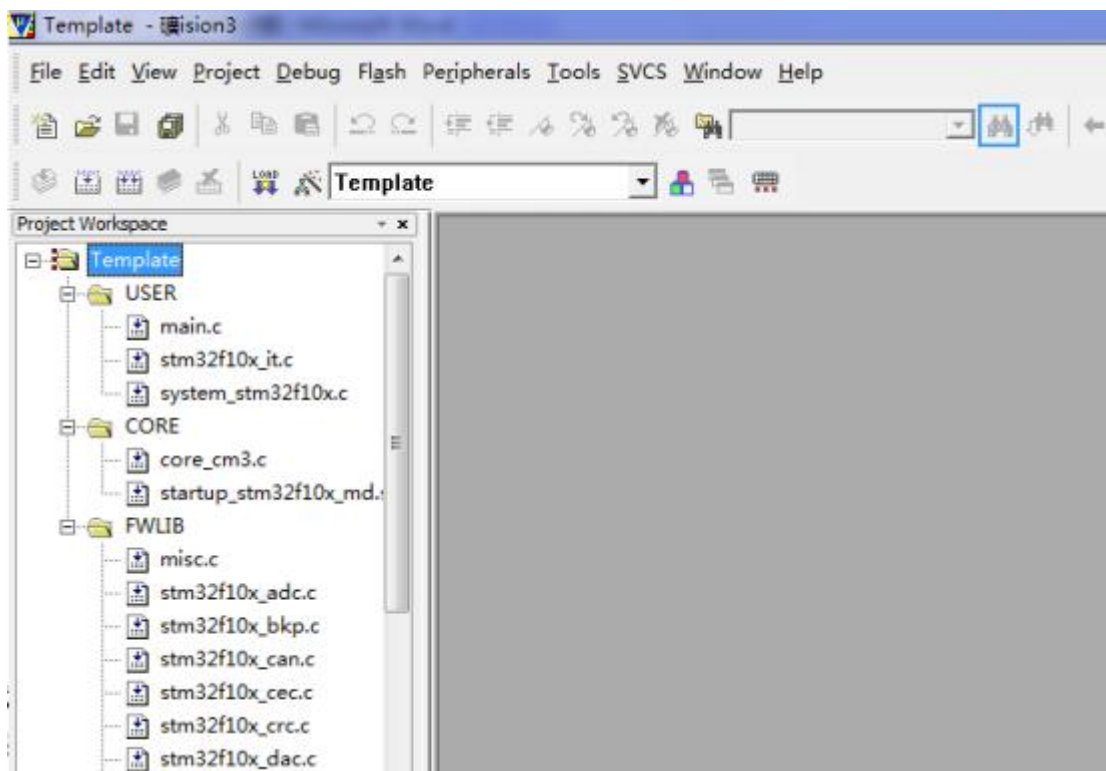
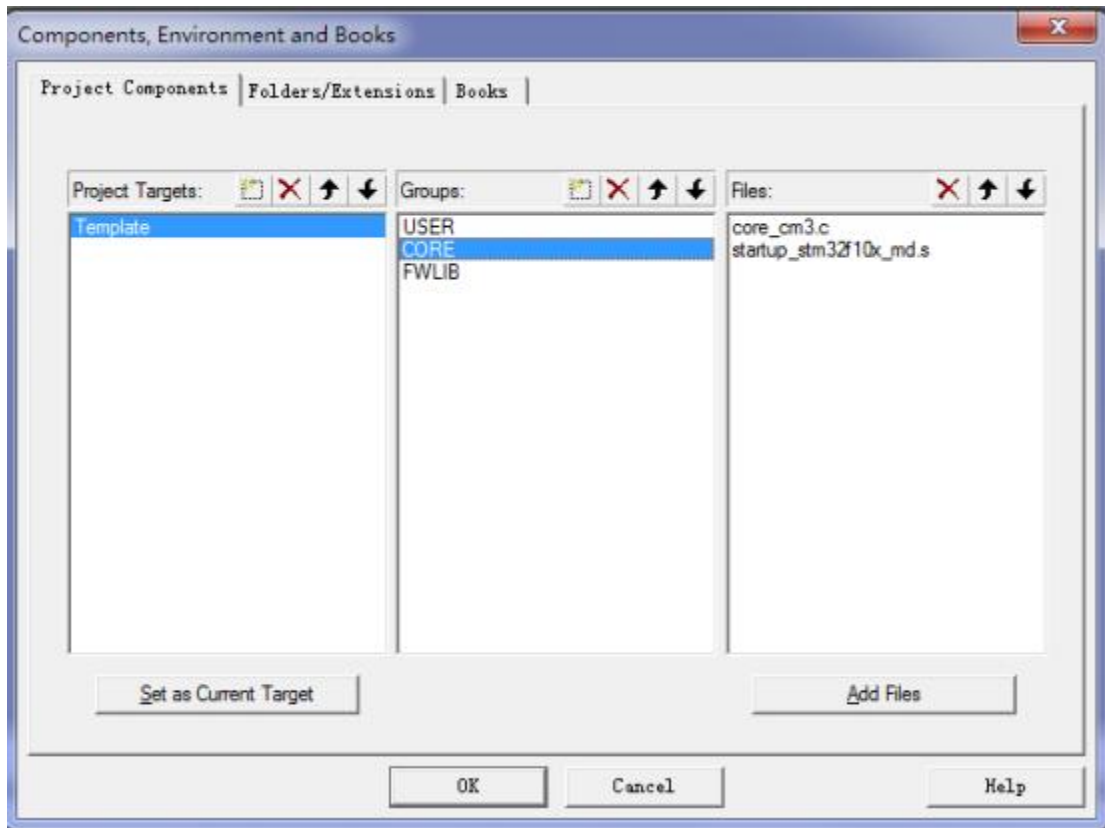
这里需要说明一下，对于我们写代码，如果我们只用到了其中的某个外设，我们就可以不用添加没有用到的外设的库文件。例如我只用 GPIO，我可以只用添加 stm32f10x_gpio.c 而其他的可以不用添加。这里我们全部添加进来是为了后面方便，不用每次添加，当然这样的坏处是工程太大，编译起来速度慢，用户可以自行选择。



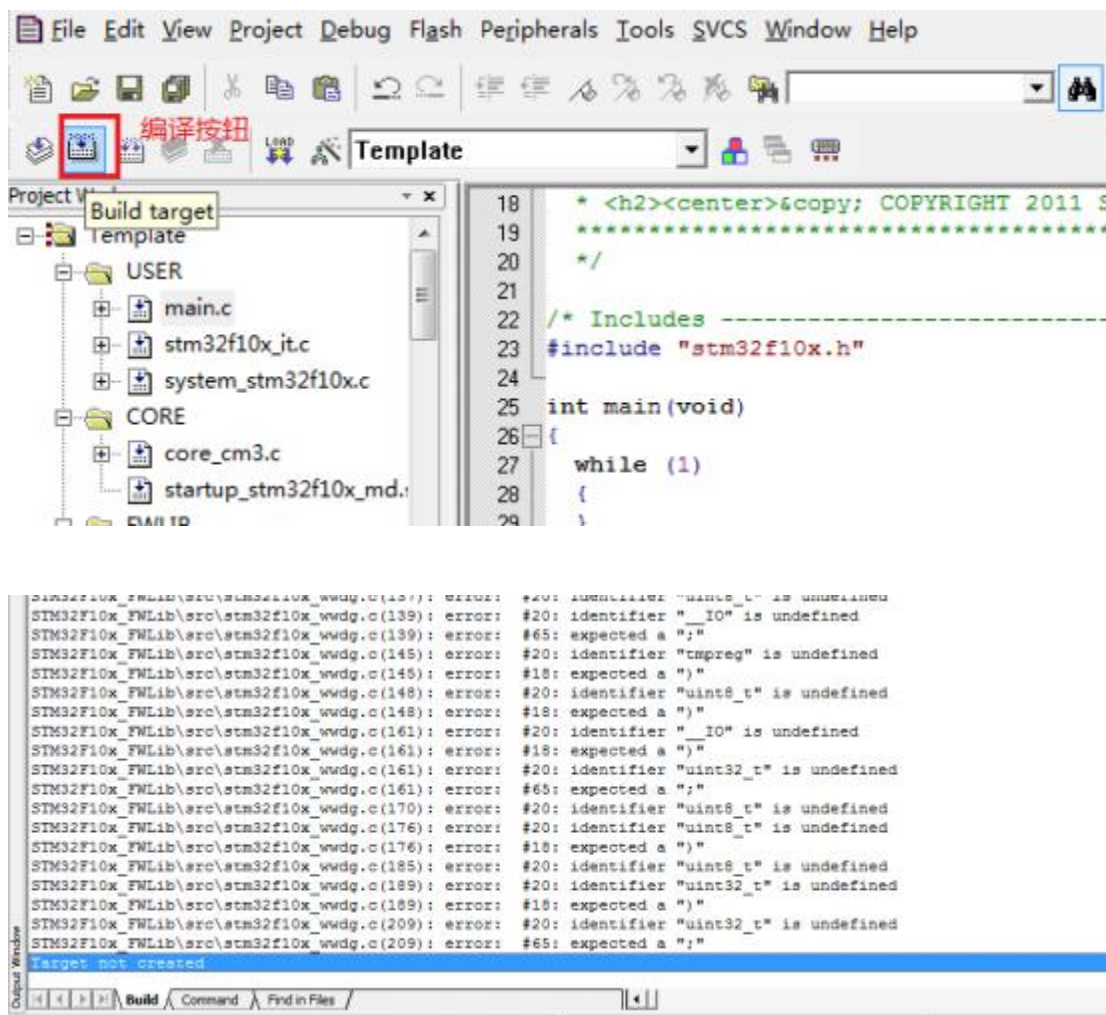
(14) 用同样的方法，将Groups 定位到CORE 和USER 下面，添加需要的文件。

这里我们的CORE 下面需要添加的文件为core_cm3.c, startup_stm32f10x_md.s, USER 目录下面需要添加的文件为main.c, stm32f10x_it.c, system_stm32f10x.c。这样我们需要添加的文件已经添加到我们的工程中去了，最后点击OK，回到工程主界面。



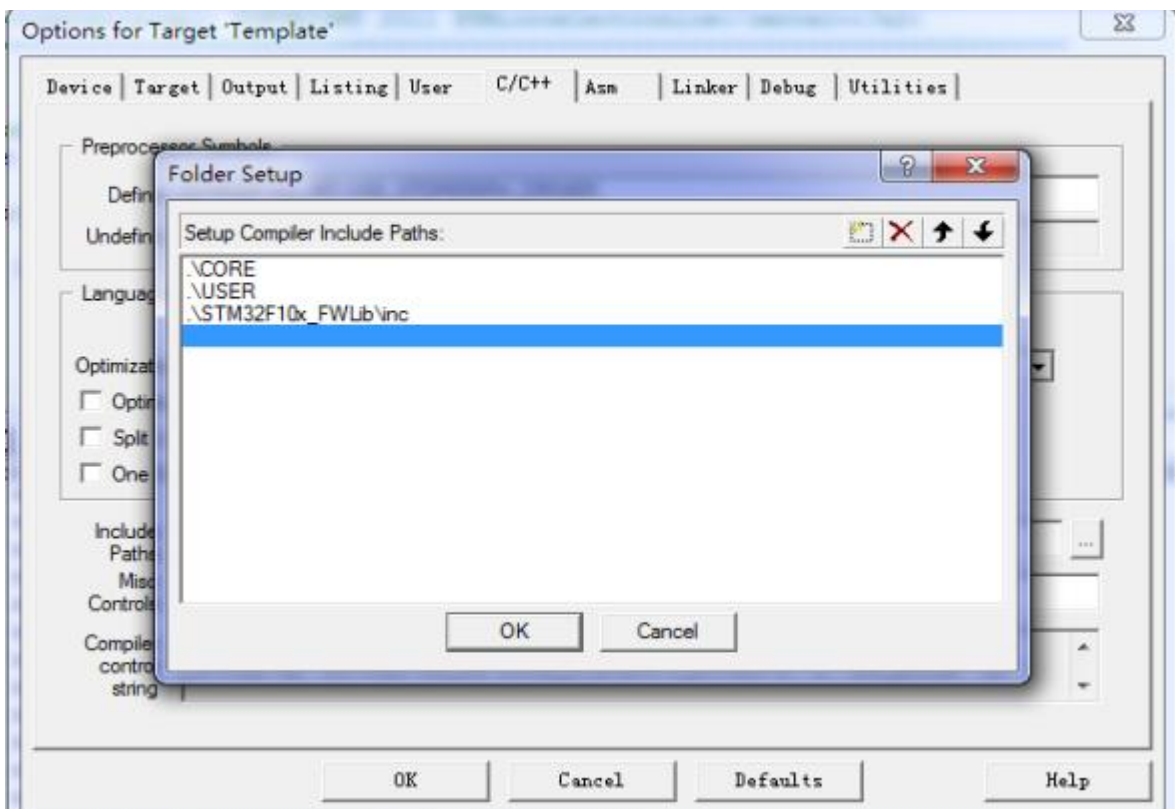
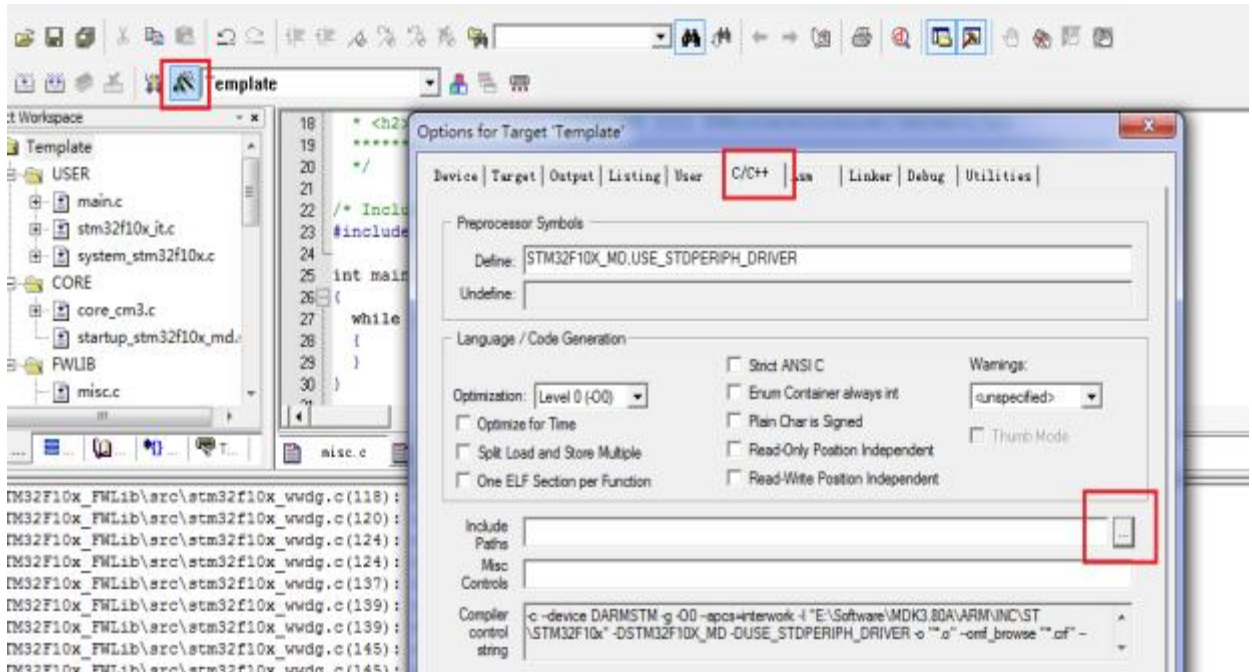


(15) 下面我们点击编译按钮  编译工程，可以看到很多报错，因为找不到库文件。



(16) 下面我们要告诉 MDK，在哪些路径之下搜索相应的文件。

回到工程主菜单，点击魔术棒 ，出来一个菜单，然后点击 c/c++选项。然后点击 IncludePaths 右边的按钮。弹出一个添加 path 的对话框，然后将图上面的 3 个目录添加进去。记住，keil 只会在一级目录查找，所以如果你的目录下面还有子目录，记得 path 一定要定位到最后一级子目录。然后点击 OK。



(17) 接下来，我们再来编译工程，可以看到又报了很多同样的错误。

为什么呢?我们可以双击错误，然后会自动定位到文件 stm32f10x.h 中出错的地方，可以看到代码：

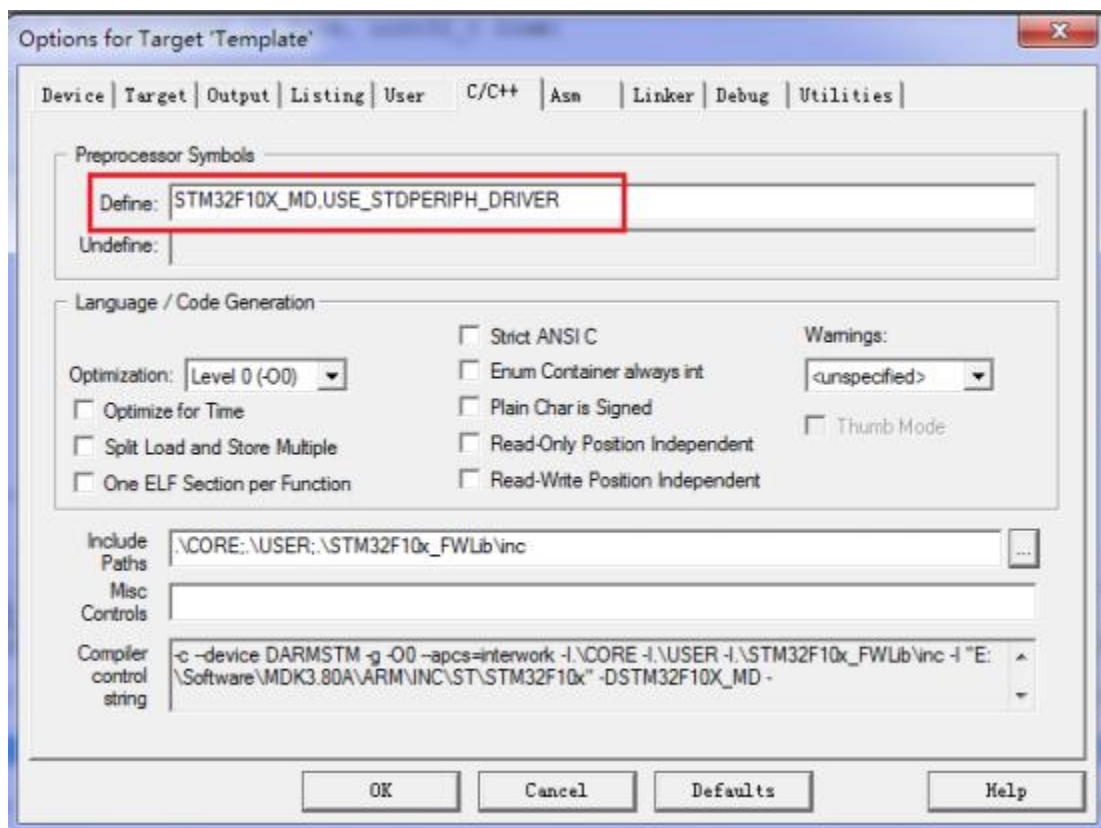
```
#ifndef defined(STM32F10X_LD)&&!defined(STM32F10X_LD_VL)&&!defined(STM32F10X_MD)&&!defined(STM32F10X_MD_VL)&&!defined(STM32F10X_HD)&&!defined(STM32F10X_HD_VL)&&!defined(STM32F10X_XL)&&!defined(STM32F10X_CL)
```

```
#error"PlaseselectfirstthetargetSTM32F10xdeviceusedinyourapplication(instm32f10x.hfile)"
```

```
#endif
```

这是因为 3.5 版本的库函数在配置和选择外设的时候通过宏定义来选择的，所以我们需要配置一个全局的宏定义变量。按照步骤（16），定位到 c/c++ 界面，然后 copy “STM32F10X_MD, USE_STDPERIPH_DRIVER” 到 Define 里面。

这里解释一下，如果你用的是大容量那么 STM32F10X_MD 修改为 STM32F10X_HD，小容量修改为 STM32F10X_LD. 然后点击 OK。



(18) 这次在编译之前，我们记得打开工程 USUR 下面的 main.c，复制下面代码到 main.c 覆盖已有代码，然后进行编译。

(记得在代码的最后面加上一个回车，否则会有警告)

```
#include"stm32f10x.h"
```

```
GPIO_InitTypeDef g1;
```

```
int main(void)
```

```
{
```

```
SystemInit();
```

```
RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOD,ENABLE);
```

```
g1.GPIO_Pin=GPIO_Pin_0|GPIO_Pin_2;
```

```
g1.GPIO_Speed=GPIO_Speed_50MHz;
```

```
PIO_InitStructure.GPIO_Mode=GPIO_Mode_Out_PP;
```

```
GPIO_Init(GPIOD,&g1);
```

```
while(1)
```

```
{
```

```
/*SetPD0andPD2*/
```

```

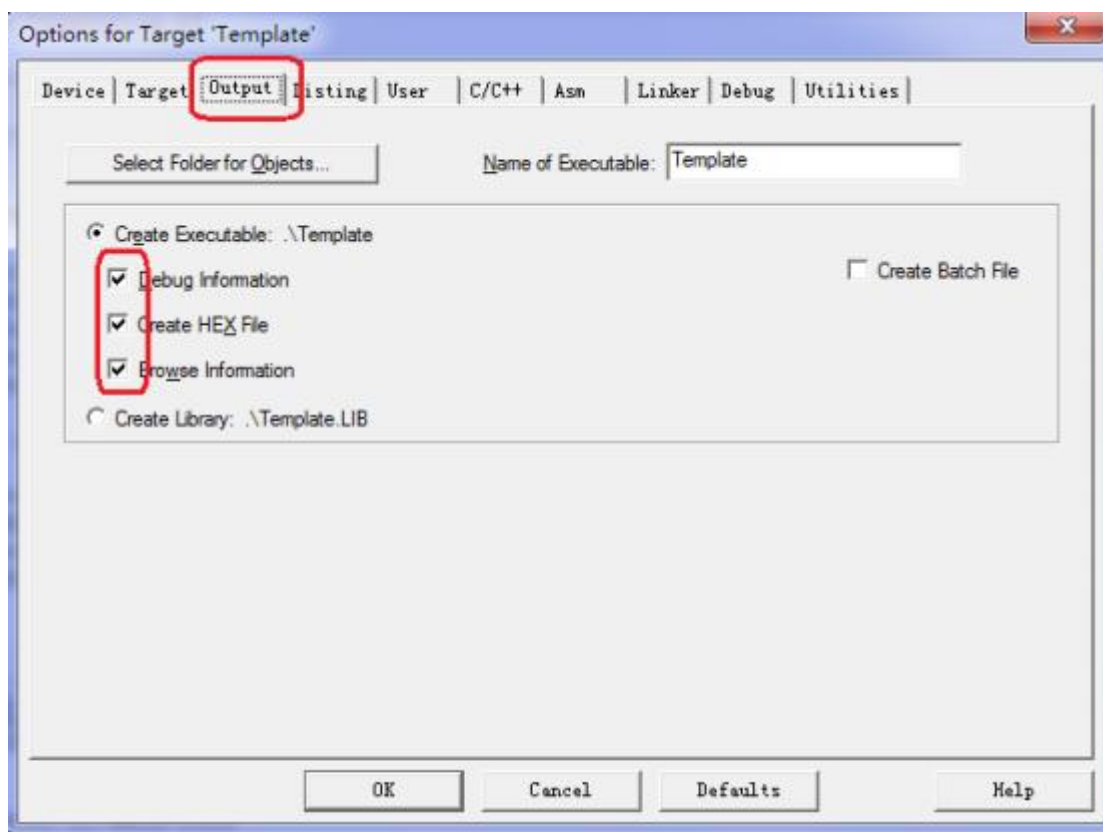
    GPIO->BSRR=0x00000005;
    /*ResetPD0andPD2*/
    GPIO->BRR=0x00000005;
}
}
#ifdefUSE_FULL_ASSERT

voidassert_failed(uint8_t*file,uint32_tline)
{
    while(1)
    {
    }
}
#endif

```

- (19) 这次编译可以看出，已经成功了。这样一个工程模版建立完毕。下面还需要配置，让编译之后能够生成 hex 文件。同样点击魔术棒，进入配置菜单，选择 Output。然后勾上下三个选项。其中 CreateHEXfile 是编译生成 hex 文件，BrowserInformation 是可以查看变量和函数定义，这里我们不做过多解释，在我们的不完全手册里面有讲解。





(20) 重新编译代码，可以看到生成了 hex 文件，这个文件我们用 mcuisp 下载到 mcu 即可。

2.8 工程样例下载测试

解压工程样例及模板，学习工程参数的修改、编译、下载、调试。

群文件 > 实验资料

文件

-  MINISTM32综合测试程序.rar
-  工程模板.zip
-  STM32F10x_StdPeriph_Lib_V3.5.0.zip

下载群文件中的这 3 个压缩文件，解压缩并下载调试前 2 个工程。阅读最后一个函数库，了解库函数基本结构。

? 思考题

工程模板中需要让两个发光二极管交替闪烁，闪烁需要对发光二极管引脚 PA8 和 PD2 取反，有哪些方法可以对引脚取反，请修改“工程模板”对应程序，测试你所想到的各种方法？

3. 实验 2—GPIO 输出实验-LED 闪烁实验

STM32 最简单的外设莫过于 IO 口的高低电平控制了,本章将通过一个经典的跑马灯程序,带大家开启 STM32 之旅,通过本章的学习,你将了解到 STM32 的 IO 口作为输出使用的方法。在本章中,我们将通过代码控制 ALIENTEK MiniSTM32 开发板上的两个 LED: DS0 和 DS1 交替闪烁,实现类似跑马灯的效果。

3.1 STM32 IO 简介

本章将要实现的是控制 ALIENTEK miniSTM32 开发板上的两个 LED 实现一个类似跑马灯 的效果,该实验的关键在于如何控制 STM32 的 IO 口输出。了解了 STM32 的 IO 口如何输出的,就可以实现跑马灯了。通过这一章的学习,你将初步掌握 STM32 基本 IO 口的使用,而这是迈向 STM32 的第一步。

首先要提一下,在固件库中,GPIO 端口操作对应的库函数函数以及相关定义在文件 stm32f10x_gpio.h 和 stm32f10x_gpio.c 中。

STM32 的 IO 口相比 51 而言要复杂得多,所以使用起来也困难很多。首先 STM32 的 IO 口可以由软件配置成如下 8 种模式:

- (1) 输入浮空
- (2) 输入上拉
- (3) 输入下拉
- (4) 模拟输入
- (5) 开漏输出
- (6) 推挽输出
- (7) 推挽式复用功能
- (8) 开漏复用功能

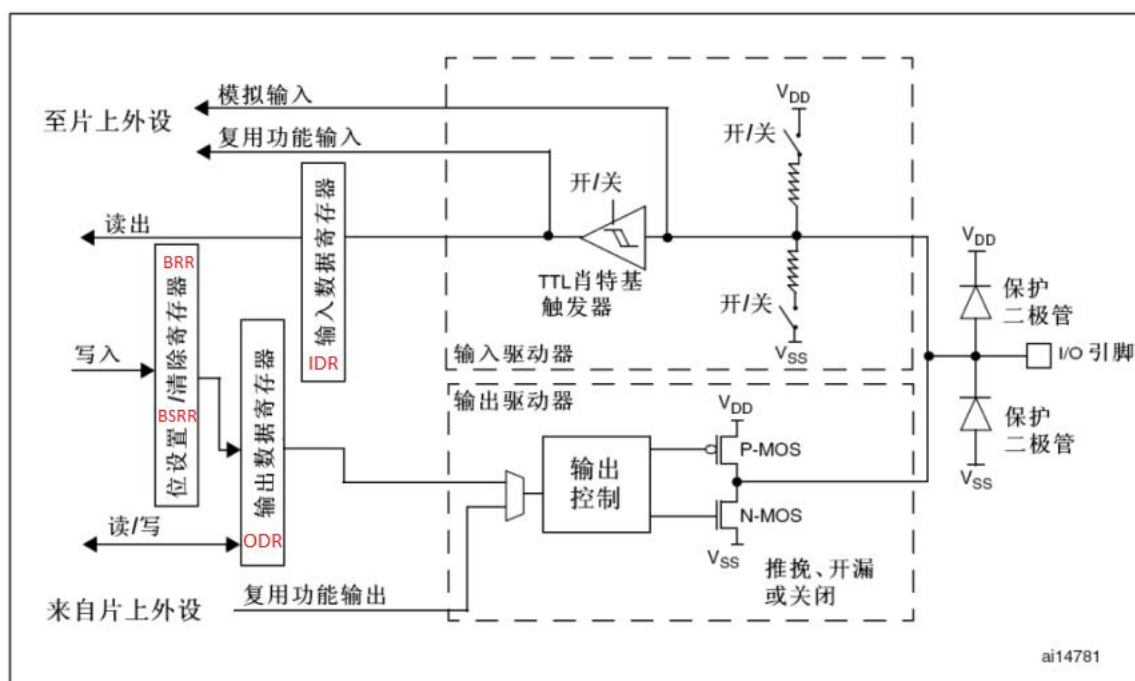


图 3.1 IO 的基本结构

每个 IO 口可以自由编程，但 IO 口寄存器必须要按 32 位字被访问。STM32 的很多 IO 口都是 5V 兼容的，这些 IO 口在与 5V 电平的外设连接的时候很有优势，具体哪些 IO 口是 5V 兼容的，可以从该芯片的数据手册管脚描述章节查到(I/O Level 标 FT 的就是 5V 电平兼容的)。

STM32 的每个 IO 端口都有 7 个寄存器来控制。他们分别是：配置模式的 2 个 32 位的端口配置寄存器 CRL 和 CRH；2 个 32 位的数据寄存器 IDR 和 ODR；1 个 32 位的置位/复位寄存器 BSRR；一个 16 位的复位寄存器 BRR；1 个 32 位的锁存寄存器 LCKR。大家如果想要了解每个寄存器的详细使用方法，可以参考《STM32 中文参考手册 V10》P105~P129。

CRL 和 CRH 控制着每个 IO 口的模式及输出速率。

STM32 的 IO 口位配置表如表 6.1 所示：

表 3.1 STM32 的 IO 口位配置表

配置模式		CNF1	CNF0	MODE1	MODE0	PxODR寄存器		
通用输出	推挽(Push-Pull)	0	0	01 10 11 见下表		0 或 1		
	开漏(Open-Drain)		1			0 或 1		
复用功能输出	推挽(Push-Pull)	1	0					不使用
	开漏(Open-Drain)		1					不使用
输入	模拟输入	0	0	00				不使用
	浮空输入		1					不使用
	下拉输入	1	0			0		
	上拉输入					1		

STM32 输出模式配置如表 3.2 所示：

表 3.2 STM32 输出模式配置表

MODE[1:0]	意义
00	保留
01	最大输出速度为10MHz
10	最大输出速度为2MHz
11	最大输出速度为50MHz

接下来我们看看端口低配置寄存器 CRL 的描述，如图 3.2 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF7[1:0]		MODE7[1:0]		CNF6[1:0]		MODE6[1:0]		CNF5[1:0]		MODE5[1:0]		CNF4[1:0]		MODE4[1:0]	
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF3[1:0]		MODE3[1:0]		CNF2[1:0]		MODE2[1:0]		CNF1[1:0]		MODE1[1:0]		CNF0[1:0]		MODE0[1:0]	
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
位31:30		CNFy[1:0]: 端口x配置位(y = 0...7) (Port x configuration bits) 软件通过这些位配置相应的I/O端口, 请参考表17端口位配置表。 在输入模式(MODE[1:0]=00): 00: 模拟输入模式 01: 浮空输入模式(复位后的状态) 10: 上拉/下拉输入模式 11: 保留 在输出模式(MODE[1:0]>00): 00: 通用推挽输出模式 01: 通用开漏输出模式 10: 复用功能推挽输出模式 11: 复用功能开漏输出模式													
27:26															
23:22															
19:18															
15:14															
11:10															
7:6															
3:2															
位29:28		MODEy[1:0]: 端口x的模式位(y = 0...7) (Port x mode bits) 软件通过这些位配置相应的I/O端口, 请参考表17端口位配置表。 00: 输入模式(复位后的状态) 01: 输出模式, 最大速度10MHz 10: 输出模式, 最大速度2MHz 11: 输出模式, 最大速度50MHz													
25:24															
21:20															
17:16															
13:12															
9:8, 5:4															
1:0															

图 3.2 端口低配置寄存器 CRL 各位描述

该寄存器的复位值为 0X4444 4444, 从图 3.1 可以看到, 复位值其实就是配置端口为浮空 输入模式。从上图还可以得出: STM32 的 CRL 控制着每组 IO 端口(A~G)的低 8 位的模式。每个 IO 端口的位占用 CRL 的 4 个位, 高两位为 CNF, 低两位为 MODE。这里我们可以记住几个常用的配置, 比如 0X0 表示模拟输入模式(ADC 用)、0X3 表示推挽输出模式(做输出口用, 50M 速率)、0X8 表示上/下拉输入模式(做输入口用)、0XB 表示复用输出(使用 IO 口的第二 功能, 50M 速率)。

CRH 的作用和 CRL 完全一样, 只是 CRL 控制的是低 8 位输出口, 而 CRH 控制的是高 8 位输出口。这里我们对 CRH 就不做详细介绍了。下面我们讲解一下怎样通过固件库设置 GPIO 的相关参数和输出。

GPIO 相关的函数和定义分布在固件库文件 stm32f10x_gpio.c 和头文件 stm32f10x_gpio.h 文件中。

在固件库开发中, 操作寄存器 CRH 和 CRL 来配置 IO 口的模式和速度是通过 GPIO 初始化 函数完成:

```
void GPIO_Init(GPIO_TypeDef* GPIOx, GPIO_InitTypeDef* GPIO_InitStruct);
```

这个函数有两个参数, 第一个参数是用来指定 GPIO, 取值范围为 GPIOA~GPIOG。

第二个参数为初始化参数结构体指针, 结构体类型为 GPIO_InitTypeDef。下面我们看看这个结 构体的定义。首先我们打开我们光盘的跑马灯实验, 然后找到 FWLib 组下面的 stm32f10x_gpio.c 文件, 定位到 GPIO_Init 函数体处, 双击入口参数类型 GPIO_InitTypeDef 后右键选择 “Go to definition of ...” 可以查看结构体的定义:

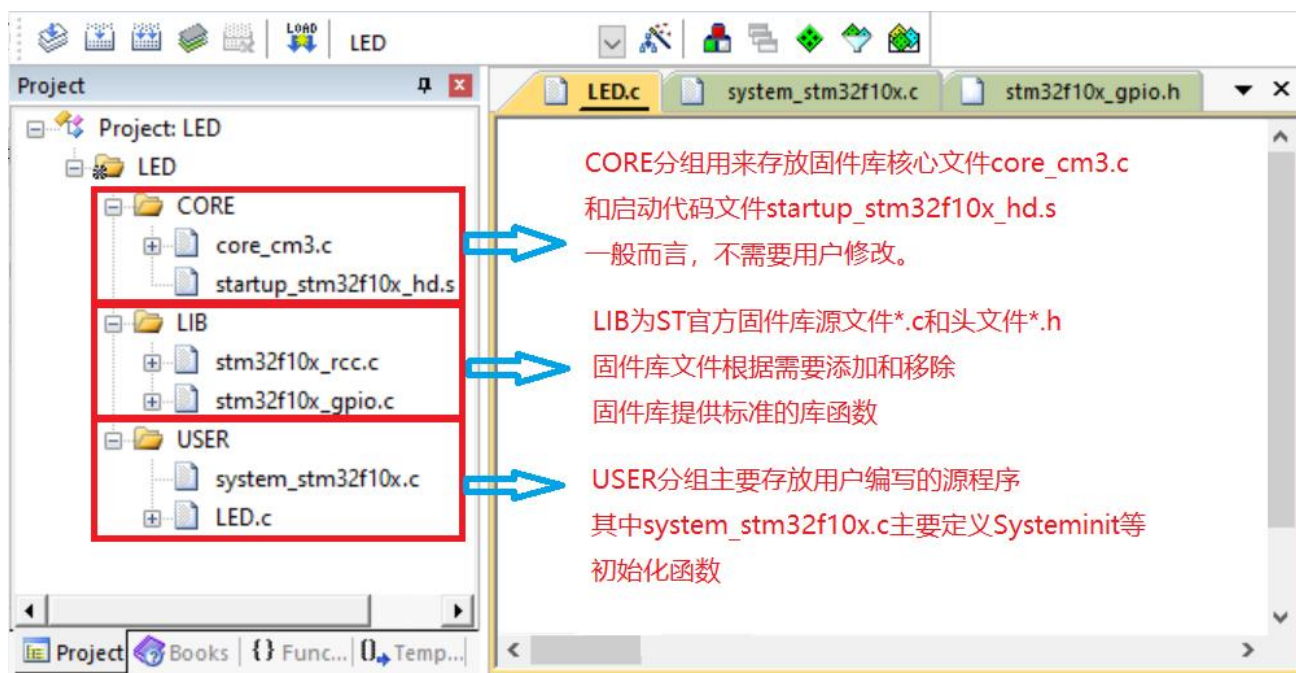


图 3.3 跑马灯实验目录结构

```
typedef struct
{
    uint16_t GPIO_Pin;

    GPIOSpeed_TypeDef GPIO_Speed;
    GPIOMode_TypeDef GPIO_Mode;
}GPIO_InitTypeDef;
```

下面我们通过一个 GPIO 初始化实例来讲解这个结构体的成员变量的含义。

通过初始化结构体初始化 GPIO 的常用格式是：

```
GPIO_InitTypeDef  g1;

g1.GPIO_Pin = GPIO_Pin_5;           //LED0-->PB.5  端口配置

g1.GPIO_Mode = GPIO_Mode_Out_PP;    //推挽输出

g1.GPIO_Speed = GPIO_Speed_50MHz;  //速度 50MHz
```

上面代码的意思是设置 GPIOB 的第 5 个端口为推挽输出模式，同时速度为 50M。从上面初始化代码可以看出，结构体 g1 的第一个成员变量 GPIO_Pin 用来设置是要初始化哪个或者哪些 IO 口；第二个成员变量 GPIO_Mode 是用来设置对应 IO 端口的输出输入模式，这些模式是上面我们讲解的 8 个模式，在 MDK 中是通过一个枚举类型定义的：

```
typedef enum
{
    GPIO_Mode_AIN = 0x0,           //模拟输入
    GPIO_Mode_IN_FLOATING = 0x04,  //浮空输入
    GPIO_Mode_IPD = 0x28,          //下拉输入
    GPIO_Mode_IPU = 0x48,          //上拉输入
    GPIO_Mode_Out_OD = 0x14,       //开漏输出
    GPIO_Mode_Out_PP = 0x10,       //通用推挽输出
    GPIO_Mode_AF_OD = 0x1C,        //复用开漏输出
    GPIO_Mode_AF_PP = 0x18         //复用推挽
}GPIO_Mode_TypeDef;
```

第三个参数是 IO 口速度设置，有三个可选值，在 MDK 中同样是通过枚举类型定义：

```
typedef enum
{
    GPIO_Speed_10MHz = 1,
    GPIO_Speed_2MHz,
    GPIO_Speed_50MHz
}GPIO_Speed_TypeDef;
```

要想知道某个 IO 口的电平状态，你只要读 IDR 这个寄存器，再看某个位的状态就可以了。使用起来是比较简单的。

在固件库中操作 IDR 寄存器读取 IO 端口数据是通过 GPIO_ReadInputDataBit 函数实现的：

```
uint8_t GPIO_ReadInputDataBit(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)
```

比如我要读 GPIOA.5 的电平状态，那么方法是：

```
GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_5);
```

返回值是 1 (Bit_SET) 或者 0 (Bit_RESET)；

ODR 是一个端口输出数据寄存器，也只用了低 16 位。该寄存器为可读写，从该寄存器读出来的数据可以用于判断当前 IO 口的输出状态。而向该寄存器写数据，则可以控制某个 IO 口的输出电平。

在固件库中设置 ODR 寄存器的值来控制 IO 口的输出状态是通过函数 GPIO_Write 来实现的：

```
void GPIO_Write(GPIO_TypeDef* GPIOx, uint16_t PortVal);
```

该函数一般用来往一次性一个 GPIO 的多个端口设置。

BSRR 寄存器是端口位设置/清除寄存器。该寄存器和 ODR 寄存器具有类似的作用，都可以用来设置 GPIO 端口的输出位是 1 还是 0。

该寄存器通过举例子可以很清楚了解它的使用方法。例如你要设置 GPIOA 的第 1 个端口值为 1，那么你只需要往寄存器 BSRR 的低 16 位对应位写 1 即可：

```
GPIOA->BSRR=1<<1;
```

如果你要设置 GPIOA 的第 1 个端口值为 0，你只需要往寄存器高 16 位对应为写 1 即可：

```
GPIOA->BSRR=1<<(16+1)
```

该寄存器往相应位写 0 是无影响的，所以我们要设置某些位，我们不用管其他位的值。

BRR 寄存器是端口位清除寄存器。该寄存器的作用跟 BSRR 的高 16 位雷同，这里就不做详细讲解。在 STM32 固件库中，通过 BSRR 和 BRR 寄存器设置 GPIO 端口输出是通过函数 GPIO_SetBits() 和函数 GPIO_ResetBits() 来完成的。

```
void GPIO_SetBits(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);
```

```
void GPIO_ResetBits(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)
```

在多数情况下，我们都是采用这两个函数来设置 GPIO 端口的输入和输出状态。比如我们要设置 GPIOB.5 输出 1，那么方法为：

```
GPIO_SetBits(GPIOB, GPIO_Pin_5);
```

反之如果要设置 GPIOB.5 输出位 0，方法为：

```
GPIO_ResetBits (GPIOB, GPIO_Pin_5);
```

GPIO 相关的函数我们先讲解到这里。虽然 IO 操作步骤很简单，这里我们还是做个概括性的总结，操作步骤为：

- 1) 使能 IO 口时钟。调用函数为 RCC_APB2PeriphClockCmd()。
- 2) 初始化 IO 参数。调用函数 GPIO_Init();
- 3) 操作 IO。操作 IO 的方法就是上面我们讲解的方法。

上面我们讲解了 STM32 IO 口的基本知识以及固件库操作 GPIO 的一些函数方法，下面我们来讲解我们的跑马灯实验的硬件和软件设计。

3.2 硬件设计

本章用到的硬件只有 LED (DS0 和 DS1)。其电路在 ALIENTEK MiniSTM32 开发板上默认 是已经连接好了的。DS0 接 PA8，DS1 接 PD2。所以在硬件上不需要动任何东西。其连接原理

图如图 3.4 下：

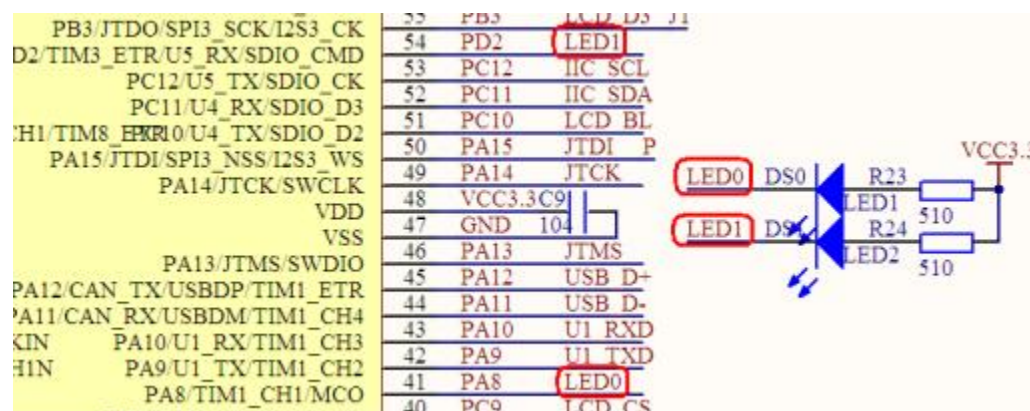


图 3.4 LED 与 STM32 连接原理图

3.3 软件设计

跑马灯实验我们主要用到的固件库文件是：

```
stm32f10x_gpio.c /stm32f10x_gpio.h
```

```
stm32f10x_rcc.c/stm32f10x_rcc.h
```

其中 stm32f10x_rcc.h 头文件在每个实验中都要引入，因为系统时钟配置函数以及相关的外设时钟使能函数都在这个其源文件 stm32f10x_rcc.c 中。

看看代码：

```
RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA|
                        RCC_APB2Periph_GPIOD, ENABLE);    //使能 GPIOA,GPIOD 端口时钟
```

这行代码的作用是使能 APB2 总线上的 GPIOA 和 GPIOD 的时钟。

在配置完时钟之后，LED_Init 配置了 GPIOA.8 和 GPIOD.2 的模式为推挽输出，并且默认输出 1。这样就完成了对这两个 IO 口的初始化。函数代码是：

```
g1.GPIO_Pin = GPIO_Pin_8;                //LED0-->PA8 端口配置
g1.GPIO_Mode = GPIO_Mode_Out_PP;         //推挽输出
g1.GPIO_Speed = GPIO_Speed_50MHz;        //IO 口速度为 50MHz
GPIO_Init(GPIOB, &g1);                   //初始化 PA8
GPIO_SetBits(GPIOA,GPIO_Pin_8);          //PA8 输出高

g1.GPIO_Pin = GPIO_Pin_2;                //LED1-->PD2 推挽输出
GPIO_Init(GPIOE, &g1);                   //初始化 PD2
GPIO_SetBits(GPIOD,GPIO_Pin_2);          //PD2 输出高
```

这里需要说明的是，因为 GPIOA 和 GPIOD 的 IO 口的初始化参数都是设置在结构体变量 g1 中，因为两个 IO 口的模式和速度都一样，所以我们只用初始化一次，在 GPIOD.2 的初始化的时候就不需要再重复初始化速度和模式了。最后一行代码：

```
GPIO_SetBits(GPIOD,GPIO_Pin_2);
```

的作用是在初始化中将 GPIOD.2 输出设置为高。

我们分析一下图 3.1 中下半部分为输出电路，输出引脚 PIN.x 受控于 ODR.x，其中 x=0~15，对应每个端口的 16 个引脚。那么输出可以直接控制 ODR 来实现，对应的标准库函数为 GPIO_Write，库函数中该函数定义如下：

```
void GPIO_Write(GPIO_TypeDef* GPIOx, uint16_t PortVal)
{
    /* Check the parameters */
    assert_param(IS_GPIO_ALL_PERIPH(GPIOx));
    GPIOx->ODR = PortVal;
}
```

该函数中就 2 条语句，第 1 条语句调用 assert_param 函数检查传递过来的参数是否正确。assert_param 函

数是 STM32 库函数中的一种宏，它是 STM32 开发工具包中的自带宏，主要用于检测参数有效性，也就是说，当你使用 STM32 函数时，它会检测函数参数是否正确，如果参数不正确就在编译时报错并提醒用户。假设我们的参数都是有效的，那么后续我们就忽略 `assert_param` 函数，因此实际上 `GPIO_Write` 函数就一条功能语句“`GPIOx->ODR = PortVal`”，也就是把 16 位数据 `PortVal` 输出到 `GPIOx` 的 16 个管脚，该输出同时输出（影响）16 个管脚。

那问题来了，我只想输出 1 个或几个引脚而不影响剩下的其它引脚，怎么办？

1) 使用直接输出数据的方法

直接使用数据输出寄存器 `ODR`。第一步，把原来的 `ODR` 读回来送到 `y` 变量；第二步，将读回来的 `y` 变量修改需要输出的位（不影响其它位）；第三步，将修改以后的数据 `y` 输出到 `ODR`。

如果使用库函数，那我们就要用到读入 `ODR` 函数 `GPIO_ReadInputData`，其函数定义如下：

```
uint16_t GPIO_ReadInputData(GPIO_TypeDef* GPIOx)
{
    /* Check the parameters */
    assert_param(IS_GPIO_ALL_PERIPH(GPIOx));
    return ((uint16_t)GPIOx->IDR);
}
```

那我们举例：假设我们想让 `GPIOA` 的 `PA2=0`, `PA9=0`, `PA5=1`, `PA12=1`，那分三步如下：

第一步，读入 `ODR`：

```
uint16_t y;
y = GPIO_ReadInputData(GPIOA);
```

第二步，修改 `y` 相应的位：首先用与操作清零 `y` 的第 2、5、9、12 位（对应 `PA2`、`PA5`、`PA9`、`PA12`），然后第 5、12 位输出 1，所以用或操作置位 `y` 的第 5、12 位：

```
y = (y & (~((1<<2 | 1<<5 | 1<<9 | 1<<12)))) | (1<<5 | 1<<12);
```

第三步，输出 `y`：

```
GPIO_Write(GPIOA, y);
```

完整写出输出 `GPIOA` 的 `PA2=0`, `PA9=0`, `PA5=1`, `PA12=1` 程序如下：

```
uint16_t y;
y = GPIO_ReadInputData(GPIOA);
y = (y & (~((1<<2 | 1<<5 | 1<<9 | 1<<12)))) | (1<<5 | 1<<12);
GPIO_Write(GPIOA, y);
```

进一步简化，可以不定义中间变量 `y`，如下：

```
GPIO_Write(GPIOA, GPIO_ReadInputData(GPIOA) & (~((1<<2|1<<5|1<<9|1<<12)) | (1<<5|1<<12)));
```

当然我们直接使用寄存器，可以减少函数调用产生的消耗，程序效率更高，代码如下：

```
GPIOA->ODR = GPIOA->ODR & (~((1<<2|1<<5|1<<9|1<<12)) | (1<<5|1<<12));
```

2) 使用置位/清零的方法

间接使用输出数据寄存器 ODR，我看图 3.1 中 ODR 的前面设置了 2 个寄存器—清零寄存器 BRR 和置位寄存器 BSRR。BRR 和 BSRR 是通过清零和置位 ODR 中的相应位来达到输出指定管脚的目的。我们同样以上面要求为例，给出让 GPIOA 的 PA2=0, PA9=0, PA5=1, PA12=1 的输出程序如下：

```
GPIO_SetBits(GPIOA, GPIO_Pin_5 | GPIO_Pin_12); //PA5=1, PA12=1
GPIO_ResetBits(GPIOA, GPIO_Pin_2 | GPIO_Pin_9); //PA2=0, PA9=0
```

使用的 2 个库函数定义如下：

```
void GPIO_SetBits(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)
{
    /* Check the parameters */
    assert_param(IS_GPIO_ALL_PERIPH(GPIOx));
    assert_param(IS_GPIO_PIN(GPIO_Pin));
    GPIOx->BSRR = GPIO_Pin;
}

void GPIO_ResetBits(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)
{
    /* Check the parameters */
    assert_param(IS_GPIO_ALL_PERIPH(GPIOx));
    assert_param(IS_GPIO_PIN(GPIO_Pin));
    GPIOx->BRR = GPIO_Pin;
}
```

当然为了减少函数调用，我们也可以优化程序，使用寄存器编程如下：

```
GPIOA->BSRR = GPIOA, GPIO_Pin_5 | GPIO_Pin_12; //PA5=1, PA12=1
GPIOA->BRR = GPIOA, GPIO_Pin_2 | GPIO_Pin_9;    //PA2=0, PA9=0
```

其中 GPIO_Pin_2 是等于 (1<<2) 的常量，GPIO_Pin_5 是等于 (1<<5) 的常量，.....

3.4 仿真与下载

此代码，我们先进行软件仿真，看看结果对不对，根据软件仿真的结果，然后再下载到 ALIENTEKminiSTM32 板子上面看运行是否正确。

首先，我们进行软件仿真(请先确保 Options for Target^④Debug 选项卡里面已经设置为 Use

Simulator)。先按  开始仿真，接着按 ，显示逻辑分析窗口，点击 Setup，新建两个信号 PORTA.8 和 PORTD.2，如图 3.5 所示：

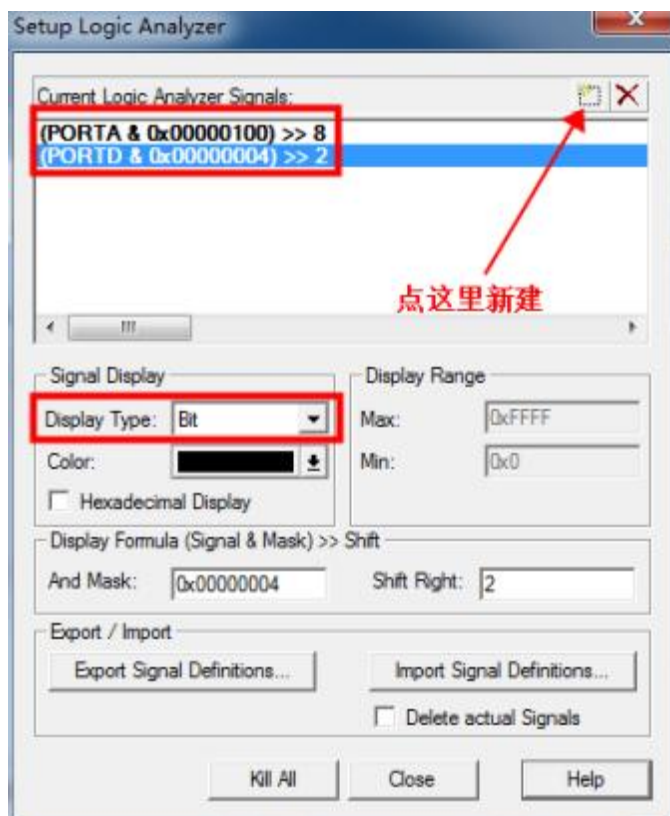


图 3.5 逻辑分析设置

Display Type 选择 bit，然后单击 Close 关闭该对话框，可以看到逻辑分析窗口出来了两个信号，如图 3.6 所示：

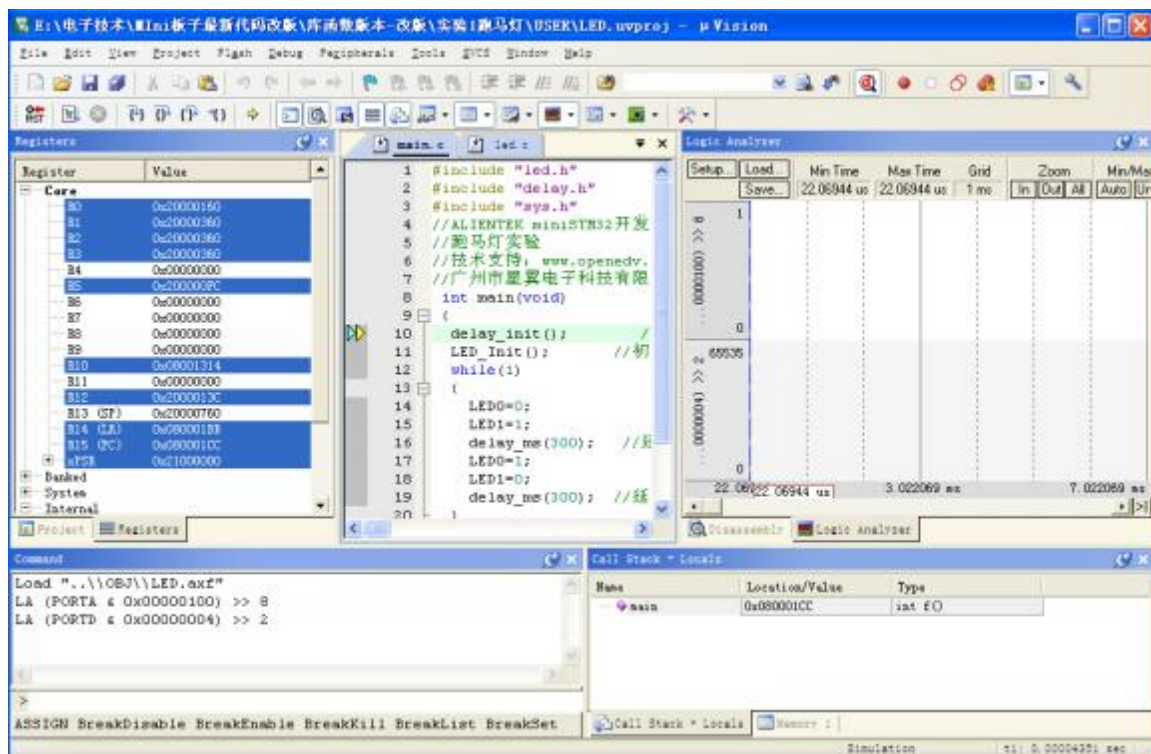


图 3.6 设置后的逻辑分析窗口

接着，点击 ，开始运行。运行一段时间之后，按  按钮，暂停仿真回到逻辑分析窗口，

可以看到如图 3.7 所示的波形：

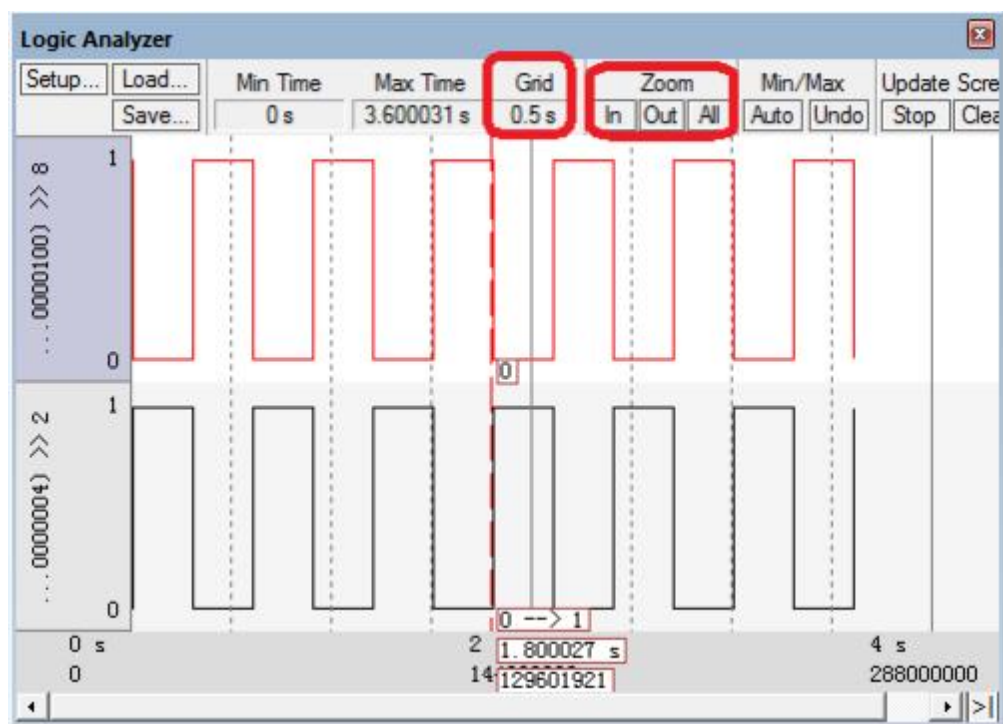


图 3.7 仿真波形

这里注意 Grid 要调节到 0.5s 左右比较合适，可以通过 Zoom 里面的 In 按钮来放大波形，通过 Out 按钮来缩小波形，或者按 All 显示全部波形。从上图中可以看到 PORTA.8 和 PORTD.2 交替输出，周期可以通过中间那根红线来测量。至此，我们的软件仿真已经顺利通过。

在软件仿真没有问题了之后，我们就可以把代码下载到开发板上，看看运行结果是否与我们仿真的一致。运行结果如图 3.8 所示：

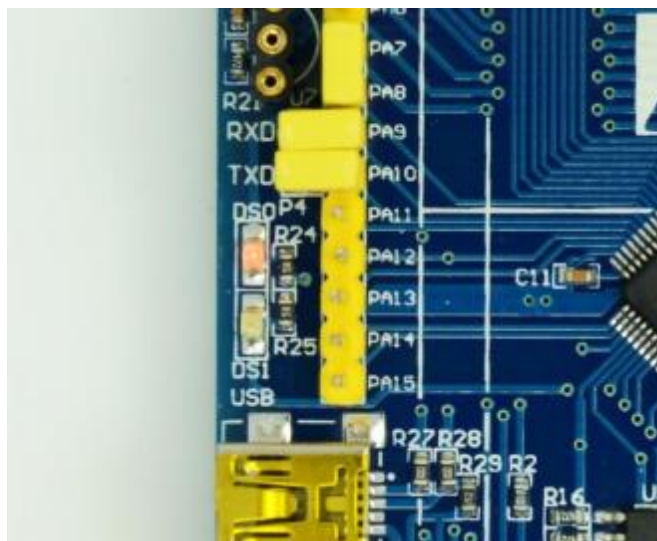


图 3.8 LED 显示效果图

4. 实验 3~4—GPIO 输出实验-蜂鸣器发声实验

蜂鸣器是一种将音频信号转换为声音信号的发声装置。它通常由直流电压供电。广泛应用于报警器、电脑、打印机等电子产品中作为音响设备。

4.1 蜂鸣器发声原理

蜂鸣器是一种一体化结构的电子讯响器，广泛应用于电子产品中作发声报警。BUZZER 蜂鸣器的分类：

- (1) 按其驱动方式的原理分，可分为：有源蜂鸣器（内含驱动线路，也叫自激式蜂鸣器）和无源蜂鸣器（外部驱动，也叫他激式蜂鸣器）；
- (2) 按构造方式的不同，可分为：电磁式蜂鸣器和压电式蜂鸣器；
- (3) 按封装的不同，可分为：DIP BUZZER（插针蜂鸣器）和 SMD BUZZER（贴片式蜂鸣器）；
- (4) 按电流的不同，可分为：直流蜂鸣器和交流蜂鸣器，其中，以直流最为常见压电式蜂鸣器，用的是压电材料，即当受到外力导致压电材料发生形变时压电材料会产生电荷。同样，当通电时压电材料会发生形变。

电磁式蜂鸣器，主要是利用通电导体会产生磁场的特性，用一个固定的永久磁铁与通电导体产生磁力推动固定在线圈上的鼓膜。

由于两种蜂鸣器发音原理不同，压电式结构简单耐用但音调单一，适用于报警器等设备。而电磁式由于音质好，所以多用于语音、音乐等设备。

电磁式蜂鸣器有两种类型：有源蜂鸣器和无源蜂鸣器。有源蜂鸣器内部带振荡源，无源蜂鸣器内部不带振荡源。这里所说的“源”不是指“电源”，而是指“振荡源”。

有源蜂鸣器和无源蜂鸣器的主要差别是对输入信号的要求不一样，有源蜂鸣器工作的理想信号是直流电，通电后有源蜂鸣器按固有的频率发声，频率不可改变。

无源蜂鸣器工作的理想信号是方波，必须输入带方波频率的电源，无源蜂鸣器接直流电是不会工作的，无源蜂鸣器输入不同的频率源输出不同音调的声音。所以无源电磁式蜂鸣器多用于语音、音乐等设备。

4.2 miniIO 蜂鸣器电路

为了尽量保证音质音量，miniIO 扩展板上选用无源电磁式蜂鸣器，其电路图如下：

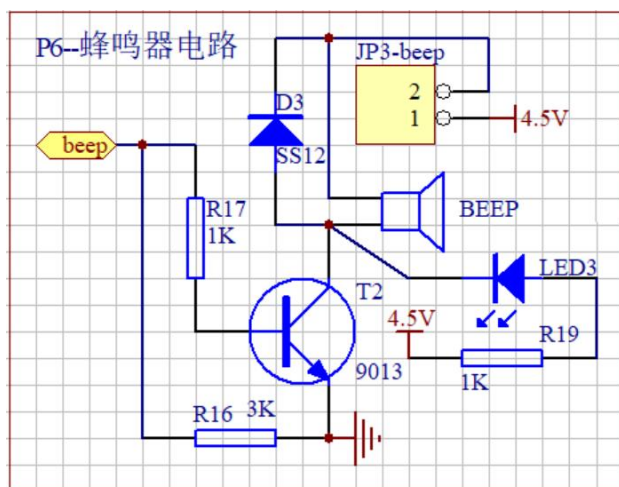


图 4.1 蜂鸣器原理图

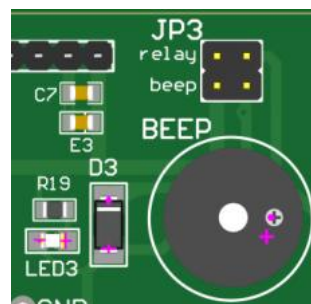


图 4.2 蜂鸣器实物效果图

其中蜂鸣器控制端为“beep”输入引脚，由杜邦线连接到 miniSTM32 的 GPIO 引脚，如下例中“beep”


```
for(i=0;i<u;i++) for(j=0;j<12;j++);
}
```

软件延时，靠指令执行时间来延时，本子程序完成 u 微秒延时。软件延时的优点是：仅靠指令延时，不需要额外的硬件，硬件简单。软件延时的缺点：延时精度差，延时精度受中断及指令影响，不容易精确计算延时时间，精度也差，本子程序 Delayus 在没有中断的情况下也只能达到微秒级。

软件延时与定时器延时相比较的特点是：

优点：软件延时有不需要额外的硬件支撑（比如不需要定时器硬件），因此硬件简单、灵活；

软件延时的缺点 1 是：与硬件定时器相比需要额外的空间和时间，效率低。软件延时是靠指令运行时间来延时，指令也就是程序，程序需要占用内存空间，运行需要占用 CPU 时间；

软件延时的缺点 2 是：与硬件定时器相比延时精度差。软件延时时间需要计算指令运行时间，统计工作量大，精度与指令运行时间有关。同时，在运行延时指令工程中发生中断，那么延时时间延长，延长时
时间=中断响应时间+中断处理时间+中断返回时间，也就是说延时时间加长，并且时间与中断等有关不能完全确定，因此精度较差。

② IO 口初始化子程序

```
void BEEP_init()    //GPIO 初始化
{
    GPIO_InitTypeDef g1;    //定义 GPIO 初始化结构体变量 g1
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE); //使能 GPIOA 时钟
    g1.GPIO_Pin = GPIO_Pin_6;    //beep-->PA.6 端口配置
    g1.GPIO_Speed = GPIO_Speed_50MHz;
    g1.GPIO_Mode = GPIO_Mode_Out_PP; //推挽输出
    GPIO_Init(GPIOA, &g1); //初始化 GPIOA.6
    GPIOA->ODR ^= ~(1<<6); //GPIOA->BRR=1<<6;//PA6=0
}
```

通过杜邦线 PA6 连接蜂鸣器，PA6 初始化为 GPIO 输出，输出速度 50MHz、推挽输出，GPIO 初始化有单独的初始化结构体类型 GPIO_InitTypeDef，调用标准库函数：

void GPIO_Init(GPIO_TypeDef* GPIOx, GPIO_InitTypeDef* GPIO_InitStruct)，完成相应管脚（PA6）的初始化。

调用初始化函数前一定要使能相应端口时钟，如：RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE)，STM32 上电时默认关闭所有 IO 时钟，以降低功耗，关闭某个设备时钟类似于关闭该设备电源。

③ 救护车警报声

首先发声的频率（音乐中的音调）是由控制引脚（这里是 PA6）的频率确定的，比如发出 $f=2\text{KHz}$ 频率，那么它的周期 $T=1/f=500\mu\text{s}$ ，一个周期由两半部分组成：高电平部分 t_1 和低电平部分 t_2 ，也就是 $T=t_1+t_2$ ，当使用占空比为 50% 信号时，也就是高低电平时间相等，也就是 $t_1=t_2=T/2=250\mu\text{s}$ 。那么 PA6 要发出 2KHz 频率源，就需要发出连续一系列高电平 $250\mu\text{s}$ 、低电平 $250\mu\text{s}$ 。高低电平通过取反 PA6 完成，也就是“GPIOA->ODR ^= 1<<6;”语句（异或逻辑关系）完成。

值得注意的是 PA6=1 时蜂鸣器通电，流过峰值约近 300mA，频率越低电流越大，频率越高电流越小；当 PA6=0 时，蜂鸣器上没有电流通过。那么蜂鸣器不发声时，我们一般采用“GPIOA->BRR = 1<<6;”来关闭蜂鸣器，节约功耗。

救护车警报声基础频率由持续 300ms 的 2KHz 频率源和持续 300ms 的 1.5KHz 频率源组成，发出指定频率方法有了，那么如何确定持续 300ms 呢？从上面的分析中我们发现发出频率 2KHz 的最基本时间片是 $T/2$ ($250\mu\text{s}$)，那 $300\text{ms}=300\text{ms}/250\mu\text{s}=1200$ 次 $T/2$ ，那我们就可以循环 1200 次半周期 ($T/2$)

来完成持续时间（音乐中的节拍）。同理 $f=1.5\text{KHz}$ 频率，那么它的周期 $T=1/f=666\mu\text{s}$ ， $T/2=333\mu\text{s}$ ，300ms 节拍需要 900 个 $T/2$ ，因此程序如下：

```
void jiuhu()    //救护车警报声
{
    short i;
    for(i=0;i<1200;i++) //300ms
    {
        GPIOA->ODR ^= 1<<6; //取反 PA6
        Delayus(250);    //250us, 2KHz
    }
    for(i=0;i<900;i++) //300ms
    {
        GPIOA->ODR ^= 1<<6; //取反 PA6
        Delayus(333);    //333us, 1.5KHz
    }
    GPIOA->BRR = 1<<6; //PA6=0, 关闭蜂鸣器
}
```

④ 枪声子程序

```
void qiang()    //枪声
{
    short t=325;
    for(t=170;t<325;t++) //300ms
    {
        GPIOA->ODR ^= 1<<6; //取反 PA6
        Delayus(t); //250us, 2KHz
    }
    GPIOA->ODR &= ~(1<<6); //GPIOA->BRR=1<<6; //PA6=0, 关闭蜂鸣器
}
```

⑤ 消防车警报声子程序

```
void xiaofang() //消防车警报声
{
    short i,t;
    for(t=555;t<833;t++)
    {
        for(i=0;i<(10000/t);i++) //300ms
        {
            GPIOA->ODR ^= 1<<6; //取反 PA6
            Delayus(t);
        }
    }
    for(t=833;t>555;t--)
    {
```

```

        for(i=0;i<(10000/t);i++)    //300ms
        {
            GPIOA->ODR ^= 1<<6; //取反 PA6
            Delayus(t);
        }
    }
    GPIOA->BRR = 1<<6; //PA6=0, 关闭蜂鸣器
}

```

4.5 蜂鸣器播放音乐

(1) 蜂鸣器可以播放音乐的原理

音乐中不同的音调实际上是发声体震动的速度(即频率)不同。所以要让蜂鸣发出不同的音调只需要使蜂鸣器开启和关闭的频率改变(即用与音调相应的频率不断使输出口在高低电平之间转换,类似于输出一个占空比为 50%而频率可变的方波)。可以想到,这可以通过定时来实现,在定时到达时切换蜂鸣器的开关,不断改变定时时间即可改变音调。

定时最好是采用定时器中断完成,在没有定时器或者像我们现在没用学习定时器时,也可以采用软件延时。

而节拍就是让蜂鸣器在固定的频率工作一段时间,只需要通过软件软件延时或定时器定时就可达到此效果。通过上述分析不难看出,音乐的音调和节拍在要求不严格或没有中断等事件时,完全可以由调整软件延时时间来完成。

(2) 音乐库数据结构

从上可见,每个音符是两部分组成:音调(频率)、节拍(发出频率时长),那我们可以构建音乐数组:音调 1、节拍 1,音调 2、节拍 2,.....,音调 n、节拍 n, 0、0。其中音调和节拍不可能是 0,因此我们可以用音调和节拍同时为 0 表示本曲结束;用 0、x 表示发声暂停 x 拍时间, $x \neq 0$ 。

(3) 音调常数 (频率)

/******	#define FAM 698
音调 (频率) 常数, 单位 Hz	#define SOM 784
*****/	#define LAM 880
//低音	#define SIM 988
#define DOL 262	//高音
#define REL 294	#define DOH 1046
#define MIL 330	#define REH 1175
#define FAL 349	#define MIH 1318
#define SOL 392	#define FAH 1397
#define LAL 440	#define SOH 1568
#define SIL 494	#define LAH 1760
//中音	#define SIH 1976
#define DOM 523	//暂停
#define REM 587	#define STOPNS 0
#define MIM 659	

(4) 节拍常数

```

/*****
拍数，节拍常数，单位 us/每分钟 120 拍
*****/
#define B41 2000000UL    //4 拍
#define B31 1500000UL    //3 拍
#define B21 1000000UL    //2 拍
#define B32 750000UL     //1.5 拍
#define B11 500000UL     //1 拍
#define B34 375000UL     //4 分之 3 拍
#define B12 250000UL     //2 分之 1 拍
#define B14 125000UL     //4 分之 1 拍

```

(5) 示例音乐库数组

```

u32 music1[]=
{
    LAM,B11,LAM,B11,LAM,B11,LAM,B14,DOH,B14,REH,B14,MIH,B14,
    LAM,B11,LAM,B11,LAM,B12,SOM,B12,SOM,B12,LAM,B12,
    LAM,B11,LAM,B11,LAM,B11,LAM,B14,DOH,B14,REH,B14,MIH,B14,
    LAM,B11,LAM,B11,LAM,B12,FAH,B12,FAH,B12,MIH,B12,
    LAM,B11,LAM,B11,LAM,B11,LAM,B14,DOH,B14,REH,B14,MIH,B14,
    LAM,B11,LAM,B11,LAM,B12,LAM,B12,SOM,B12,LAM,B12,
    LAM,B11,LAM,B11,LAM,B14,LAM,B14,DOH,B14,REH,B14,MIH,B14,
    LAM,B32,SOM,B12,SOM,B12,LAM,B12,LAM,B11,
    STOPNS,B32,
    LAM,B12,LAM,B12,SOM,B14,SOM,B14,SOM,B12,LAM,B12,
    SOM,B12,MIM,B12,MIM,B11,SOM,B14,MIM,B21,
    STOPNS,B32,
    LAM,B12,LAM,B12,SOM,B12,LAM,B12,SIM,B12,
    DOH,B11,SIM,B11,LAM,B12,SIM,B14,LAM,B14,SOM,B11,
    0,0//结束标志
}

```

(6) 播放函数

```

//播放一个音符
void OneNote(u32 fr,u32 cyus)
{
    u32 frus;
    u32 beatNS;
    if(fr==0)
    {
        GPIOA->BRR = 1<<6;    //频率输出低电平
        Delayus(cyus);
        return;
    }
    frus=1000000UL/fr;//音调频率的周期
    beatNS=cyus/frus;    //该节拍可以发出多少个音频周期
    while(beatNS)
    {

```

```

        GPIOA->BSRR = 1<<6;    //频率输出高电平
        Delayus(frus/2);
        GPIOA->BRR = 1<<6;    //频率输出低电平
        Delayus(frus/2);
        beatNS--;
    }
}

//播放一曲
void OneMusic(u32 *p)
{
    u32 i,j;
    while(1){
        i=*p++;
        j=*p++;
        if(i==0 && j==0) break; //结束播放
        OneNote(i,j);
    }
}

```

4.6 实验连线

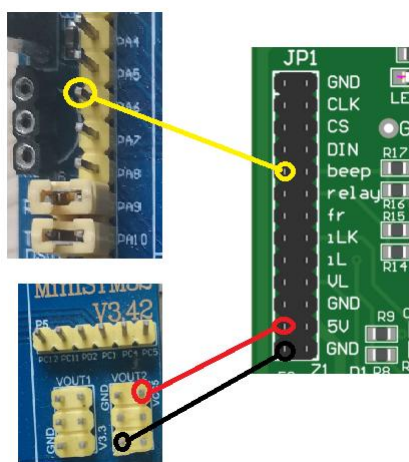


图 4.4 蜂鸣器接线图

接线如右图所示，需要从 miniSTM32 连接 3 根杜邦线到 miniIO，分别是：

红色线连接 5V 电源；

黑色线连接电源地 GND；

黄色线，连接 miniSTM32 的 PA6 到 miniIO 的 beep

同时短接 miniIO 板上的 JP3-beep 跳线帽。

4.7 ISP 下载程序

STM32 的程序下载有多种方法：USB、串口、JTAG、SWD 等，这几种方式，都可以用来给 STM32 下载代码。不过，我们最常用的，最经济的，就是通过串口给 STM32 下载代码。

STM32 的串口下载一般是通过串口 1 下载的，ALIENTEK miniSTM32 开发板，不是通过 RS232 串口下载的，而是通过自带的 USB 串口来下载。看起来像是 USB 下载(只需一根 USB 线，并不需要串口线)的，实际上，是通过 USB 转成串口，然后再下载的。

首先要在板子上设置一下，在板子上把 RXD 和 PA9(STM32 的 TXD)，TXD 和 PA10(STM32 的 RXD)通过跳线帽连接起来，这样我们就把 CH340G 和 MCU 的串口 1 连接上了。这里由于 ALIENTEK 这款开发板

自带了一键下载电路，所以我们并不需要去关心 BOOT0 和 BOOT1 的状态。



图 4.5 ISP 下载界面图

ISP 下载程序步骤如下：

- ① 打开相应 ISP 程序

双击 FlyMcu.exe 打开程序。

- ② 选择下载端口

选择板载 CH340，可能是端口 COM0~COM31 中的一个，如下所示：

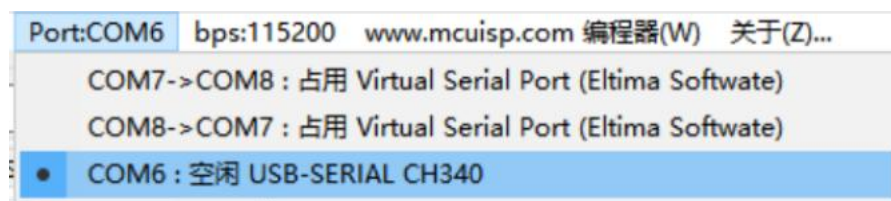


图 4.6 ISP 下载端口选择图

- ③ 选择“STMISP”

- ④ 测试芯片

点击“读器件信息(R)”，显示如图信息“向您报告，命令执行完毕，一切正常”说明测试通过。



图 4.7 芯片测试信息图

⑤ 选择下载的应用程序文件

点击选择打开编译好的*.hex 下载代码文件，如下图所示：

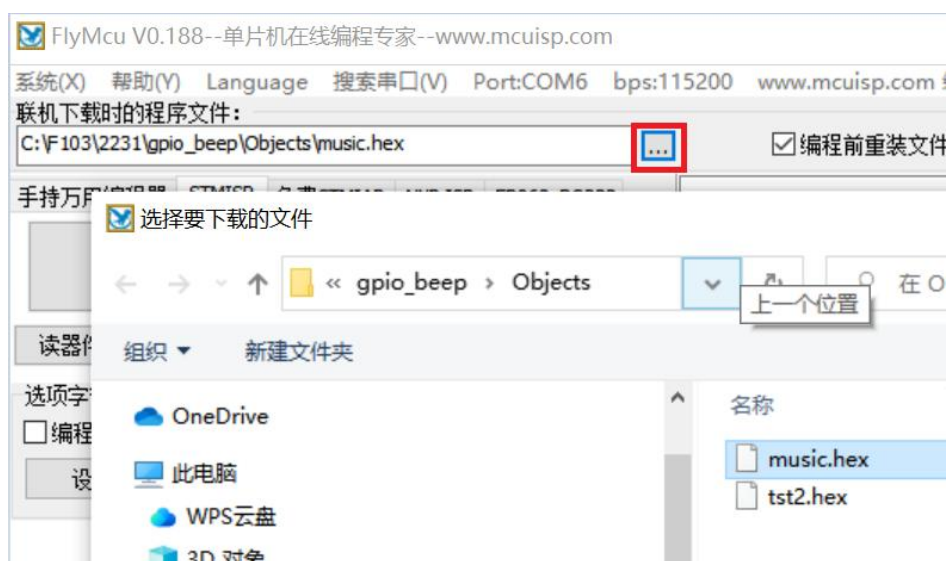


图 4.8 选择下载代码文件

⑥ ISP 下载程序

点击“开始编程(P)”开始下载程序，显示如图信息“向您报告，命令执行完毕，一切正常”说明已经正确下载，同时信息栏显示过程及相关参数。

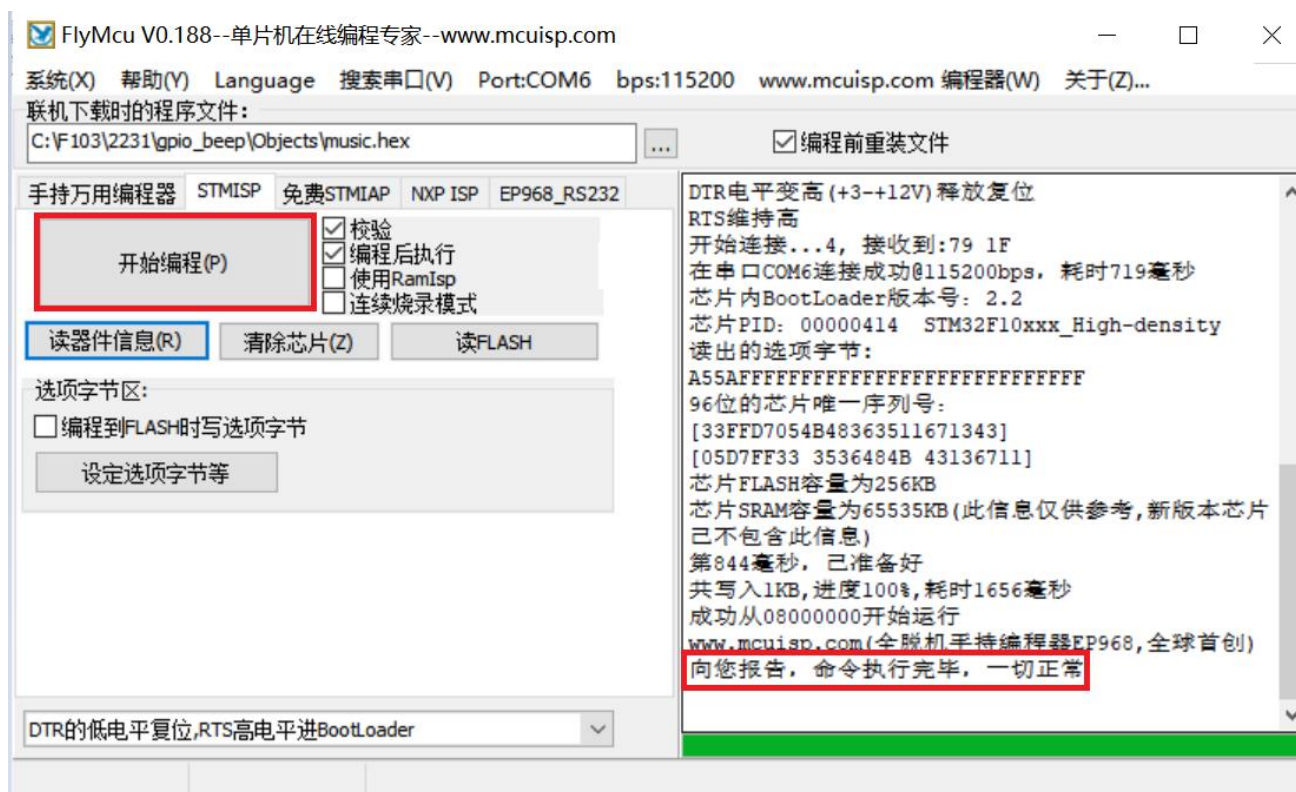


图 4.9 ISP 下载程序

4.8 实验步骤

- ① 画出程序流程图;
- ② 编写源程序;
- ③ 下载调试程序;
- ④ 修改源程序;
- ⑤ 撰写实验报告。

本章有 2 次实验课完成:

第一次实验课完成 ISP 下载程序, 画出流程图及编写: 救护车、消防车、枪声源程序, 下载调试、编写实验报告。

第二次实验课完成音乐程序设计, 画出流程图及编写播放至少 2 首音乐程序, 下载调试、编写实验报告。

4.9 程序流程图

程序流程图就是用规定的符号描述一个程序中所需的各种操作或者判断的图表。通过对数据的整理、计算和分析, 来对计算机的运行步骤和内容标识出来, 程序流程图是一种图形化的表示方法, 将计算机所用的程序数据, 通过图形的方式表现出来, 使得数据更加的直观、清晰、更容易理解。

程序流程图举例: 救护车发声实验流程图救护车警报声为连续发出 300ms 的 2KHz 频率源, 然后发出 300ms 的 1.5KHz 频率源, 其程序流程图如下图所示。大家可以以此为参考, 在以后实验前画出所有流程图。

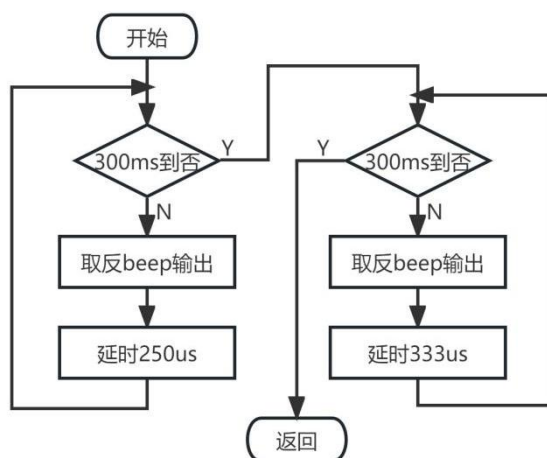


图 4.10 救护车发声函数流程图

4.10 C 语言源程序

```

int main(void)
{
    short i, j;
    BEEP_init(); //初始化 beep 对应的 PA6
    while(1)
    {
        for(j=0; j<5; j++)
        {
            for(i=0; i<3; i++) {qiang(); Delayus(500);}
            Delayus(500000);
        }
        for(i=0; i<5; i++) jiuhu();
        Delayus(200000);
        xiaofang();
        Delayus(200000);
    }
}
    
```

4.11 思考题

回答如下思考：程序中产生频率和节拍都采用软件延时，软件延时有何特点？还有哪些方式可以完成频率和节拍延时？

5. 实验 5—按键输入实验

前两次实验，我们学习了 STM32 的 IO 口作为输出的使用，这一章，我们将向大家介绍如何使用 STM32 的 IO 口作为输入用。在本章中，我们将利用板载的 3 个按键，来控制板载的两个 LED 的亮灭。通过本章的学习，你将了解到 STM32 的 IO 口作为输入口的使用方法。

5.1 按键输入原理

STM32 的 IO 口做输入使用的时候，是通过调用函数 `GPIO_ReadInputDataBit()` 来读取 IO 口的状态寄存器 IDR 来实现的。了解了这点，就可以开始我们的代码编写了。

我们将通过 MiniSTM32 开发板上载有的 3 个按钮 (KEY0/KEY1/WK_UP)，来控制板上的 2 个 LED，其中 KEY0 控制 DS0，按一次亮，再按一次，就灭。KEY1 控制 DS1，效果同 KEY0。WK_UP 按键则同时控制 DS0 和 DS1，按一次，他们的状态就翻转一次。

本实验用到的硬件资源有：

- 1) 指示灯 DS0、DS1
- 2) 3 个按键：KEY0、KEY1 和 KEY_UP。

DS0、DS1 和 STM32 的连接在上一章已经介绍了，在 MiniSTM32 开发板上的按键 KEY0 连接在 PC5 上、KEY1 连接在 PA15 上、WK_UP 连接在 PA0 上。如图 5.1 所示：

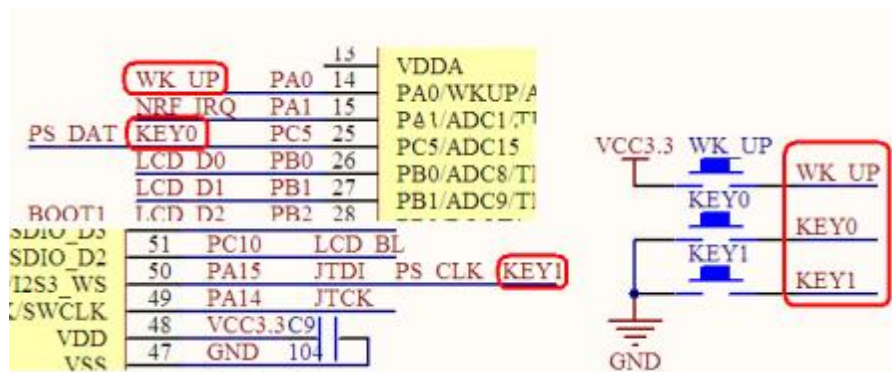


图 5.1 按键与 STM32 连接原理图

这里需要注意的是：KEY0 和 KEY1 是低电平有效的，而 WK_UP 是高电平有效的，除了 KEY1 有上拉电阻(与 JTDI 共用)，其他两个都没有上下拉电阻，所以需要在 STM32 内部设置上下拉。

从图中我们不难看出，KEY0 和 KEY1 需要设置为上拉工作模式，WK_UP 需要设置为下拉工作模式。

按键在按下和弹起来过程中会产生不稳定的按下、非按下两种交替状态，如何消除这种抖动现象呢？我们以 KEY0 为例，先看看 KEY0 按下和弹起瞬间按键引脚 PC5 电平变化曲线：

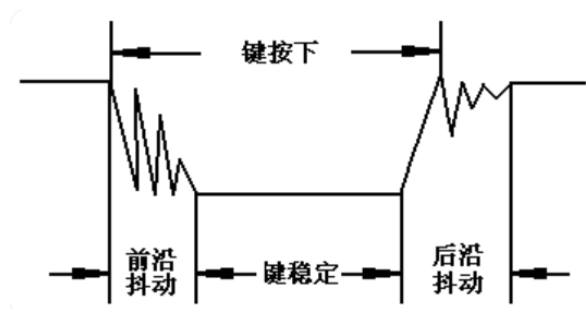


图 5.2 按键信号波形变化图

一般前后沿抖动时间一般在 5~20ms 之间,那么按键消抖程序常常在第一次检测到按键按下后延时 10ms 再次检测确认按键已经按下,当前后 2 次均检测到按键按下才认为按键按下,否则认为按键未按下;按键弹起消抖方法与此一致。

5.2 实验设备与软件环境

设备:

- ① PC 电脑, 至少需要 2 个 type-A 的 USB 插口;
- ② ALIENTEK MiniSTM32 实验套件

环境: window10、keil 5 开发软件、仿真器驱动程序

5.3 实验任务

- ① 按给出的演示程序验证: KEY0 按下发光二极管 DS0 亮, KEY0 弹起来 DS0 灭;
- ② 自行编写程序: KEY1 按下发光二极管 DS1 亮, KEY1 弹起来 DS1 灭; WK_UP 按下发光二极管 DS0 亮、DS1 灭, WK_UP 弹起来 DS0 灭、DS1 亮;
- ③ 自行编写程序: KEY0 按第一次发光二极管 DS0 亮, KEY0 按第二次次发光二极管 DS0 灭, 以此类推; KEY1 按第一次发光二极管 DS1 亮, KEY1 按第二次次发光二极管 DS1 灭, 以此类推; WK_UP 按下发按第一次发光二极管 DS0、DS1 亮, WK_UP 按第二次次发光二极管 DS0、DS1 灭, 以此类推;
- ④ 解决按键延时去抖动。

5.4 相关库函数分析

- ① 读入端口输入数据 (IDR), 库函数定义如下:

```
uint16_t GPIO_ReadInputData(GPIO_TypeDef* GPIOx)
{
    /* Check the parameters */
    assert_param(IS_GPIO_ALL_PERIPH(GPIOx));

    return ((uint16_t)GPIOx->IDR);
}
```

GPIO_ReadInputData 函数一次读入 GPIOx 端口 16 个引脚的 16 位数据。

- ② 读入端口的某一位管脚的位数据 (IDR.GPIO_Pin), 库函数定义如下:

```
uint8_t GPIO_ReadInputDataBit(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)
{

```

```
uint8_t bitstatus = 0x00;

/* Check the parameters */
assert_param(IS_GPIO_ALL_PERIPH(GPIOx));
assert_param(IS_GET_GPIO_PIN(GPIO_Pin));

if ((GPIOx->IDR & GPIO_Pin) != (uint32_t)Bit_RESET)
{
    bitstatus = (uint8_t)Bit_SET;
}
else
{
    bitstatus = (uint8_t)Bit_RESET;
}

return bitstatus;
}
```

GPIO_ReadInputDataBit 函数其实也是同时读入 IDR 的 16 位，然后使用 if...else 结构判断指定的 GPIO_Pin，该位为 1 则函数返回 1，该位为 0 则返回 0。也就是说读入端口的某一位管脚的位数据也可以使用读入端口输入数据 GPIO_ReadInputData 函数，然后判断其中指定的某一位或指定的多位。

5.5 演示程序分析

- ① 按键端口引脚初始化自定义函数如下：

```
void Key0Init()
{
    GPIO_InitTypeDef g2;    //定义 GPIO 初始化结构体变量 g2
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOC, ENABLE);    //使能 PC 端口时钟
    g2.GPIO_Mode = GPIO_Mode_IPU;    //上拉输入
    g2.GPIO_Pin = GPIO_Pin_5;    //PC5
    GPIO_Init(GPIOC, &g2);    //初始化 PC5
}
```

- ② 按键输入宏定义：

```
#define KEY0 GPIO_ReadInputDataBit(GPIOC, GPIO_Pin_5)
```

当我们经常使用某段小程序完成简单功能时，为了增强可阅读和可维护性，我们经常使用宏定义来完成特点功能。比如我们这里定义 KEY0 来实现按键 KEY0 的读取。

- ③ 发光二极管输出亮和灭宏定义：

```
#define DS00N GPIO_ResetBits(GPIOA, GPIO_Pin_8)
#define DS00FF GPIO_SetBits(GPIOA, GPIO_Pin_8)
```

这里宏 DS00N 完成发光二极管 DS0 发光，宏 DS00FF 完成熄灭发光二极管 DS0。

- ④ 主程序如下：

```
LedInit();
KeyInit();
while(1)
{
    if(KEY0==0)
        LED1ON;
    else
        LED1OFF;
```

```
}
```

该程序是主程序的核心部分，完善后调试运行时 KEY0 按下时 DS0 就亮，KEY0 弹起来时 DS0 就灭。

⑤ 延时去抖动程序：

```
if(KEY0==0)
{
    Delay(10); //延时 10ms
    if(KEY0==0) LED1ON; //再次判键 KEY0 任然按下则确认 KEY0 按下
}
else
{
    Delay(10); //延时 10ms
    if(KEY0==1) LED1OFF; //再次判键 KEY0 任然弹起则确认 KEY0 弹起
}
```

该程序和④中程序除了按键去抖动，其它功能完全一样。

5.6 实验步骤

- ① 画出程序流程图；
- ② 编写源程序；
- ③ 下载调试程序；
- ④ 修改源程序；
- ⑤ 撰写实验报告。

5.7 思考题

回答如下思考：工业实际 GPIO 输入过程中存在干扰，就像按键过程中的抖动现象，软件、硬件设计过程中有哪些消除这种干扰？

6. 实验 6—外部中断实验

这一章，我们将向大家介绍如何使用 STM32 的外部输入中断。在前面几章的学习中，我们掌握了 STM32 的 IO 口最基本的操作。本章我们将介绍如何将 STM32 的 IO 口作为外部中断输入，在本章中，我们将以中断的方式，实现按键控制发光二极管。

6.1 STM32 外部中断简介

STM32 的每个 IO 都可以作为外部中断的中断输入口，这点也是 STM32 的强大之处。STM32F103 的中断控制器支持 19 个外部中断/事件请求。每个中断设有状态位，每个中断/事件都有独立的触发和屏蔽设置。STM32F103 的 19 个外部中断为：

另外四个 EXTI 线的连接方式如下：

- EXTI 线 16 连接到 PVD 输出
- EXTI 线 17 连接到 RTC 闹钟事件
- EXTI 线 18 连接到 USB 唤醒事件
- EXTI 线 19 连接到以太网唤醒事件(只适用于互联型产品，STM32F103 无此功能)

STM32 供 IO 口使用的中断线只有 16 个，但是 STM32 的 IO 口却远远不止 16 个，那么 STM32 是怎么把 16 个中断线和 IO 口一一对应起来的呢？于是 STM32 这样设计，GPIO 的管教 $\text{GPIOx.0} \sim \text{GPIOx.15}$ ($x=A, B, C, D, E, F, G$) 分别对应中断线 $15 \sim 0$ 。这样每个中断线对应了最多 7 个 IO 口，以线 0 为例：它对应了 GPIOA.0 、 GPIOB.0 、 GPIOC.0 、 GPIOD.0 、 GPIOE.0 、 GPIOF.0 、 GPIOG.0 。而中断线每次只能连接到 1 个 IO 口上，这样就需要通过配置来决定对应的中断线配置到哪个 GPIO 上了。下面我们看看 GPIO 跟中断线的映射关系图如图 6.1 所示。

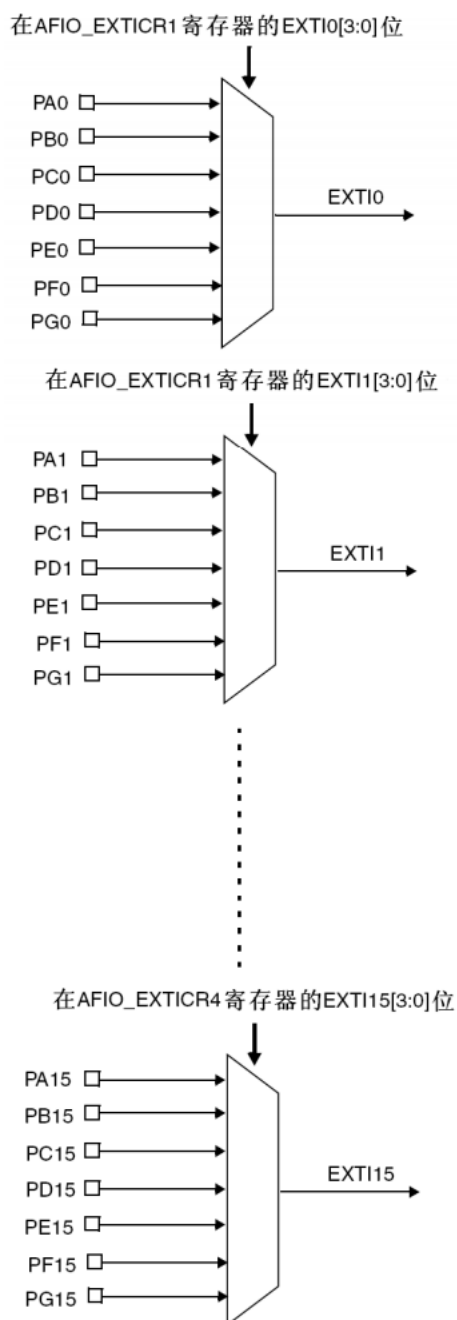


图 6.1 GPIO 和中断线的映射关系图

6.2 实验设备与软件环境

设备：

- ① PC 电脑，至少需要 2 个 type-A 的 USB 插口；
- ② ALIENTEK MiniSTM32 实验套件

环境： window10、keil 5 开发软件、仿真器驱动程序

6.3 实验任务

- ① 按给出的演示程序验证：WK_UP 第一次按下发光二极管 DS0 和 DS1 同时亮，WK_UP 第二次按下 DS0 和 DS1 灭；
- ② 使用外中断自行编写程序：第一次按下 KEY0 发光二极管 DS0 亮，第二次按下 KEY0 发光二极管 DS0 灭，以此类推；KEY1 按第一次发光二极管 DS1 亮，KEY1 按第二次次发光二极管 DS1 灭；
- ③ 考虑 EXTI9_5_IRQHandler()和 EXTI15_10_IRQHandler()中断中有多个线中断判断，重新编写程序。

6.4 演示程序

完成功能：WK_UP 第一次按下发光二极管 DS0 和 DS1 同时亮，WK_UP 第二次按下 DS0 和 DS1 灭。

6.4.1 工程结构

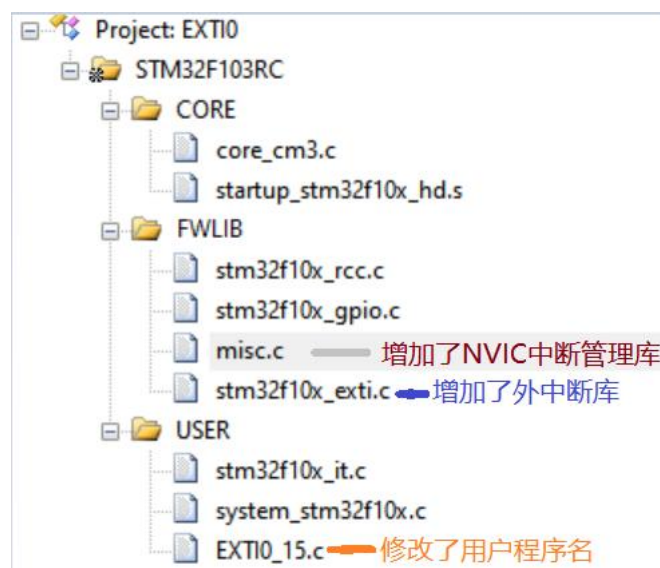


图 6.2 工程管理结构

本工程中我们将用到 NVIC 和 EXTI 两个设备，因此在库函数目录 FWLIB 下需要添加两个库，NVIC 对应的库 misc.c 和 EXTI 对应的库 stm32f10x_exti.c。具体做法是鼠标右键点击 ，选择

Add Existing Files to Group 'FWLIB'...

，在目录 “\STM32F10x_FWLib\src” 下添加该 2 个文件，添加后如图 6.2 所示。

6.4.2 外中断编程步骤及演示程序

外中断编程按以下 (0) ~ (4) 共五个步骤完成:

(0) 设置中断优先级分组 (1) GPIO设置 (2) EXTI配置 (3) NVIC初始化 (4) 中断服务函数

图 6.3 EXTI 编程步骤

演示程序如下:

```
//(4) 中断服务函数
void EXTI0_IRQHandler(void)
{
    Delay(10); //按键延时去抖动
    if( GPIO_ReadInputDataBit(GPIOA,GPIO_Pin_0)==1)//直接读取 PA0 判断是否真实的按键
    {
        if(GPIO_ReadOutputDataBit(GPIOA, GPIO_Pin_8)) GPIO_ResetBits(GPIOA,GPIO_Pin_8);
        else GPIO_SetBits(GPIOA,GPIO_Pin_8);
        if(GPIO_ReadOutputDataBit(GPIOD, GPIO_Pin_2)) GPIO_ResetBits(GPIOD,GPIO_Pin_2);
        else GPIO_SetBits(GPIOD,GPIO_Pin_2);
    }
    EXTI_ClearITPendingBit(EXTI_Line0); //清除 EXTI_Line0 中断标志
}
int main(void)
{
    GPIO_InitTypeDef    g1;
    EXTI_InitTypeDef     g2;
    NVIC_InitTypeDef     g3;
    LED_Init();
    //(0) 设置中断优先级分组
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);    // 设置中断优先级分组 2
    //(1) GPIO 设置
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);    //使能 PA 端口时钟
    g1.GPIO_Pin = GPIO_Pin_0;//PA0
    g1.GPIO_Mode = GPIO_Mode_IPD; //下拉输入
    GPIO_Init(GPIOA, &g1);    //初始化 GPIOA.0
    //(2) EXTI 配置
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_AFIO,ENABLE); //打开 AFIO 时钟
    GPIO_EXTILineConfig(GPIO_PortSourceGPIOA, GPIO_PinSource0); //设置 IO 口与中断线的映射关系
    g2.EXTI_Line = EXTI_Line0;
    g2.EXTI_Mode = EXTI_Mode_Interrupt;
    g2.EXTI_Trigger = EXTI_Trigger_Rising;
    g2.EXTI_LineCmd = ENABLE;
    EXTI_Init(&g2); //选择 AFIO 外中断 0 引脚
    //(3) NVIC 初始化
    g3.NVIC_IRQChannel = EXTI0_IRQn;    //使能按键所在的外部中断通道
    g3.NVIC_IRQChannelPreemptionPriority = 0x02;    //抢占优先级 2
    g3.NVIC_IRQChannelSubPriority = 0x02;    //子优先级 2
    g3.NVIC_IRQChannelCmd = ENABLE;    //使能外部中断通道
    NVIC_Init(&g3);    //根据 g3 中指定的参数初始化外设 NVIC 寄存器
    while(1)
    {
    }
}
```

6.4.3 多线外中断判断

中断线 EXTI5~EXTI9 共用一个中断源，那么如何在中断服务函数 void EXTI9_5_IRQHandler(void)中区分是哪个中断线引起的中断呢？STM32 提供了中断挂起寄存器 EXTI->PR 寄存器，标识哪个中断产生，我们可以直接使用寄存器 EXTI->PR，也可以使用库函数 EXTI_GetFlagStatus 来读取，该函数定义如下：

```
FlagStatus EXTI_GetFlagStatus(uint32_t EXTI_Line)
{
    FlagStatus bitstatus = RESET;
    /* Check the parameters */
    assert_param(IS_GET_EXTI_LINE(EXTI_Line));

    if ((EXTI->PR & EXTI_Line) != (uint32_t)RESET)
    {
        bitstatus = SET;
    }
    else
    {
        bitstatus = RESET;
    }
    return bitstatus;
}
```

通过 EXTI->PR 判断哪条先中断函数结构如下：

```
void EXTI9_5_IRQHandler(void)
{
    u32 temp;
    temp=EXTI->PR;    //读入中断挂起标志（Line 中断标志）
    if(temp&EXTI_Line5)    //判断是否 Line5 中断
    {
        .....
        EXTI_ClearITPendingBit(EXTI_Line5);    //清除 EXTI_Line5 中断标志
    }
    .....
    else if(temp&EXTI_Line9)    //判断是否 Line9 中断
    {
        .....
        EXTI_ClearITPendingBit(EXTI_Line9);    //清除 EXTI_Line9 中断标志
    }
}
```

6.5 实验步骤

- ① 画出程序流程图；
- ② 编写源程序；
- ③ 下载调试程序；
- ④ 修改源程序；
- ⑤ 撰写实验报告。

6.6 思考题

- (1) 回答如下思考：GPIO 处理的查询方式和中断方式有何特点？以按键处理为例说明 EXTI 方式有何优点，EXTI 与查询方式比较需要增加哪些硬件？
- (2) 回答如下思考，给出解决方法：多个中断线共用一个中断源，STM32 提供了中断挂起寄存器 EXTI->PR 判断中断源。那么 STM32F103 外中断有上升下降沿同时中断，也就是当初初始化时 `g2.EXTI_Trigger = EXTI_Trigger_Rising_Falling`，我们又该如何区分是上升沿还是下降沿引起的中断呢？STM32F103 外中断有没有提供寄存器判断是上升下降沿，如果没有，我们又该如何处理？

7. 实验 7—串口发送实验

前面章介绍了 STM32 的 IO 口操作。这一章我们将学习 STM32 的串口，教大家如何使用 STM32 的串口来发送数据。本章将实现如下功能：STM32 通过串口向上位机发送信息。

7.1 STM32 的 USART 简介

STM32F103RC 有 3 个 USART 和 2 个 UART，串口库文件 stm32f10x_usart.c 中定义了如下 USART 结构体指针：

```
#define USART1          ((USART_TypeDef *) USART1_BASE)
#define USART2          ((USART_TypeDef *) USART2_BASE)
#define USART3          ((USART_TypeDef *) USART3_BASE)
#define UART4           ((USART_TypeDef *) UART4_BASE)
#define UART5           ((USART_TypeDef *) UART5_BASE)
```

本章我们着重介绍 USART。

STM32 串口有关的常用库函数如下：

(1) 串口初始化库函数

void USART_Init(USART_TypeDef* USARTx, USART_InitTypeDef* USART_InitStruct)，初始化函数定义参见标准函数库。

其中参数 1：USARTx 的 x 取指 1~5；参数 2 为初始化结构体指针，包含 6 个成员：波特率、字长、停止位、奇偶校验位、流控、收发模式，其定义如下：

```
typedef struct
{
    uint32_t USART_BaudRate;    //波特率
    uint16_t USART_WordLength;  //字长
    uint16_t USART_StopBits;    //停止位数
    uint16_t USART_Parity;      //奇偶校验
    uint16_t USART_Mode;        //接收和发送（收发模式）
    uint16_t USART_HardwareFlowControl; //数据流控制
} USART_InitTypeDef;
```

① 波特率

常见的波特率取值有 1200、2400、4800、9600、14400、19200、38400、43000、57600、76800、115200、128000、230400、256000、460800、921600、1382400、2250000、4500000 等，单位为 bps；

② 字长

字长有 8bit 和 9bit，常数定义如下：

```
#define USART_WordLength_8b          ((uint16_t)0x0000)
#define USART_WordLength_9b          ((uint16_t)0x1000)
```

③ 停止位

STM32 支持 0.5bit、1bit、1.5bit、2bit，定义如下：

```
#define USART_StopBits_1              ((uint16_t)0x0000)
#define USART_StopBits_0_5            ((uint16_t)0x1000)
#define USART_StopBits_2              ((uint16_t)0x2000)
#define USART_StopBits_1_5            ((uint16_t)0x3000)
```

④ 奇偶校验位

STM32 支持无校验、偶校验、奇校验，分别定义如下：

```
#define USART_Parity_No          ((uint16_t)0x0000)
#define USART_Parity_Even        ((uint16_t)0x0400)
#define USART_Parity_Odd         ((uint16_t)0x0600)
```

⑤ 接收和发送控制

STM32 串口可以是单独的接收 (USART_Mode_Rx)、单独的发送 (USART_Mode_Tx)，也可以是收发模式 (USART_Mode_Rx|USART_Mode_Tx)，常量定义如下：

```
#define USART_Mode_Rx            ((uint16_t)0x0004)
#define USART_Mode_Tx            ((uint16_t)0x0008)
```

⑥ 数据流控制

这里讲到的“流”，指的是数据流；在数据通信中，流控制是管理两个节点之间数据传输速率的过程，以防止出现接收端的数据缓冲区已满，而发送端依然继续发送数据，所导致数据丢失。

当接收端的数据缓冲区已满，无法处理数据来时，就发出“不再接收”的信号，发送端则停止发送，直到发送端收到“可以继续发送”的信号再发送数据。计算机中常用的两种流控制分别是硬件流控制 (RTS/CTS、DTR/DSR 等)和软件流控制(XON /XOFF)。

我们在编写程序时往往不使用数据流控制，也就是 USART_HardwareFlowControl_None。

```
#define USART_HardwareFlowControl_None ((uint16_t)0x0000)
#define USART_HardwareFlowControl_RTS ((uint16_t)0x0100)
#define USART_HardwareFlowControl_CTS ((uint16_t)0x0200)
#define USART_HardwareFlowControl_RTS_CTS ((uint16_t)0x0300)
```

当我们给串口初始化结构体变量 (如: S1) 的 6 个以上成员赋值后就可以调用串口初始化函数初始化串口，例如: USART_Init(USART1, &S1)。

(2) 发送库函数

标准库函数中发送函数定义如下：

```
void USART_SendData(USART_TypeDef* USARTx, uint16_t Data)
{
    /* Check the parameters */
    assert_param(IS_USART_ALL_PERIPH(USARTx));
    assert_param(IS_USART_DATA(Data));

    /* Transmit Data */
    USARTx->DR = (Data & (uint16_t)0x01FF);
}
```

不管发送寄存器 DR 是否空，该函数直接写发送寄存器 DR，为了避免写入覆盖前面数据，正常情况下我们需要在调用该函数前先读取状态寄存器，判断发送寄存器 DR 是否可写，如果可以我们才调用该发送函数发送一个字节。

(3) 接收库函数

标准库函数中接收函数定义如下：

```
uint16_t USART_ReceiveData(USART_TypeDef* USARTx)
{
    /* Check the parameters */
```

```

    assert_param(IS_USART_ALL_PERIPH(USARTx));
    /* Receive Data */
    return (uint16_t)(USARTx->DR & (uint16_t)0x01FF);
}

```

同样，不管接收寄存器 DR 是否收到数据，该函数直接读接收寄存器 DR，为了避免读到无效数据，正常情况下我们需要在调用该函数前先读取状态寄存器，判断寄存器 DR 是否收到数据，如果接收到，我们才调用该接收函数接收一个字节。

(4) 读取串口状态函数

标准库函数中其函数定义如下：

```

FlagStatus USART_GetFlagStatus(USART_TypeDef* USARTx, uint16_t USART_FLAG)
{
    FlagStatus bitstatus = RESET;
    /* Check the parameters */
    assert_param(IS_USART_ALL_PERIPH(USARTx));
    assert_param(IS_USART_FLAG(USART_FLAG));
    /* The CTS flag is not available for UART4 and UART5 */
    if (USART_FLAG == USART_FLAG_CTS)
    {
        assert_param(IS_USART_123_PERIPH(USARTx));
    }
    if ((USARTx->SR & USART_FLAG) != (uint16_t)RESET)
    {
        bitstatus = SET;
    }
    else
    {
        bitstatus = RESET;
    }
    return bitstatus;
}

```

其中发送完成的判断为：USART_GetFlagStatus(USART1,USART_FLAG_TC) == SET

接收到数据的判断为：USART_GetFlagStatus(USART1,USART_FLAG_RXNE) == SET

7.2 实验设备与软件环境

设备：

- ① PC 电脑，至少需要 2 个 type-A 的 USB 插口；
- ② 杜邦线；
- ③ ALIENTEK MiniSTM32 实验套件

环境： window10、keil 5 开发软件、仿真器驱动程序、PC 串口调试软件

PC 串口调试工具软件非常多，我们简单介绍 XCOM 软件，其界面如下：

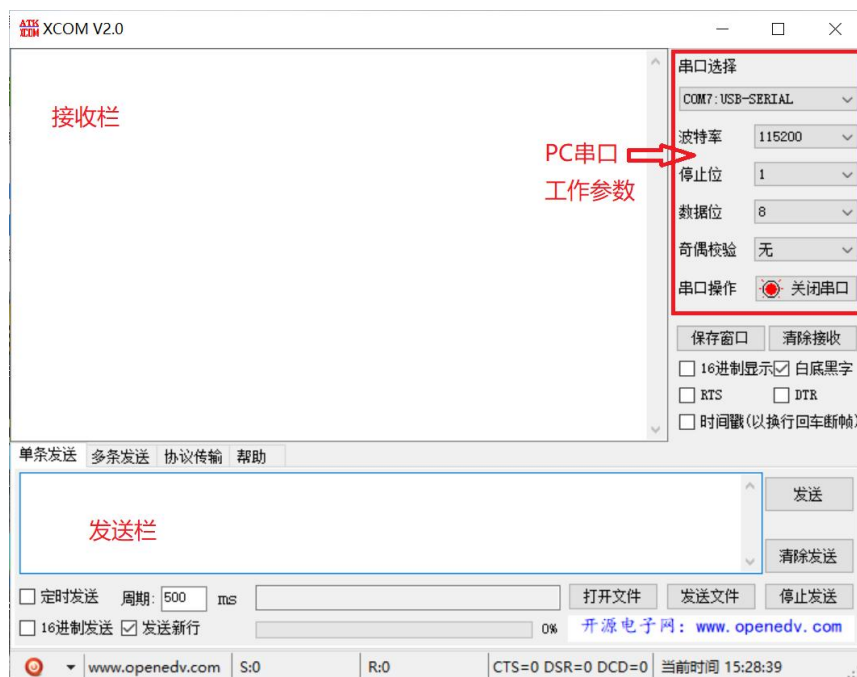


图 7.1 XCOM 界面图

XCOM 串口调试助手是一种常用的串口数据通信工具，它可以通过串口连接与目标设备进行通信，以实现数据的发送与接收。通常，在嵌入式系统开发、电子电路设计、自动化控制等领域都会使用到串口调试助手。串口调试助手可以对串口数据进行实时监测、调试和记录，方便用户进行数据分析和故障排查。

7.3 实验任务

- ① 按给出的演示程序验证：发送固定的字符串到 PC 串口；
- ② 学习 printf 的使用；
- ③ 查询方式按键消息发送：该方式要解决 3 个按键状态变化时发送一次消息，而无变化时不发送；
- ④ 外中断方式按键消息发送：该方式同样在 3 个按键状态变化时发送一次消息该，但按键检测采用外中断的上下沿中断完成。

7.4 演示程序

完成功能：发送固定的字符串到 PC 串口和 printf 函数使用方法。

7.4.1 工程结构

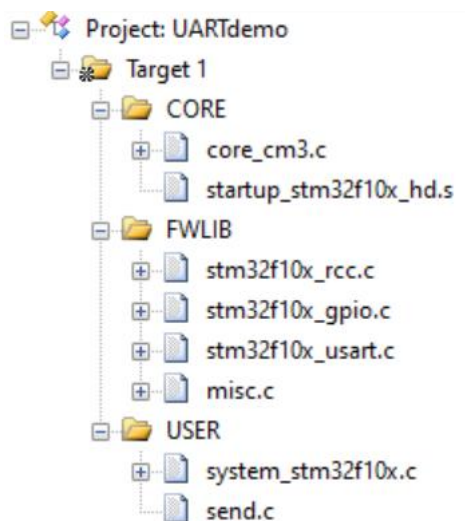


图 7.2 工程管理结构

本工程中我们将用到 USART，因此在 FWLIB 管理目录下需要添加“stm32f10x_usart.c”文件库，添加后如图 7.2 所示。

7.4.2 串口初始化过程

- | | |
|--------------------------|----------------------|
| ① 时钟使能：GPIO 时钟、串口复用时钟使能； | ③ 给初始化结构体成员赋值并初始化串口； |
| ② GPIO 端口模式设置； | ④ 使能串口。 |

串口初始化同样有专用的串口初始化结构体类型，它具有 6 个成员：波特率、字长、停止位、奇偶校验位、流控、收发模式，具体初始化参照 7.4.3，逐步体会初始化函数 `void uart_init(u32 bound)`。

7.4.3 发送字符串

发送字符串演示程序如下：

```
#include "stm32f10x.h"
void uart_init(u32 bound)
{
    GPIO_InitTypeDef S1;
    USART_InitTypeDef S2;
    //①时钟使能：GPIO 时钟使能、串口复用时钟使能
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1 | RCC_APB2Periph_GPIOA, ENABLE);
    //②GPIO 端口模式设置
    S1.GPIO_Pin = GPIO_Pin_9;           //USART1_TX PA.9
    S1.GPIO_Speed = GPIO_Speed_50MHz;
    S1.GPIO_Mode = GPIO_Mode_AF_PP; //复用推挽输出
    GPIO_Init(GPIOA, &S1);           //初始化 GPIOA.9
    S1.GPIO_Pin = GPIO_Pin_10;          //USART1_RX PA.10
    S1.GPIO_Mode = GPIO_Mode_IN_FLOATING; //浮空输入
    GPIO_Init(GPIOA, &S1);           //初始化 GPIOA.10
    //③串口参数初始化，给初始化结构体成员赋值并初始化
    S2.USART_BaudRate = bound;          //波特率设置
    S2.USART_WordLength = USART_WordLength_8b; //字长为 8 位
    S2.USART_StopBits = USART_StopBits_1; //一个停止位
    S2.USART_Parity = USART_Parity_No;    //无奇偶校验位
    S2.USART_HardwareFlowControl = USART_HardwareFlowControl_None; //无硬件数据流控制
    S2.USART_Mode = USART_Mode_Rx | USART_Mode_Tx; //收、发模式
    USART_Init(USART1, &S2);           //初始化串口
    //④使能串口
    USART_Cmd(USART1, ENABLE);
}
void sendbyte(u8 ch)
{
    // while((USART1->SR&0X40)==0); //等待前一字符发送完毕
    // USART1->DR = (u8) ch;        //发送单个字符 ch
    while(USART_GetFlagStatus(USART1, USART_FLAG_TC) != SET); //等待前一字符发送完毕
    USART_SendData(USART1, ch); //发送单个字符 ch
}
void sendstring(char *p)
{
    while(*p != '\0') sendbyte(*p++);
}
int main(void)
{
    char mss[] = "USART1 is ready!\r\n";
    uart_init(115200);
    sendstring(mss); //发送 mss
    while(1)
    {
        ; //空循环
    }
}
```

7.4.4 printf 函数使用

C 语言中，printf 函数默认的输出位置是 stdout，即调试终端。但是对于单片机来说，没有 stdout，所以一般将其重定向至串口来作输出。

printf 函数包含在"stdio.h"头文件中，是 C 语言中常用的输出函数，其功能是将信息输出到标准控制台，C 语言中 printf 函数输出默认的输出设备是显示器。

printf 函数属于系统调用，在不同编译环境下的实现方法略有不同，但其本质上会调用一个向 stdout 输出字符的系统调用函数。在 keil MDK 环境下，这个系统函数是 fputc，printf 主要通过调用 fputc 函数来实现字符输出。如果我们想把输出的设备转到串口，我们需要重定向输出设备到串口，也就是改写 fputc 函数输出到指定的串口，重定向程序如下：

```
//以下 20 行是 printf 串口调试所需程序
#pragma import(__use_no_semihosting)
//标准库需要的支持函数
struct __FILE
{
    int handle;
};
FILE __stdout;
//定义_sys_exit()以避免使用半主机模式
void _sys_exit(int x)
{
    x = x;
}
//重定义 fputc 函数
int fputc(int ch, FILE *f)
{
    while(USART_GetFlagStatus(USART1,USART_FLAG_TC)!=SET); //等待前一字符发送完毕
    USART_SendData(USART1,ch); //发送单个字符 ch
    return ch;
}
```

如果我们想在 7.4.3 程序中用 printf 循环输出信息至串口，可以在“//空循环位置”加入如下程序：

```
Delaysms(1000);
t++;
printf("The program has been run %d times\r\n",t);
```

当然需要在前面编写软件延时函数和定义整形类变量 t。

7.5 实验步骤

- ① 画出程序流程图；
- ② 编写源程序；

- ③ 下载调试程序；
- ④ 修改源程序；
- ⑤ 撰写实验报告。

7.6 思考题

- (1) 回答如下思考：printf 函数在我们调试程序时有哪些帮助？与 SWD 调试相比有哪些特点？

8. 实验 8—UART 中断接收的数学运算实验

前一章介绍了 STM32 的串口来发送数据。本章将实现串口接收和发送。

8.1 实验设备与软件环境

设备：

- ④ PC 电脑，至少需要 2 个 type-A 的 USB 插口；
- ⑤ 杜邦线；
- ⑥ ALIENTEK MiniSTM32 实验套件

环境： window10、keil 5 开发软件、仿真器驱动程序、PC 串口调试软件

8.2 查询方式接收发送

(1) 串口查询接收

STM32 串口查询接收，首先查询状态寄存器 USARTx->SR 中的接收数据就绪位 USART_FLAG_RXNE，如果该位为 0 就继续等待，如果该位为 1 就读取串口数据。

库函数方式查询并接收程序如下：

```
while(USART_GetFlagStatus(USART1, USART_FLAG_RXNE) != SET);    //等待接收到数据
ch=USART_ReceiveData(USART1);    //接收单个字符
```

寄存器方式查询并接收程序如下：

```
while((USART1->SR&0X20)==0);    //等待接收到数据
ch=USART1->DR;    //接收单个字符
```

寄存器方式接收函数如下：

```
char receivebyte()
{
    while((USART1->SR&0X20)==0); //等待接收到数据
    return USART1->DR;    //接收单个字符
}
```

(2) 串口查询发送

STM32 串口查询发送，首先查询状态寄存器 USARTx->SR 中的发送完成位 USART_FLAG_TC，如果该位为 0 就继续等待，如果该位为 1 就向串口发送一个字节。

库函数方式查询并发送程序如下：

```
while(USART_GetFlagStatus(USART1, USART_FLAG_TC) != SET); //等待上一字节数据发送完成
USART_SendData(USART1, ch);    //发送单个字符
```

寄存器方式查询并发送程序如下：

```
while((USART1->SR&0X20)==0);    //等待上一字节数据发送完成
USART1->DR = ch;    //发送单个字符
```

寄存器方式发送函数如下：

```
void sendbyte(u8 ch)
{
```

```

        while((USART1->SR&0X40)==0); //等待前一字符发送完毕
        USART1->DR = ch; //发送单个字符
    }

```

发送字符串函数如下：

```

void sendstring(char *p)
{
    while(*p!='\0') sendbyte(*p++);
}

```

(3) 将接收到的字符原样发送程序

将接收到的字符原样发送程序如下，其中初始化函数 `uart_init` 见上一章：

```

int main(void)
{
    char mss[]="COM1 is ready!\r\n";
    uart_init(115200);
    sendstring(mss);
    while(1)
    {
        sendbyte(receivebyte()); //也可以是 USART1->DR = USART1->DR
    }
}

```

8.3 中断方式接收

串口初始化函数与上一章类似，由于使用 NVIC 中断多出了“④Usart1 NVIC 配置”和“⑤打开串口相应中断”，其中串口相应中断有如下几种：

```

#define USART_IT_PE      ((uint16_t)0x0028)
#define USART_IT_TXE     ((uint16_t)0x0072)
#define USART_IT_TC      ((uint16_t)0x0062)
#define USART_IT_RXNE    ((uint16_t)0x0052)
#define USART_IT_IDLE    ((uint16_t)0x0042)
#define USART_IT_LBD     ((uint16_t)0x0084)
#define USART_IT_CTS     ((uint16_t)0x0096)
#define USART_IT_ERR     ((uint16_t)0x0006)
#define USART_IT_ORE     ((uint16_t)0x0036)
#define USART_IT_NE      ((uint16_t)0x0026)
#define USART_IT_FE      ((uint16_t)0x0016)

```

中断接收一个字符，并原样输出到串口程序如下：

```

#include "stm32f10x.h"
void uart_init(u32 bound)
{
    GPIO_InitTypeDef g1;
    USART_InitTypeDef s1;
    NVIC_InitTypeDef n1;
    //①时钟使能：GPIOA 时钟使能、USART1 串口时钟使能
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1|RCC_APB2Periph_GPIOA, ENABLE);
}

```

```
//②GPIO 端口模式设置          /*USART1_TX  GPIOA.10 初始化*/
g1.GPIO_Pin = GPIO_Pin_9; //PA.9
g1.GPIO_Speed = GPIO_Speed_50MHz;
g1.GPIO_Mode = GPIO_Mode_AF_PP; //复用推挽输出
GPIO_Init(GPIOA, &g1); //初始化 GPIOA.9
/*USART1_RX  GPIOA.10 初始化*/
g1.GPIO_Pin = GPIO_Pin_10; //PA10
g1.GPIO_Mode = GPIO_Mode_IN_FLOATING; //浮空输入
GPIO_Init(GPIOA, &g1); //初始化 GPIOA.10
//③串口参数初始化, 给初始化结构体成员赋值并初始化          /*USART 初始化设置*/
s1.USART_BaudRate = bound; //串口波特率
s1.USART_WordLength = USART_WordLength_8b; //字长为 8 位数据格式
s1.USART_StopBits = USART_StopBits_1; //一个停止位
s1.USART_Parity = USART_Parity_No; //无奇偶校验位
s1.USART_HardwareFlowControl = USART_HardwareFlowControl_None; //无硬件数据流控制
s1.USART_Mode = USART_Mode_Rx | USART_Mode_Tx; //收发模式
USART_Init(USART1, &s1); //初始化串口 1
//④Usart1 NVIC 配置
n1.NVIC_IRQChannel = USART1_IRQn;
n1.NVIC_IRQChannelPreemptionPriority=3; //抢占优先级 3
n1.NVIC_IRQChannelSubPriority = 3; //子优先级 3
n1.NVIC_IRQChannelCmd = ENABLE; //IRQ 通道使能
NVIC_Init(&n1); //根据指定的参数初始化 VIC 寄存器
//⑤打开串口相应中断
USART_ITConfig(USART1, USART_IT_RXNE, ENABLE); //开启串口接收中断
//⑥使能串口
USART_Cmd(USART1, ENABLE); //使能串口 1
}
void USART1_IRQHandler(void) //串口 1 中断服务程序
{
    if(USART_GetITStatus(USART1, USART_IT_RXNE) != RESET) //判断是否是接收中断
    {
        USART1->DR = USART1->DR; //如何理解自己送给自己?
    }
}
int main(void)
{
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); // 设置中断优先级分组 2
    uart_init(115200);
    while(1)
    {
        ; //在此可以做其它事情
    }
}
```

8.4 简单通讯协议格式

前面我们完成了单个字符接收发送, 单个字符就好比英文里的字母, 那么我们如何把字母变成单词、单词如何成句。串口通讯就是通讯的格式(软件协议), 如何判断一个完整的数据包尤为重要。在字符(ASCII 码)通讯中我们常以回车换行(字符‘\r’和‘\n’)表示一个包的结束, 在二进制数据通讯中常以时间停顿一定时间作为数据包的结束。

在 ASCII 码传送中，我们传输字符串作为一个数据包。比如 PC 想通过串口向 STM32 发送信息 I' m a student!，注意其中的空格不是停顿，而是空格的一个 ASCII 码字符，那么我们要发送的字符串应该定义为：char mss[]=" I' m a student!\r\n"，如果我们在 STM32 接收到一个完整的字符串（一个完整的数据包）后再把字符串返回，如向 PC 发送“I have received a string: I' m a student!”，当采用中断接收并判断数据包结束（断句）程序如下：

(1) 中断接收服务函数

```
volatile u8  RX_BUF[257]; //接收缓冲,最大有效字符 256+1 个结束符
volatile u16 RX_STA=0; //接收状态标记, //bit15 收到完成标志('n',0x0a), //bit14 收到完成标志('r',0x0d)
//bit0~7 收到的有效字符字节数
void USART1_IRQHandler(void) //串口 1 中断服务程序
{
    u8 Res;
    if(USART_GetITStatus(USART1, USART_IT_RXNE) != RESET) //接收中断(数据以 0x0d 0x0a 结尾)
    {
        Res=USART_ReceiveData(USART1); //读取接收到的数据
        if((RX_STA&0x8000)==0) //接收未完成
        {
            if(RX_STA&0x4000) //接收到了 0x0d
            {
                if(Res != 0x0a) RX_STA=0; //接收错误,重新开始
                else //收到了'n', 接收完成了
                {
                    RX_STA |= 0x8000;
                    RX_BUF[RX_STA & 0X00FF] = '\0';
                }
            }
            else //还没收到 0X0D
            {
                if(Res == 0x0d) RX_STA |= 0x4000; //收到了'r'
                else
                {
                    RX_BUF[RX_STA++ & 0X00FF]=Res; //收到有效字符
                    if(RX_STA>(REC_LEN-1)) RX_STA=0; //接收数据错误,重新开始接收
                }
            }
        }
    }
}
```

程序中 volatile 说明遇到该变量的语句不能被优化，全局数组 RX_BUF 用于存储接收到的数据，最多 256 个有效字符；全局变量 RX_STA 高八位用于存放状态，低八位是收到的有效字符字节数。这两个全局变量在中断函数中修改，在主程序中使用，主程序中判断 RX_STA 的标志，使用完后需要软件清除这些软件标志。

(2) 主程序判断并处理

在主程序中对收到的数据包加上接收标志“I have received a string:...”返回给 PC，程序如下：

```
if((RX_STA&0xC000) == 0xC000) //中断接收一个完整数据包标志
{
    printf("I have received a string:%s\r\n",RX_BUF);
    RX_STA=0; //清除标志，准备接收下一个数据包
}
```

图 8.1 完整数据包接收发送结果图



当 PC 发生 12345678 后，STM32 返回如上字符。

(3) 如何在字符串中转换整形数据

如上传输中，接收到 `RX_BUF[] = "12345678"`，我们如何把字符串转换为数据存放在变量 `x` 中呢？程序如下：

```
x=0;    //初值一定每次设为 0
i=0;    //初值一定每次设为 0
while(RX_BUF[i]!='\0' && RX_BUF[i]!='\n') x=x*10+RX_BUF[i++]-'0';
该程序段运行以后 x 中就是数据 12345678。
```

那如果收到 `RX_BUF[] = "12345678+12341234"`，想做相应算数运算，那么我们只要按上述操作计算着两个操作数 `x`、`y`，并且把运算符送给字符变量 `fun`，那么通过判断 `fun` 字符转入相应运算就可以了。

8.5 实验任务

- (1) 用查询方式将接收到的单个或多个字符原样转发；
- (2) 用中断接收方式将接收到的单个或多个字符原样转发；
- (3) 用中断接收方式接收以回车换行结束的完整数据包存放到接收数组，数据包接收完后再加上头尾信息转发；
- (4) 串口运算程序设计

十进制数算数逻辑运算+、-、*、\、%、&、|运算命令，要求如下面的③所示。

① 练习：PC 串口输入无符号十进制数到变量 a

如">123\r\n"则 a=123，同时将结果返回 PC 串口

② 练习：PC 串口输入两个用逗号分隔的无符号十进制数到变量 a 和 b

如">123,45\r\n"则 a=123, b=45，同时将结果返回 PC 串口

③ 运算练习：

如 PC 发送"123+4000=?\r\n", 则 STM32 向 PC 串口返回"123+4000=4123\r\n"。

8.6 实验步骤

- ① 画出程序流程图；
- ② 编写源程序；
- ③ 下载调试程序；
- ④ 修改源程序；
- ⑤ 撰写实验报告。

8.7 思考题

- (1) 回答如下思考：如何理解将收到的单字符原样转发语句"USART1->DR = USART1->DR;"中等号两边是同一个寄存器？
- (2) 在完整数据包使用回车换行结束是一种简单通讯格式或叫协议，那么在串口通讯中还有哪些常见的标准软件通讯协议？

9. 实验 9~10—定时器更新中断实验

STM32F10X 有 3 种类型的定时器 (Timer)：高级 (Advance Control) 定时器、通用 (General Purpose)、基本 (Base) 定时器。本章首先掌握定时功能—更新中断。

9.1 实验设备与软件环境

设备：

- ① PC 电脑，至少需要 2 个 type-A 的 USB 插口；
- ② ALIENTEK MiniSTM32 实验套件

环境： window10、keil 5 开发软件、仿真器驱动程序、PC 串口调试软件

9.2 STM32 定时器基本原理

9.2.1 STM32F103 定时器概况

STM32F103RC 系列总共有 8 个定时器，基本情况如下表。

其中

高级定时器：TIM1、TIM8；

通用定时器：TIM2、TIM3、TIM4、TIM5；

基本定时器：TIM6、TIM7。

表 9.1 STM32F10x 系列 CPU 常用定时器基本信息

Pin Count	Part Number	CPU Max. Clock (MHz)	Program Memory (bytes)	RAM (bytes)	FSMC	Timer			Com		
						16-bit General Purpose (IC/OC/PWM)	16-bit Advance Control (IC/OC/PWM)	16-bit Basic	SPI	I2C	USART * +UART
64	STM32F103RB	72	128K	20K		3(12/12/12)	1(4/4/6)		2	2	3
	STM32F103RC	72	256K	48K		4(16/16/16)	2(8/8/12)	2	3	2	3+2
	STM32F103RD	72	384K	64K		4(16/16/16)	2(8/8/12)	2	3	2	3+2
	STM32F103RF	72	768K	96K		10(24/24/24)	2(8/8/12)	2	3	2	3+2
	STM32F103RG	72	1024K	96K		10(24/24/24)	2(8/8/12)	2	3	2	3+2
100	STM32F103V8	72	64K	20K		3(12/12/12)	1(4/4/6)		2	2	3
	STM32F103VC	72	256K	48K	•	4(16/16/16)	2(8/8/12)	2	3	2	3+2
	STM32F103VD	72	384K	64K	•	4(16/16/16)	2(8/8/12)	2	3	2	3+2
	STM32F103VG	72	1024K	96K	•	10(24/24/24)	2(8/8/12)	2	3	2	3+2
144	STM32F103ZC	72	256K	48K	•	4(16/16/16)	2(8/8/12)	2	3	2	3+2
	STM32F103ZE	72	512K	64K	•	4(16/16/16)	2(8/8/12)	2	3	2	3+2
	STM32F103ZG	72	1024K	96K	•	10(24/24/24)	2(8/8/12)	2	3	2	3+2

STM32 的通用 TIMx (TIM2~TIM5) 定时器功能特点包括：

位于低速的 APB1 总线上(APB1)

16 位向上、向下、向上/向下(中心对齐)计数模式，自动装载计数器 (TIMx_CNT)。

16 位可编程(可以实时修改)预分频器(TIMx_PSC)，计数器时钟频率的分频系数 为 1~65535 之间的任意数值。

4 个独立通道 (TIMx_CH1~4)，这些通道具有以下功能：

输入捕获

输出比较

PWM 生成(边缘或中间对齐模式)

单脉冲输出

可使用外部信号 (TIMx_ETR) 控制定时器和定时器互连 (可以用 1 个定时器控制另外一个定时器) 的同步电路。

9.2.2 定时器的计数模式

通用定时器可以向上计数、向下计数、向上向下双向计数模式。

① 向上计数模式：计数器从 0 计数到自动加载值(TIMx_ARR)，然后重新从 0 开始计数并且产生一个计数器溢出事件。

② 向下计数模式：计数器从自动装入的值(TIMx_ARR)开始向下计数到 0，再减 1 溢出时从自动装入的值重新开始，并产生一个计数器向下溢出事件。

③ 中央对齐模式 (向上/向下计数)：计数器从 0 开始计数到自动装入的值-1，产生一个计数器溢出事件，然后向下计数到 1 并且产生一个计数器溢出事件；然后再从 0 开始重新计数。

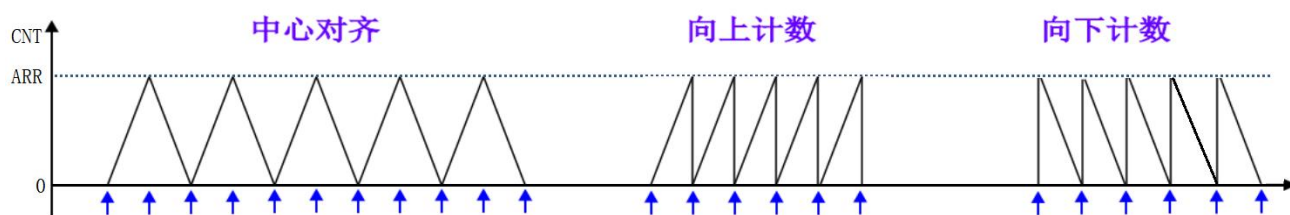


图 9.1 通用定时器计数模式

9.2.3 通用定时器电路结构

(1) 内部时钟

工程模板初始化时，默认调用 SystemInit 函数后各系统时钟情况下：

SYSCLK=72M

AHB 时钟=72M

APB1 时钟=36M

所以 APB1 的分频系数=AHB/APB1 时钟=2

如图 9.2 所示，通用定时器时钟 CK_INT=2*36M=72M，在默认的工程模板中 STM32 总线及定时器时钟配置参数如下，从图中我们可以分析出，进入 TIM1~TIM8 的定时器输入时钟都是 72MHz，经过 PSC=N-1 分频控制寄存器控制后，定时计数器的计数频率 CK_CNT=72MHz/N。

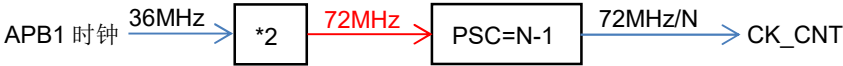


图 9.2 定时器内部时钟结构图

(2) 通用定时器功能结构

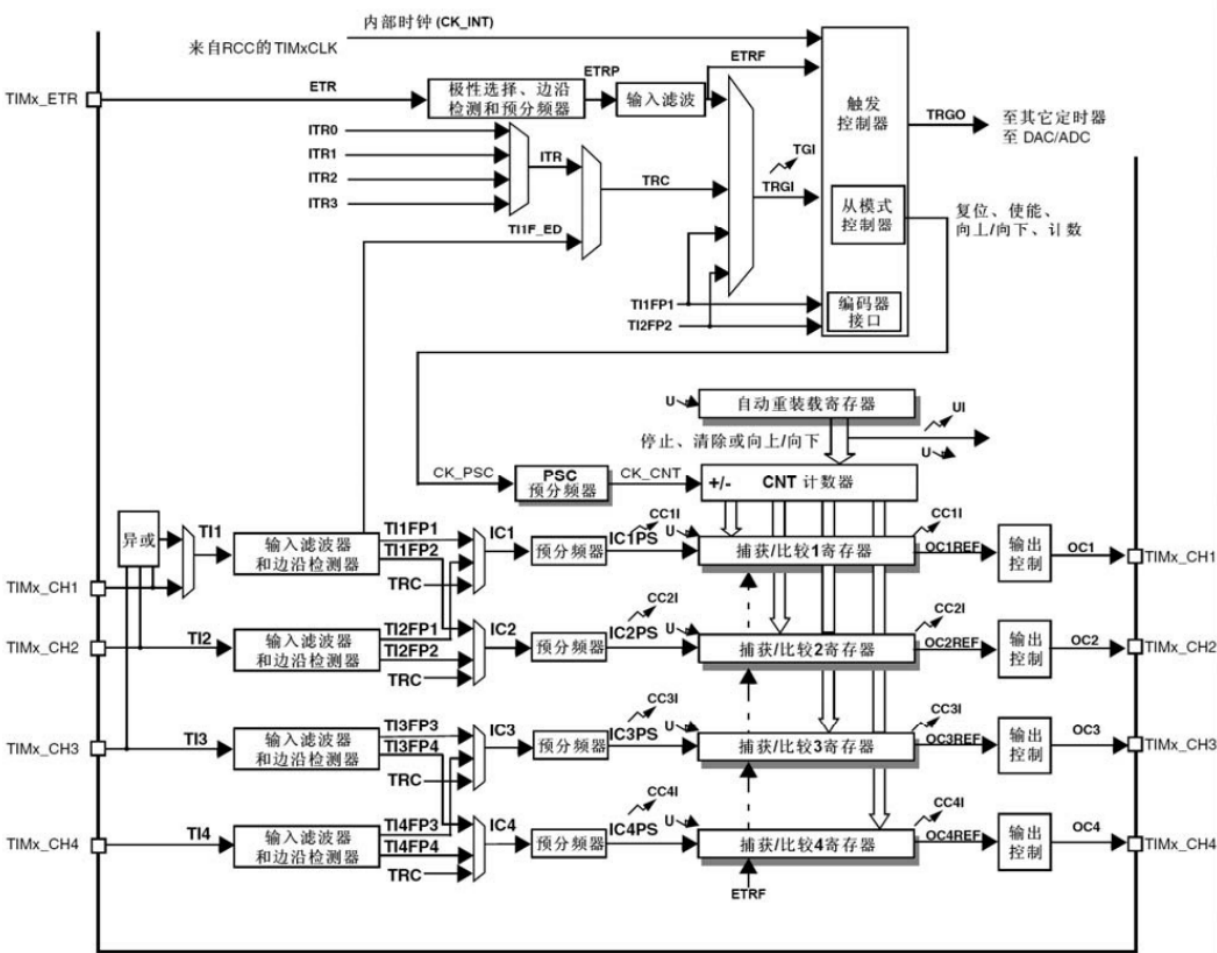


图 9.3 通用定时器功能结构图

如图 9.3 所示，定时/计数器 TIMx 的核心部件是计数单元 CNT，图中 CNT 以上部分是定时器的控制和外部计数输入引脚，其中包括：CR 控制寄存器、PSC 预分频器、ARR 自动重装载寄存器等等。TIMx_ETR 是定时/计数器外部 TIMx_ETR 时钟输入端，程序可控制为定时方式（使用内部时钟 CK_INT 计数），和外部计数方式（使用外部引脚 TIMx_ETR 输入的时钟计数）。

9.3 更新中断程序设计

围绕 STM32F103 每个定时器有 20 寄存器，标准库函数提供了 80 多个标准函数供用户使用，其中常用的如下：

(1) 初始化结构体

```
typedef struct
{
    uint16_t TIM_Prescaler;      /*!< Specifies the prescaler value used to divide the TIM clock.
                                   This parameter can be a number between 0x0000 and 0xFFFF */
    uint16_t TIM_CounterMode;    /*!< Specifies the counter mode.
                                   This parameter can be a value of @ref TIM_Counter_Mode */
    uint16_t TIM_Period;        /*!< Specifies the period value to be loaded into the active
                                   Auto-Reload Register at the next update event.
                                   This parameter must be a number between 0x0000 and 0xFFFF. */
    uint16_t TIM_ClockDivision; /*!< Specifies the clock division.
                                   This parameter can be a value of @ref TIM_Clock_Division_CKD */
    uint8_t TIM_RepetitionCounter; /*!< Specifies the repetition counter value. Each time the RCR
                                   downcounter reaches zero, an update event is generated
                                   and counting restarts from the RCR value (N).
                                   This means in PWM mode that (N+1) corresponds to:
                                   - the number of PWM periods in edge-aligned mode
                                   - the number of half PWM period in center-aligned mode
                                   This parameter must be a number between 0x00 and 0xFF.
                                   @note This parameter is valid only for TIM1 and TIM8. */
} TIM_TimeBaseInitTypeDef;
```

其中：

TIM_Prescaler 是预分频系数，取值范围 0~65535，TIM_Prescaler=0 表示不分频（0+1 分之 1 分频），TIM_Prescaler=n 表示 n+1 分之 1 分频；

TIM_Period 是时间常数，取值范围 0~65535，由于计数是从 0 开始，TIM_Period=m，表示 m+1 次计数时间常数；

TIM_CounterMode 是计数模式，有：向上计数、向下计数、中央对齐（向上/向下计数），库函数中为计数模式定义了如下 5 种常数可供选择，其中后三种为中央对齐计数：

```
#define TIM_CounterMode_Up          ((uint16_t)0x0000)
#define TIM_CounterMode_Down        ((uint16_t)0x0010)
#define TIM_CounterMode_CenterAligned1 ((uint16_t)0x0020)
#define TIM_CounterMode_CenterAligned2 ((uint16_t)0x0040)
#define TIM_CounterMode_CenterAligned3 ((uint16_t)0x0060)
```

TIM_ClockDivision 是进入预分频器 TIM_Prescaler 前的分频系数，有如下 3 种常数可供选择：

```
#define TIM_CKD_DIV1      ((uint16_t)0x0000)
#define TIM_CKD_DIV2      ((uint16_t)0x0100)
#define TIM_CKD_DIV4      ((uint16_t)0x0200)
```

TIM_RepetitionCounter 是重复计数器，高级定时器才有，通用定时器和基本定时器无此功能。

(2) 初始化函数

```
void TIM_TimeBaseInit(TIM_TypeDef* TIMx, TIM_TimeBaseInitTypeDef* TIM_TimeBaseInitStruct);
```

该函数对 TIMx (x 取值范围 1~8) 用上述初始化结构体成员给定参数初始化指定的 TIMx 定时器，注意：调用该初始化函数前需要先调用函数“RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIMx, ENABLE);”使能该 TIMx 定时器时钟；调用初始化函数后需要调用函数“TIM_Cmd(TIMx, ENABLE);”启动指定的 TIMx 定时器；一般情况需要使用中断还需要调用 TIM_ITConfig 函数使能定时器相应的中断和对应的 NVIC 初始化。

(3) 定时器相应的中断使能函数

```
void TIM_ITConfig(TIM_TypeDef* TIMx, uint16_t TIM_IT, FunctionalState NewState);
```

其中：第一个参数是指定哪个 TIMx 定时器；第二个参数指定该 TIMx 定时器的哪个中断功能，中断功能有以下 8 种常数（中断）可供选择；第三个参数有 DISABLE 和 ENABLE 两个常数可供选择。

```
#define TIM_IT_Update      ((uint16_t)0x0001)  //更新中断
#define TIM_IT_CC1        ((uint16_t)0x0002)  //捕获/比较中断 1
#define TIM_IT_CC2        ((uint16_t)0x0004)  //捕获/比较中断 2
#define TIM_IT_CC3        ((uint16_t)0x0008)  //捕获/比较中断 3
#define TIM_IT_CC4        ((uint16_t)0x0010)  //捕获/比较中断 4
#define TIM_IT_COM         ((uint16_t)0x0020)  //仅高级定时器 TIM1 和 TIM8 才有
#define TIM_IT_Trigger     ((uint16_t)0x0040)  //触发事件中断
#define TIM_IT_Break       ((uint16_t)0x0080)  //仅高级定时器 TIM1 和 TIM8 才有
```

(4) 中断服务函数

```
void TIMx_IRQHandler(void)    //TIMx 中断，注意：x 代表的取值为 0~8
{
    if (TIM_GetITStatus(TIMx, TIM_IT_Update) != RESET) //检查更新中断源发生与否
    {
        TIM_ClearITPendingBit(TIMx, TIM_IT_Update); //清除 TIMx 的中断待处理位
        ..... //在此添加中断处理程序
    }
}
```

(5) 常用的定时器初始化函数举例

```
void TIM3_Init(u16 arr,u16 psc)
{
    TIM_TimeBaseInitTypeDef TIM_ini;
    NVIC_InitTypeDef NVIC_ini;

    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM3, ENABLE); //TIM3 时钟使能

    TIM_ini.TIM_Period = arr;    //自动重装载寄存器，设置下一个周期值
    TIM_ini.TIM_Prescaler = psc; //设置用来作为 TIMx 时钟频率除数的预分频值
    TIM_ini.TIM_ClockDivision = TIM_CKD_DIV1;    //设置时钟分割:TDTS = Tck_tim
    TIM_ini.TIM_CounterMode = TIM_CounterMode_Up; //TIM 向上计数模式
```

```

    TIM_TimeBaseInit(TIM3, &TIM_ini); //根据 TIM_TimeBaseInitStruct 中指定的参数初始化
    TIM_ITConfig(TIM3, TIM_IT_Update, ENABLE); //使能 TIM3 中断
    NVIC_ini.NVIC_IRQChannel = TIM3_IRQn; //TIM3 中断
    NVIC_ini.NVIC_IRQChannelPreemptionPriority = 0; //抢占优先级 0 级
    NVIC_ini.NVIC_IRQChannelSubPriority = 3; //响应优先级 3 级
    NVIC_ini.NVIC_IRQChannelCmd = ENABLE; //IRQ 通道被使能
    NVIC_Init(&NVIC_ini); //根据 NVIC_InitStruct 中指定的参数初始化外设 NVIC 寄存器

    TIM_Cmd(TIM3, ENABLE); //启动 TIM3 外设
}

```

9.4 实验任务

- (1) 实验 9：使用定时器定时，让发光二极管 LED1 和 LED2 按照 1Hz 频率定时闪烁；
- (2) 实验 10：题目一数字钟设计

要求：

- ① 通用定时器定时 1ms；
- ② 定义时钟日历结构体，计算年月日星期和时分秒，包括闰年大小月星期计算等；
- ③ 当前日历时钟通过串口每秒送 PC 显示；

选做：

- ④ 设计通讯格式（协议）通过串口修改当前时间；
- ⑤ 当前日历时钟通过 SPI 接口送 MAX7219 驱动数码管显示；
- ⑥ 闹钟功能：设定多组闹钟，闹钟时间到 BEEP 蜂鸣器发出闹钟声音。

9.5 实验步骤

- (1) 画出程序流程图；
- (2) 编写源程序；
- (3) 下载调试程序；
- (4) 修改源程序；
- (5) 撰写实验报告。

9.6 思考题

- (1) 硬件定时与软件延时的区别？
- (2) 用定时器设计的时钟日历有那些误差？有哪些方法可以校正这些误差？

10. 实验 11—PWM 实验

STM32F10X 除了更新中断，还具有输入捕获/输出比较引脚，其功能框图如图 9.3 所示。本章我们完成 PWM 实验，通过 PWM 输出如何控制灯光亮度（调光）。

10.1 实验设备与软件环境

设备：

- ① PC 电脑，至少需要 2 个 type-A 的 USB 插口；
- ② ALIENTEK MiniSTM32 实验套件

环境： window10、keil 5 开发软件、仿真器驱动程序

10.2 PWM 简介

脉冲宽度调制(PWM)，是英文“Pulse Width Modulation”的缩写，简称脉宽调制，是利用微处理器的数字输出来对模拟电路进行控制的一种非常有效的技术。简单一点，就是对脉冲宽度的控制。

PWM 可以由专门的 PWM 电路产生，也可由 CPU 定时器发生。PWM 可用于电机控制、开关电源控制、调光，以及完成 DAC 功能等等。

STM32 的定时器除了基本定时器 TIM6 和 7。其他的定时器都可以用来产生 PWM 输出。其中高级定时器 TIM1 和 TIM8 每个可以同时产生多达 7 路的 PWM 输出。而每个通用定时器也能同时产生多达 4 路的 PWM 输出，这样 STM32 最多可以同时产生 30 路 PWM 输出！本实验仅使用 TIM1 的 CH1 产生一路 PWM 输出控制 LED 灯亮度。

要使 STM32 的高级定时器 TIM1 产生 PWM 输出，除了上一章介绍的几个寄存器 ARR、PSC、CR1 等外，我们还会用到 4 个寄存器（通用定时器则只需要 3 个），来控制 PWM 的输出。这四个寄存器分别是：捕获/比较模式寄存器(TIMx_CCMR1/2)、捕获/比较使能寄存器 (TIMx_CCER)、捕获/比较寄存器(TIMx_CCR1~4) 以及高级定时器才有的刹车和死区寄存器(TIMx_BDTR)。我们使用库函数编程，这四个寄存器如何使用可参见教材或 STM32 芯片说明书。

10.3 PWM 工作过程

PWM 是定时器的一种应用，它的基础是定时器，PWM 的周期由定时器定重载时间寄存器 ARR 决定，占空时间由比较输出寄存器 CCRx 决定，如图所示：

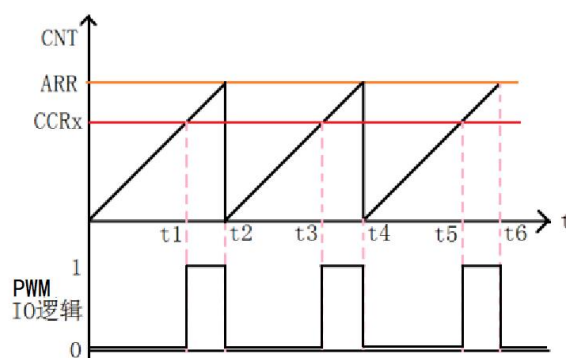


图 10.1 PWM 工作过程

10.4 PWM 程序设计过程

本实验要实现通过 TIM1_CH1 输出 PWM 来控制 DS0 的亮度。下面我们介绍配置步骤：

(1) 开启 TIM1 时钟，配置 PA8 为复用输出

要使用 TIM1，我们必须先开启 TIM1 的时钟，这点相信大家看了这么多代码，应该明白了。这里我们还要配置 PA8 为复用输出(当然还要时能 PORTA 的时钟)，这是因为 TIM1_CH1 通道将使用 PA8 的复用功能作为输出。库函数使能 TIM1 时钟的方法是：

```
RCC_APB1PeriphClockCmd(RCC_APB2Periph_TIM1,ENABLE); //使能定时器 1 时钟
```

设置 PA8 为复用功能输出的方法在前面的几个实验都有类似的讲解，相信大家很明白,这里简单列出 GPIO 初始化的一行代码即可：

```
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP; //复用推挽输出
```

(2) 设置 TIM1 的 ARR 和 PSC

在开启了 TIM1 的时钟之后，我们要设置 ARR 和 PSC 两个寄存器的值来控制输出 PWM 的周期。当 PWM 周期太慢(低于 50Hz)的时候，我们就会明显感觉到闪烁了。因此，PWM 周期在这里不宜设置的太小。这在库函数是通过 TIM_TimeBaseInit 函数实现的，在上一节定时器中断章节我们已经有讲解，这里就不详细讲解，调用的格式为：

```
TIM_TimeBaseStructure.TIM_Period=arr; //设置自动重装载值
TIM_TimeBaseStructure.TIM_Prescaler=psc; //设置预分频值
TIM_TimeBaseStructure.TIM_ClockDivision=0; //设置时钟分割:TDS=Tck_tim
TIM_TimeBaseStructure.TIM_CounterMode=TIM_CounterMode_Up; //向上计数模式
TIM_TimeBaseInit(TIM1,&TIM_TimeBaseStructure); //根据指定的参数初始化 TIMx 的
```

(3) 设置 TIM1_CH1 的 PWM 模式及通道方向，使能 TIM1 的 CH1 输出

接下来，我们要设置 TIM1_CH1 为 PWM 模式(默认是冻结的)，因为我们的 DS0 是低电平亮，而我们希望当 CCR1 的值小的时候，DS0 就暗，CCR1 值大的时候，DS0 就亮，所以我们要通过配置 TIM1_CCMR1 的相关位来控制 TIM1_CH1 的模式。在库函数中，PWM 通道设置是通过函数 TIM_OC1Init()~TIM_OC4Init()来设置的，不同的通道的设置函数不一样，这里我们使用的是通道 1，所以使用的函数是 TIM_OC1Init()。

```
void TIM_OC1Init(TIM_TypeDef* TIMx, TIM_OCInitTypeDef* TIM_OCInitStruct);
```

这种初始化格式大家学到这里应该也熟悉了，所以我们直接来看看结构体 TIM_OCInitTypeDef 的定义：

```
typedef struct
```

```
{
```

```
    uint16_t TIM_OCMode;
    uint16_t TIM_OutputState;
    uint16_t TIM_OutputNState; /*
    uint16_t TIM_Pulse;
    uint16_t TIM_OCPolarity;
    uint16_t TIM_OCNPolarity;
    uint16_t TIM_OCIdleState;
    uint16_t TIM_OCNIIdleState;
```

```
} TIM_OCInitTypeDef;
```

这里我们讲解一下与我们要求相关的几个成员变量：

参数 TIM_OCMode 设置模式是 PWM 还是输出比较，这里我们是 PWM 模式。

参数 TIM_OutputState 用来设置比较输出使能，也就是使能 PWM 输出到端口。

参数 TIM_OCPolarity 用来设置极性是高还是低。

其他的参数 TIM_OutputNState, TIM_OCNPolarity, TIM_OCIdleState 和 TIM_OCNIdleState 是高级定时器 TIM1 和 TIM8 才用到的。

要实现我们上面提到的场景，方法是：

```
TIM_OCInitTypeDef  TIM_OCInitStructure;
TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM2;    //选择 PWM 模式 2
TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable; //比较输出使能
TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_High; //输出极性高
TIM_OC1Init(TIM1, &TIM_OCInitStructure);    //初始化 TIM1 OC1
```

(4) 使能 TIM1

在完成以上设置了之后，我们需要使能 TIM1。使能 TIM1 的方法前面已经讲解过：

```
TIM_Cmd(TIM1, ENABLE);    //使能 TIM1
```

(5) 设置 MOE 输出，使能 PWM 输出

普通定时器在完成以上设置了之后，就可以输出 PWM 了，但是高级定时器，我们还需要使能刹车和死区寄存器(TIM1_BDTR)的 MOE 位，以使能整个 OCx(即 PWM)输出。库函数的设置函数为：

```
TIM_CtrlPWMOutputs(TIM1,ENABLE);//MOE 主输出使能
```

(6) 修改 TIM1_CCR1 来控制占空比

最后，在经过以上设置之后，PWM 其实已经开始输出了，只是其占空比和频率都是固定的，而我们通过修改 TIM1_CCR1 则可以控制 CH1 的输出占空比。继而控制 DS0 的亮度。

在库函数中，修改 TIM1_CCR1 占空比的函数是：

```
void TIM_SetCompare1(TIM_TypeDef* TIMx, uint16_t Compare1);
```

理所当然，对于其他通道，分别有一个函数名字，函数格式为 TIM_SetComparex(x=1,2,3,4)。通过以上 6 个步骤，我们就可以控制 TIM1 的 CH1 输出 PWM 波了。

10.5 PWM 程序

PWM 初始化函数：

```
void TIM1_PWM_Init(u16 arr,u16 psc)
{
    GPIO_InitTypeDef GPIO_InitStructure;
    TIM_TimeBaseInitTypeDef  TIM_TimeBaseStructure;
    TIM_OCInitTypeDef  TIM_OCInitStructure;

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_TIM1, ENABLE);    //①使能 TIM1 时钟
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);    //②使能 GPIO 外设时钟
    //③设置该引脚为复用输出功能,输出 TIM1 CH1 的 PWM 脉冲波形
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_8; //TIM_CH1
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP; //复用推挽输出
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOA, &GPIO_InitStructure);
    //④定时器初始化
    TIM_TimeBaseStructure.TIM_Period = arr; //设置周期
    TIM_TimeBaseStructure.TIM_Prescaler = psc;    //设置用来作为 TIMx 时钟频率除数的预分频值
```

```

TIM_TimeBaseStructure.TIM_ClockDivision = 0; //设置时钟分割:TDTS = Tck_tim
TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;    //向上计数
TIM_TimeBaseInit(TIM1, &TIM_TimeBaseStructure);    //初始化 TIMx
//⑤PWM 初始化
TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM2; //脉宽调制模式 2
TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable; //比较输出使能
TIM_OCInitStructure.TIM_Pulse = 0; //设置待装入捕获比较寄存器的脉冲值
TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_High; //输出极性高
TIM_OC1Init(TIM1, &TIM_OCInitStructure);    //初始化外设 TIMx
TIM_CtrlPWMOutputs(TIM1,ENABLE);    //⑥MOE 主输出使能
TIM_OC1PreloadConfig(TIM1, TIM_OCPreload_Enable);    //CH1 预装载使能
TIM_ARRPreloadConfig(TIM1, ENABLE);    //使能 TIMx 在 ARR 上的预装载寄存器
TIM_Cmd(TIM1, ENABLE);    //⑦使能 TIM1

```

主程序:

```

int main(void)
{
    u16 wide=0; //占空时间
    u8 dir=1;
    LED_Init();
    TIM1_PWM_Init(899,0);    //不分频。PWM 频率=72000/(899+1)=80Khz
    while(1)
    {
        delay_ms(10);
        if(dir)    wide++;
        else        wide--;
        if(wide>300) dir=0;
        if(wide==0) dir=1;
        TIM_SetCompare1(TIM1,wide);
    }
}

```

10.6 实验任务

- (1) 学习 PWM 基本原理及程序设计方法;
- (2) 编写 LED 亮度控制程序;
- (3) 上机调试。

10.7 实验步骤

- (1) 画出程序流程图;
- (2) 编写源程序;
- (3) 下载调试程序;
- (4) 修改源程序;
- (5) 撰写实验报告。

10.8 思考题

定时器输入输出引脚能完成哪些功能? 常用于哪些场景?

11. 实验 12—AD 实验

本章我们完成 ADC 实验。

11.1 实验设备与软件环境

设备：

- ① PC 电脑，至少需要 2 个 type-A 的 USB 插口；
- ② 杜邦线；
- ③ ALIENTEK MiniSTM32 实验套件

环境： window10、keil 5 开发软件、仿真器驱动程序

11.2 ADC 简介

STM32 拥有 1~3 个 ADC (STM32F101/102 系列只有 1 个 ADC)，这些 ADC 可以独立使用，也可以使用双重模式(提高采样率)。STM32 的 ADC 是 12 位逐次逼近型的模拟数字转换器。它有 18 个通道，可测量 16 个外部和 2 个内部信号源。各通道的 A/D 转换可以单次、连续、扫描或间断模式执行。ADC 的结果可以左对齐或右对齐方式存储在 16 位数据寄存器中。模拟看门狗特性允许应用程序检测输入电压是否超出用户定义的高/低阈值。

STM32F103 系列最少都拥有 2 个 ADC，我们选择的 STM32F103RCT 包含有 3 个 ADC。STM32 的 ADC 最大的转换速率为 1Mhz，也就是转换时间为 1us(在 ADCCLK=14M,采样周期为 1.5 个 ADC 时钟下得到)，不要让 ADC 的时钟超过 14M，否则将导致结果准确度下降。

STM32 将 ADC 的转换分为 2 个通道组：规则通道组和注入通道组。规则通道相当于你正常运行的程序，而注入通道呢，就相当于中断。在你程序正常执行的时候，中断是可以打断你的执行的。同这个类似，注入通道的转换可以打断规则通道的转换，在注入通道被转换完成之后，规则通道才得以继续转换。

通过一个形象的例子可以说明：假如你在家里的院子内放了 5 个温度探头，室内放了 3 个温度探头；你需要时刻监视室外温度即可，但偶尔你想看看室内的温度；因此你可以使用规则通道组循环扫描室外的 5 个探头并显示 AD 转换结果，当你想看室内温度时，通过一个按钮启动注入转换组(3 个室内探头)并暂时显示室内温度，当你放开这个按钮后，系统又会回到规则通道组继续检测室外温度。从系统设计上，测量并显示室内温度的过程中断了测量并显示室外温度的过程，但程序设计上可以在初始化阶段分别设置好不同的转换组，系统运行中不必再变更循环转换的配置，从而达到两个任务互不干扰和快速切换的结果。可以设想一下，如果没有规则组和注入组的划分，当你按下按钮后，需要从新配置 AD 循环扫描的通道，然后在释放按钮后需再次配置 AD 循环扫描的通道。

上面的例子因为速度较慢，不能完全体现这样区分(规则通道组和注入通道组)的好处，但在工业应用领域中有很多检测和监视探头需要较快地处理，这样对 AD 转换的分组将简化事件处理的程序并提高事件处理的速度。

STM32 其 ADC 的规则通道组最多包含 16 个转换，而注入通道组最多包含 4 个通道。关于这两个通道组的详细介绍，请参考《STM32 参考手册》的第 155 页，第 11 章。

STM32 的 ADC 可以进行很多种不同的转换模式，这些模式在《STM32 参考手册》的第 11 章也都有详细介绍，我们这里就不在一一列举了。我们本章仅介绍如何使用规则通道的单次转换模式。

STM32 的 ADC 在单次转换模式下,只执行一次转换,该模式可以通过 ADC_CR2 寄存器的 ADON 位(只适用于规则通道)启动,也可以通过外部触发启动(适用于规则通道和注入通道),这是 CONT 位为 0。

以规则通道为例,一旦所选择的通道转换完成,转换结果将被存在 ADC_DR 寄存器中,EOC(转换结束)标志将被置位,如果设置了 EOCIE,则会产生中断。然后 ADC 将停止,直到下次启动。

11.3 ADC 库函数初始化相关定义

(1) ADC 初始化结构体

```
typedef struct
{
    uint32_t ADC_Mode;          /*!< Configures the ADC to operate in independent or
                                dual mode.This parameter can be a value of @ref ADC_mode */
    FunctionalState ADC_ScanConvMode; /*!< Specifies whether the conversion is performed in
                                scan (multichannels) or Single (one channel) mode.
                                This parameter can be set to ENABLE or DISABLE */
    FunctionalState ADC_ContinuousConvMode; /*!< Specifies whether the conversion is performed in
                                Continuous or Single mode.This parameter can be set to ENABLE or DISABLE. */
    uint32_t ADC_ExternalTrigConv; /*!< Defines the external trigger used to start the analog
                                to digital conversion of regular channels. This parameter can be a value of
                                @ref ADC_external_trigger_sources_for_regular_channels_conversion */
    uint32_t ADC_DataAlign; /*!< Specifies whether the ADC data alignment is left or right.
                                This parameter can be a value of @ref ADC_data_align */
    uint8_t ADC_NbrOfChannel; /*!< Specifies the number of ADC channels that will be converted
                                using the sequencer for regular channel group. This parameter must range from 1 to 16. */
}ADC_InitTypeDef;
```

(2) ADC_Mode 选项

```
#define ADC_Mode_Independent      ((uint32_t)0x00000000)
#define ADC_Mode_RegInjecSimult   ((uint32_t)0x00010000)
#define ADC_Mode_RegSimult_AlterTrig ((uint32_t)0x00020000)
#define ADC_Mode_InjecSimult_FastInterl ((uint32_t)0x00030000)
#define ADC_Mode_InjecSimult_SlowInterl ((uint32_t)0x00040000)
#define ADC_Mode_InjecSimult      ((uint32_t)0x00050000)
#define ADC_Mode_RegSimult        ((uint32_t)0x00060000)
#define ADC_Mode_FastInterl       ((uint32_t)0x00070000)
#define ADC_Mode_SlowInterl       ((uint32_t)0x00080000)
#define ADC_Mode_AlterTrig        ((uint32_t)0x00090000)
```

(3) ADC_ScanConvMode 和 ADC_ContinuousConvMode 选项

```
typedef enum {DISABLE = 0, ENABLE = !DISABLE} FunctionalState;
```

(4) ADC_ExternalTrigConv 选项

```
#define ADC_ExternalTrigConv_T1_CC1      ((uint32_t)0x00000000) /*!< For ADC1 and ADC2 */
#define ADC_ExternalTrigConv_T1_CC2      ((uint32_t)0x00020000) /*!< For ADC1 and ADC2 */
#define ADC_ExternalTrigConv_T2_CC2      ((uint32_t)0x00060000) /*!< For ADC1 and ADC2 */
#define ADC_ExternalTrigConv_T3_TRGO     ((uint32_t)0x00080000) /*!< For ADC1 and ADC2 */
#define ADC_ExternalTrigConv_T4_CC4      ((uint32_t)0x000A0000) /*!< For ADC1 and ADC2 */
#define ADC_ExternalTrigConv_Ext_IT11_TIM8_TRGO ((uint32_t)0x000C0000) /*!< For ADC1 and ADC2 */
#define ADC_ExternalTrigConv_T1_CC3      ((uint32_t)0x00040000) /*!< For ADC1, ADC2 and ADC3 */
```

```
#define ADC_ExternalTrigConv_None ((uint32_t)0x000E0000) /*!< For ADC1, ADC2 and ADC3 */
#define ADC_ExternalTrigConv_T3_CC1 ((uint32_t)0x00000000) /*!< For ADC3 only */
#define ADC_ExternalTrigConv_T2_CC3 ((uint32_t)0x00020000) /*!< For ADC3 only */
#define ADC_ExternalTrigConv_T8_CC1 ((uint32_t)0x00060000) /*!< For ADC3 only */
#define ADC_ExternalTrigConv_T8_TRGO ((uint32_t)0x00080000) /*!< For ADC3 only */
#define ADC_ExternalTrigConv_T5_CC1 ((uint32_t)0x000A0000) /*!< For ADC3 only */
#define ADC_ExternalTrigConv_T5_CC3 ((uint32_t)0x000C0000) /*!< For ADC3 only */
```

(5) ADC_DataAlign 选项

```
#define ADC_DataAlign_Right ((uint32_t)0x00000000)
#define ADC_DataAlign_Left ((uint32_t)0x00000800)
```

(6) ADC_NbrOfChannel 选项

该选项设置转换通道数量，取值范围 1~16。

11.4 ADC 程序设计过程

下面讲解其详细设置步骤：

(1) 开启 PA 口和 ADC1 时钟，设置 PA1 为模拟输入

STM32F103RCT6 的 ADC 通道 1 在 PA1 上，所以，我们先要使能 PORTA 的时钟，然后设置 PA1 为模拟输入。使能 GPIOA 和 ADC 时钟用 RCC_APB2PeriphClockCmd 函数，设置 PA1 的输入方式，使用 GPIO_Init 函数即可。这里我们列出 STM32 的 ADC 通道与 GPIO 对应表：

表 11.1 STM32 的 AD 引脚

	ADC1	ADC2	ADC3
通道0	PA0	PA0	PA0
通道1	PA1	PA1	PA1
通道2	PA2	PA2	PA2
通道3	PA3	PA3	PA3
通道4	PA4	PA4	PF6
通道5	PA5	PA5	PF7
通道6	PA6	PA6	PF8
通道7	PA7	PA7	PF9
通道8	PB0	PB0	PF10
通道9	PB1	PB1	
通道10	PC0	PC0	PC0
通道11	PC1	PC1	PC1
通道12	PC2	PC2	PC2
通道13	PC3	PC3	PC3
通道14	PC4	PC4	
通道15	PC5	PC5	
通道16	温度传感器		
通道17	内部参照电压		

(2) 复位 ADC1，同时设置 ADC1 分频因子

开启 ADC1 时钟之后，我们要复位 ADC1，将 ADC1 的全部寄存器重设为缺省值之后我们就可以通过 RCC_CFGR 设置 ADC1 的分频因子。分频因子要确保 ADC1 的时钟(ADCLK)不要超过 14MHz。这个我们设置分频因子位 6，时钟为 $72/6=12\text{MHz}$ ，库函数的实现方法是：

```
RCC_ADCCLKConfig(RCC_PCLK2_Div6);
```

ADC 时钟复位的方法是：

```
ADC_DeInit(ADC1);
```

这个函数非常容易理解，就是复位指定的 ADC。

(3) 初始化 ADC1 参数，设置 ADC1 的工作模式以及规则序列的相关信息

在设置完分频因子之后，我们就可以开始 ADC1 的模式配置了，设置单次转换模式、触发方式选择、数据对齐方式等都在这一步实现。同时，我们还要设置 ADC1 规则序列的相关信息，我们这里只有一个通道，并且是单次转换的，所以设置规则序列中通道数为 1。这些在库函数中是通过函数 ADC_Init 实现的，下面我们看看其定义：

```
void ADC_Init(ADC_TypeDef* ADCx, ADC_InitTypeDef* ADC_InitStruct);
```

从函数定义可以看出，第一个参数是指定 ADC 号。这里我们来看看第二个参数，跟其他外设初始化一样，同样是通过设置结构体成员变量的值来设定参数。

通过上面对每个参数的讲解，下面来看看我们的初始化范例：

```
ADC_InitTypeDef ADC_InitStructure;
ADC_InitStructure.ADC_Mode = ADC_Mode_Independent;    //ADC 工作模式:独立模式
ADC_InitStructure.ADC_ScanConvMode = DISABLE;        //AD 单通道模式
ADC_InitStructure.ADC_ContinuousConvMode = DISABLE;  //AD 单次转换模式
ADC_InitStructure.ADC_ExternalTrigConv = ADC_ExternalTrigConv_None; //转换由软件而非外部触发启动
ADC_InitStructure.ADC_DataAlign = ADC_DataAlign_Right; //ADC 数据右对齐
ADC_InitStructure.ADC_NbrOfChannel = 1;              //顺序进行规则转换的 ADC 通道的数目 1
ADC_Init(ADC1, &ADC_InitStructure);                 //根据指定的参数初始化外设 ADCx
```

(4) 使能 ADC 并校准。

在设置完了以上信息后，我们就使能 AD 转换器，执行复位校准和 AD 校准，注意这两步是必须的！不校准将导致结果很不准确。

使能指定的 ADC 的方法是：

```
ADC_Cmd(ADC1,ENABLE);    //使能指定的 ADC1
```

执行复位校准的方法是：

```
ADC_ResetCalibration(ADC1);
```

执行 ADC 校准的方法是：

```
ADC_StartCalibration(ADC1);    //开始指定 ADC1 的校准状态
```

记住，每次进行校准之后要等待校准结束。这里是通过获取校准状态来判断是否校准是否结束。下面我们一一列出复位校准和 AD 校准的等待结束方法：

```
while(ADC_GetResetCalibrationStatus(ADC1)); //等待复位校准结束
```

```
while(ADC_GetCalibrationStatus(ADC1));    //等待校 AD 准结束
```

(5) 读取 ADC 值

在上面的校准完成之后，ADC 就算准备好了。接下来我们要做的就是设置规则序列 1 里面的通道，采样顺序，以及通道的采样周期，然后启动 ADC 转换。在转换结束后，读取 ADC 转换结果值就是了。这里设置规则序列通道以及采样周期的函数是：

```
void ADC_RegularChannelConfig(ADC_TypeDef* ADCx, uint8_t ADC_Channel, uint8_t Rank, uint8_t ADC_SampleTime);
```

我们这里是规则序列中的第 1 个转换，同时采样周期为 239.5，所以设置为：

```
ADC_RegularChannelConfig(ADC1,ch,1,ADC_SampleTime_239Cycles5);
```

软件开启 ADC 转换的方法是：

```
ADC_SoftwareStartConvCmd(ADC1,ENABLE);//使能指定的 ADC1 的软件转换启动功能
```

开启转换之后，就可以获取转换 ADC 转换结果数据，方法是：

```
ADC_GetConversionValue(ADC1);
```

同时在 AD 转换中，我们还要根据状态寄存器的标志位来获取 AD 转换的各个状态信息。库函数获取 AD 转换的状态信息的函数是：

```
FlagStatus ADC_GetFlagStatus(ADC_TypeDef* ADCx, uint8_t ADC_FLAG);
```

比如我们要判断 ADC1 的转换是否结束，方法是：

```
while(!ADC_GetFlagStatus(ADC1,ADC_FLAG_EOC));//等待转换结束
```

通过以上几个步骤的设置，我们就能正常的使用 STM32 的 ADC1 来执行 AD 转换操作了。

这里还需要说明一下 ADC 的参考电压，MiniSTM32 开发板使用的是 STM32F103RCT6，该芯片没有外部参考电压引脚，ADC 的参考电压直接取自 VDDA，也就是 3.3V。

11.5 ADC 程序

ADC 初始化函数：

```
void    Adc_Init(void)
{
    ADC_InitTypeDef ADC_InitStructure;
    GPIO_InitTypeDef GPIO_InitStructure;

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA |
        RCC_APB2Periph_ADC1, ENABLE );           //使能 ADC1 通道时钟
    RCC_ADCCLKConfig(RCC_PCLK2_Div6); //分频因子 6，72M/6=12,ADC 最大时间不能超过 14MHz
        //PA1 作为模拟通道输入引脚
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_1;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AIN; //模拟输入
    GPIO_Init(GPIOA, &GPIO_InitStructure); //初始化 GPIOA.1

    ADC_DeInit(ADC1); //复位 ADC1,将外设 ADC1 的全部寄存器重设为缺省值
    ADC_InitStructure.ADC_Mode = ADC_Mode_Independent; //ADC 独立模式
    ADC_InitStructure.ADC_ScanConvMode = DISABLE; //单通道模式
    ADC_InitStructure.ADC_ContinuousConvMode = DISABLE; //单次转换模式
    ADC_InitStructure.ADC_ExternalTrigConv = ADC_ExternalTrigConv_None; //转换由
    软件而不是外部触发启动
    ADC_InitStructure.ADC_DataAlign = ADC_DataAlign_Right; //ADC 数据右对齐
    ADC_InitStructure.ADC_NbrOfChannel = 1; //顺序进行规则转换的 ADC 通道的数目
    ADC_Init(ADC1, &ADC_InitStructure); //根据指定的参数初始化外设 ADCx
    ADC_Cmd(ADC1, ENABLE); //使能指定的 ADC1
    ADC_ResetCalibration(ADC1); //开启复位校准
    while(ADC_GetResetCalibrationStatus(ADC1)); //等待复位校准结束
    ADC_StartCalibration(ADC1); //开启 AD 校准
    while(ADC_GetCalibrationStatus(ADC1)); //等待校准结束
}

主程序：
u16 Get_Adc(u8 ch)
{
```

```

//设置指定 ADC 的规则组通道，设置它们的转化顺序和采样时间
ADC-RegularChannelConfig(ADC1, ch, 1, ADC_SampleTime_239Cycles5); //通道 1,规则采样顺序值
    为 1,采样时间为 239.5 周期
ADC_SoftwareStartConvCmd(ADC1, ENABLE); //使能指定的 ADC1 的软件转换功能
while(!ADC_GetFlagStatus(ADC1, ADC_FLAG_EOC)); //等待转换结束
return ADC_GetConversionValue(ADC1); //返回最近一次 ADC1 规则组的转换结果
}
u16 Get_Adc_Average(u8 ch,u8 times)
{
    u32 temp_val=0;
    u8 t;
    for(t=0;t<times;t++)
    {
        temp_val+=Get_Adc(ch);
        delay_ms(5);
    }
    return temp_val/times;
}
int main(void)
{
    u16 adcx;
    float temp;
    delay_init();
    uart_init(9600);
    LED_Init();
    Adc_Init();
    printf("ADC_CH1_VOL:0.000V\n");
    while(1)
    {
        adcx=Get_Adc_Average(ADC_Channel_1,10);
        temp=(float)adcx*(3.3/4096);
        printf("ADC_CH1_VOL:%.3fV\n",temp);
        delay_ms(250);
    }
}

```

11.6 实验任务

- (1) 学习 ADC 基本原理及程序设计方法；
- (2) 编写上述程序；
- (3) 上机调试；
- (4) 修改 Get_Adc_Averag（）函数完成中值滤波、去首尾平均值滤波，并上机调试。
- (5) 选做：给定的 DAC 程序发生了正弦和三角波，用 AD 定时采样完成该低频信号示波器波形显示功能。

11.7 实验步骤

- (1) 画出程序流程图；
- (2) 编写源程序；
- (3) 下载调试程序；
- (4) 修改源程序；
- (5) 撰写实验报告。

11.8 思考题

ADC 有哪些类型？常用于哪些应用场景？

12. 实验 13—SysTick 实验

本章我们完成 SysTick 实验。

12.1 实验设备与软件环境

设备：

- ④ PC 电脑，至少需要 2 个 type-A 的 USB 插口；
- ⑤ 杜邦线；
- ⑥ ALIENTEK MiniSTM32 实验套件

环境： window10、keil 5 开发软件、仿真器驱动程序

12.2 SysTick 简介

STM32 滴答定时器（SysTick）具有以下特点：

- (1) **24 位倒计时计数器**：SysTick 是一个 24 位的向下计数定时器，主要用于简单的延时功能。它从一个初始值开始递减，当计数到 0 时，会自动重新加载初始值并继续计数。
- (2) **系统级定时服务**：SysTick 是 ARM Cortex-M 处理器的一部分，用于提供系统级的定时服务。它可以生成定时中断，用于执行特定的任务或进行系统级的时间跟踪。
- (3) **自动重载功能**：SysTick 具有自动重载功能，当计数器达到 0 时，会自动重新加载预设的初始值并继续计数。这种机制使得 SysTick 能够持续工作，即使在睡眠模式下也不会停止。
- (4) **可配置的时钟源**：SysTick 可以选择使用处理器的系统时钟或外部低速时钟源。通常，SysTick 使用处理器的系统时钟，这样可以实现高精度的定时。
- (5) **异常处理**：SysTick 被捆绑在 NVIC（嵌套向量中断控制器）中，用于产生 SYSTICK 异常。这种设计使得 SysTick 能够在中断处理中实现精确的延时和定时功能。
- (6) **应用场景**：SysTick 常用于操作系统的时钟中断、实时系统的心跳时钟等。由于其简单高效的特点，SysTick 能够节省 MCU 资源，同时提供稳定的时间基准。

12.3 SysTick 库函数初始化相关定义

(1) SysTick 结构体

```
typedef struct
{
    __IO uint32_t CTRL;          /*!< Offset: 0x00 SysTick Control and Status Register */
    __IO uint32_t LOAD;          /*!< Offset: 0x04 SysTick Reload Value Register */
    __IO uint32_t VAL;           /*!< Offset: 0x08 SysTick Current Value Register */
    __IO uint32_t CALIB;         /*!< Offset: 0x0C SysTick Calibration Register */
} SysTick_Type;
```

(2) SysTick 初始化函数原型

```
static __INLINE uint32_t SysTick_Config(uint32_t ticks)
{
```

```

if (ticks > SysTick_LOAD_RELOAD_Msk) return (1); /* Reload value impossible */
SysTick->LOAD = (ticks & SysTick_LOAD_RELOAD_Msk) - 1; /* set reload register */
NVIC_SetPriority (SysTick_IRQn, (1<<__NVIC_PRIO_BITS) - 1);/*set Priority for Cortex-M0 System Interrupts */
SysTick->VAL = 0; /* Load the SysTick Counter Value */
SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
                SysTick_CTRL_TICKINT_Msk |
                SysTick_CTRL_ENABLE_Msk; /* Enable SysTick IRQ and SysTick Timer */
return (0); /* Function successful */
}

```

12.4 SysTick 毫秒定时器延时程序设计过程

下面讲解以 ms 为单位的延时详细设置步骤：

(1) 1ms 定时初始化

SysTick 减计数频率为系统频率，也就是 72MHz，因此 1ms 需要计数 72000 个数字量。初始化程序为：
SysTick_Config(72000uL);

(2) 中断服务函数

```

void SysTick_Handler(void)
{
    if (msDelay) msDelay--;
}

```

中断服务函数对延时毫秒数的全局变量 msDelay 自减 1，直到 0 为止。msDelay=0 表示定时时间到。

(3) 延时实例

比如需要延时 500ms 如下：

```

msDelay=500; //预置 500ms 时间
while(msDelay); //等待延时时间到

```

该延时方式，如果全局变量定义为：
unsigned long msDelay;
则延时时间范围：1~4,294,967,295 ms

12.5 SysTick 微秒定时器延时程序设计过程

下面讲解以 $1/72 \mu s$ 为单位的延时详细设置步骤：

(1) 微秒定时初始化

SysTick 减计数频率为系统频率，也就是 72MHz，因此定时最小时间为 $1/72 \mu s$ ，我们可以设置 x 个 $1/72 \mu s$ 。初始化程序为：

```
SysTick_Config(x); //定时  $x/72 \mu s$ 
```

(2) 中断服务函数

```

void SysTick_Handler(void)
{
    TimeOut=1; //时间到
}

```

中断服务函数设置时间到标志，TimeOut=1。

(3) 延时实例

比如需要延时 100 μ s 如下:

```
SysTick_Config(7200uL);
```

```
TimeOut=0;
```

```
while(TimeOut==0);    //等待延时时间到
```

则理论延时时间范围: 0.014 μ s~233016.875 μ s

12.6 实验任务

- (1) 学习 SysTick 基本原理及程序设计方法;
- (2) 编写上述程序;
- (3) 上机调试;
- (4) 用 SysTick 延时修改发光二极管闪烁程序;
- (5) 用 SysTick 延时修改发声及音乐播放程序。

12.7 实验步骤

- (1) 画出程序流程图;
- (2) 编写源程序;
- (3) 下载调试程序;
- (4) 修改源程序;
- (5) 调试。

12.8 思考题

SysTick 定时延时与软件延时比较有哪些特点? SysTick 常用于哪些应用场景?